



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

Sistema de generación automática de componentes de PC

Autor:

Juan Ramírez Morales
49093416P

Tutores:

Miguel Ángel Olivero González
José Manuel Sánchez Ruiz

Departamento:

Lenguajes y Sistemas Informáticos

Sevilla, Junio 2026

Control de versiones

Versión	Motivo del cambio	Responsable	Fecha
1.0	Creación del documento, portada, índice e introducción.	Juan Ramírez Morales	31/01/2026
1.1	Planificación temporal	Juan Ramírez Morales	08/02/2026
1.2	Gestión de riesgos	Juan Ramírez Morales	10/02/2026
1.3	Estimación de costes	Juan Ramírez Morales	12/02/2026
1.4	Herramientas empleadas	Juan Ramírez Morales	13/02/2026
1.5	Scrum	Juan Ramírez Morales	16/02/2026
1.6	PMBOK	Juan Ramírez Morales	17/02/2026
1.7	Descripción de Design Thinking	Juan Ramírez Morales	18/02/2026
1.7.1	Corrección de la memoria	Juan Ramírez Morales	6/03/2026
1.8	Documentación del desarrollo	Juan Ramírez Morales	15/04/2026
1.8.1	Corrección de la memoria	Juan Ramírez Morales	18/04/2026
1.9	Documentación de Pruebas	Juan Ramírez Morales	01/05/2026
1.9.1	Corrección de la memoria	Juan Ramírez Morales	05/05/2026
1.10	Descripción de Design Thinking	Juan Ramírez Morales	25/05/2026
1.11	Despliegue	Juan Ramírez Morales	27/05/2026
1.12	Gestión del Proyecto	Juan Ramírez Morales	28/05/2026
1.13	Uso de la IA y Conclusiones	Juan Ramírez Morales	29/05/2025
1.14	Desviación de tiempo y costes	Juan Ramírez Morales	31/05/2025
1.14.1	Corrección	Juan Ramírez Morales	02/06/2025

Resumen

En este Trabajo de Fin de Grado se desarrolló un sistema web para la configuración de equipos informáticos. El sistema adoptó el paradigma de Líneas de Productos de Software: se construyó un Modelo de Características, formalizado en UVL y razonado en tiempo real con FlamaPy, que modela la familia completa de configuraciones posibles junto con sus restricciones y dependencias. Sobre esta base, el sistema no solo validó las elecciones del usuario, sino que generó configuraciones completas y equilibradas a partir de un perfil de uso y un presupuesto, repartiendo el gasto entre las distintas categorías de componentes y resolviendo de forma automática las incompatibilidades de hardware.

El sistema se complementó con la interpretación de consultas en lenguaje natural mediante inteligencia artificial. Todo ello se integró en una arquitectura moderna y mantenible, con un backend en FastAPI, un frontend en React, persistencia en MongoDB y el apoyo de Google Gemini para el procesamiento de lenguaje natural. El desarrollo se guió por las metodologías Scrum, PMBOK y Design Thinking. El resultado fue un sistema funcional, desplegado y validado, que cumplió el objetivo de hacer accesible la configuración de un PC a cualquier perfil de usuario.

Agradecimientos

Quiero expresar mi agradecimiento a la Universidad de Sevilla (US) y, en particular, a la Escuela Técnica Superior de Ingeniería Informática (ETSI), por la formación recibida a lo largo de estos años. Hago extensivo este agradecimiento a todos los profesores que me han acompañado durante la carrera.

De manera muy especial, agradezco a mis dos tutores, José Manuel Sánchez Ruiz y Miguel Ángel Olivero González, su dedicación, orientación y apoyo a lo largo del desarrollo de este Trabajo de Fin de Grado. Sus consejos y su seguimiento han sido fundamentales para llevar este proyecto a buen puerto.

Índice general

1. Introducción	10
1.1 Motivación	10
1.2 Objetivos	10
Objetivos técnicos	11
Objetivos académicos	11
1.3 Justificación	11
2. Gestión del proyecto	12
2.1 Planificación temporal del problema	12
2.1.1 Seguimiento y análisis de desviaciones	17
2.1.2 Política de versionado del documento	19
2.2 Estimación de costes y esfuerzo	20
2.2.1 Coste real y análisis de desviación	21
2.3 Gestión de riesgos	22
2.3.1 Identificación de riesgos	22
2.3.2 Evaluación de riesgos	23
2.3.3 Mitigación de riesgos	24
2.3.4 Monitoreo y control	25
3. Metodología	27
3.1 Scrum	27
Planificación de Sprints	27
3.2 PMBOK	28
3.3 Design Thinking	28
4. Herramientas para la realización de actividades	29
5. Fase de investigación	33
5.1 Entendimiento y empatía	34
5.2 Definir	35
5.3 Idear	37
5.4 Decidir	38
5.5 Prototipar	39
5.6 Validar	43
5.7 Retrospectiva	44
6. Análisis y Diseño	45
6.1 Requisitos del sistema	45
6.1.1 Requisitos Funcionales	45
6.1.2 Requisitos No Funcionales	46
6.1.3 Requisitos de Información	46
6.2 Arquitectura del sistema	47
6.3 Diagrama de clases	48
6.4 Diagrama de secuencia	49
6.4.1 Generación automática de configuración	49
6.4.2 Configuración manual paso a paso	50
6.4.3 Consulta en lenguaje natural (Gemini)	51
6.5 Diagrama de actividades	52
7. Modelo de Características	54
7.1 Diseño del modelo	54
7.1.1 Estructura jerárquica	54
7.1.2 Restricciones entre características	56
7.2 Propagación de restricciones	58
8. Implementación	59
8.1 Backend	59
8.1.1 Servicio FlamaPy	59
8.1.2 Catálogo y base de datos	60
8.1.3 Motor de configuración	61

8.1.4 Servicio Gemini	64
8.2 Frontend	65
8.2.1 Flujo del asistente	65
8.2.2 Estado global con Zustand	65
8.2.3 Indicador de gasto	66
8.2.4 Notificaciones educativas	67
8.2.5 Comparador de precios	68
8.2.6 Modificación de configuración generada	68
8.2.7 Exportación de configuraciones	68
8.2.8 Integración con Google Gemini	69
9. Pruebas	71
9.1 Pruebas implementadas y resultados.....	72
9.1.1 Tests del Modelo de Características (test_features.py)	73
9.1.2 Tests del Catálogo (test_productos.py)	74
9.1.3 Tests del Motor de Configuración (test_configuracion.py)	75
9.1.4 Tests del servicio de base de datos (test_db.py)	76
9.1.5 Tests del servicio Gemini (test_gemini.py)	77
9.1.6 Tests E2E de interfaz (e2e/)	77
10. Despliegue.....	79
10.1 Estrategia de entornos.....	79
10.2 Contenerización con Docker	79
10.3 Servicios en Render	80
10.4 Integración y despliegue continuos (CI/CD)	81
10.5 Configuración de CORS	82
10.6 Mecanismo de keep-alive.....	82
11. Gestión del proyecto.....	84
11.1 Estrategia de ramas	84
11.2 Issues y tablero Kanban	85
11.3 Aseguramiento de la calidad	85
12. Uso de la inteligencia artificial	87
13. Conclusiones	88
14. Bibliografía	89

Índice de tablas

Tabla 1: Planificación temporal de la fase de investigación	13
Tabla 2: Planificación temporal de la fase de planificación.....	14
Tabla 3: Planificación temporal de la fase de desarrollo	15
Tabla 4: Planificación temporal de la fase de operaciones	16
Tabla 5: Planificación temporal de la finalización	16
Tabla 6: Descripción Tutor 1	20
Tabla 7: Descripción tutor 2.....	20
Tabla 8: Descripción alumno.....	20
Tabla 9: Precios por trabajador TI.....	20
Tabla 10: Coste estimado proyecto	21
Tabla 11: Coste real proyecto	22
Tabla 12: Comparativa de horas estimadas vs. reales por fase.....	22
Tabla 13: Identificación de riesgos	23
Tabla 14: Matriz de evaluación de riesgos.....	23
Tabla 15: Identificación de riesgos (cuantitativa).....	24
Tabla 16: Estrategias de mitigación de riesgos.....	25
Tabla 17: Planificación de Sprints	27
Tabla 18: Comparativa y justificación de herramientas	32
Tabla 19: Impact Map	36
Tabla 20: Comparativas alternativas del mercado	38
Tabla 21: Requisitos funcionales	46
Tabla 22: Requisitos no funcionales	46
Tabla 23: Requisitos de información	47
Tabla 24: Modelo de características	55
Tabla 25: Clasificación de características.....	58
Tabla 26: Métodos de FlamapyService	60
Tabla 27: Porcentajes de reparto en componentes	63
Tabla 28: Herramientas para la realización de pruebas	71
Tabla 29: Número de pruebas realizadas	72
Tabla 30: Pruebas por clase en test_features.py.....	74
Tabla 31: Pruebas por clase en test_productos.py.....	75
Tabla 32: Pruebas por clase en test_configuracion.py	75
Tabla 33: Pruebas por clase en test_db.py	76
Tabla 34: Pruebas de test_gemini.py.....	77
Tabla 35: Entornos existentes.....	79
Tabla 36: Workflows existentes.....	81

Índice de ilustraciones

Ilustración 1: Diagrama de Gantt (Microsoft Project)	17
Ilustración 2: Planificación estimada vs. real (Microsoft Project)	19
Ilustración 3: Mural Design Thinking	34
Ilustración 4: Story Board	35
Ilustración 5: Story Map.....	37
Ilustración 6: Página de inicio	40
Ilustración 7: Consulta por IA.....	41
Ilustración 8: Configuración automática por perfil de uso y presupuesto	42
Ilustración 9: Configurador paso a paso	42
Ilustración 10: Resumen	43
Ilustración 11: Casos de uso.....	45
Ilustración 12: Arquitectura del sistema.....	47
Ilustración 13: Diagrama de clases	49
Ilustración 14: Diagrama de secuencia (consulta en lenguaje natural).....	52
Ilustración 15: Diagrama de actividades.....	53
Ilustración 16: Modelo de características (representación visual).....	56
Ilustración 17: Diagrama de flujo del algoritmo generar_por_presupuesto()	62
Ilustración 18: Puntos de entrada de la aplicación desde la página de inicio	65
Ilustración 19: Gasto acumulado de los componentes elegidos	67
Ilustración 20: Ejemplo de notificación educativa.....	68
Ilustración 21: Iconos de navegación a PcComponentes o Amazon	68
Ilustración 22: Acciones del paso resumen	68
Ilustración 23: PDF generado tras una configuración.....	69
Ilustración 24: Vista del apartado de la aplicación "Consulta por IA"	70
Ilustración 25: Porcentaje de cobertura en backend	73
Ilustración 26: Pruebas E2E.....	73
Ilustración 27: Estrategia de ramas	84
Ilustración 28: Tablero Kanban en GitHub project	85
Ilustración 29: Análisis de la rama develop (issues/kLoC)	86
Ilustración 30: Número de issues detectados.....	86

Índice de código

Código 1: Método propagate() de propagación de restricciones.....	60
Código 2: Algoritmo generar_por_presupuesto.....	64
Código 3: Llamada a API de Gemini con interpretar_consulta.....	64
Código 4: Lógica de elegirProducto() en Frontend	66
Código 5: Reglas y función que evalúa si un mensaje debe aparecer o no	68
Código 6: Ejemplo de test de propagación	74
Código 7: Ejemplo de test interpretar_consulta	76
Código 8: Verifica que funciona usando el catálogo JSON cuando MongoDB no está disponible.....	76
Código 9: Dockerfile del Backend	79
Código 10: Dockerfile del Frontend	80
Código 11: Workflow del disparador del despliegue de producción en Render	82
Código 12: Configuración de CORS en el archivo main.py	82

1. Introducción

1.1 Motivación

Durante mis años de estudio en la carrera de Ingeniería Informática – Ingeniería del Software, han sido frecuentes las consultas sobre qué ordenador o qué componentes comprar. En más de una ocasión he tenido que ayudar a familiares y amigos a elegir o renovar su equipo, y en todas ellas el proceso seguía el mismo patrón: horas consultando foros, comparando una cantidad abrumadora de especificaciones técnicas y, finalmente, una decisión llena de incertidumbre ante tantas opciones disponibles. Este fenómeno, conocido como el paradigma de *Mass Customization* (Personalización Masiva), genera en el usuario una sobrecarga de información que dificulta la toma de decisiones (Blecker & Abdelkafi, 2006). Estas experiencias me llevaron a plantearme cómo podría abordarse este problema desde una perspectiva formal y realizar este Trabajo de Fin de Grado.

En la actualidad, existen herramientas comerciales para la configuración personalizada de computadores, tales como PCPartPicker¹, Coolmod² o el propio configurador de PcComponentes³. Estas herramientas, por lo general, trabajan bajo un modelo de validación estática (Felfernig, Hotz, Bagley, & Tiihonen, 2014), es decir, permiten al usuario elegir piezas y alertan a posteriori si hay un error de compatibilidad. Sin embargo, este modelo de validación estática presenta una limitación fundamental: presupone que el usuario ya conoce las piezas que necesita. Por ejemplo, un usuario sin conocimientos técnicos no sabe si requiere 32 GB de RAM o qué GPU es suficiente para renderizar vídeo, sólo conoce su caso de uso y su presupuesto. Esta brecha convierte la configuración de un PC en una tarea inaccesible para el usuario medio, obligándole a documentarse extensamente o a delegar la decisión en terceros.

El presente proyecto aborda este problema proponiendo un sistema que no solo valide elecciones del usuario, sino que también tenga la capacidad de generar configuraciones a partir de requisitos de alto nivel y restricciones presupuestarias ("quiero jugar en 4K", "Quiero un PC para gaming y tengo 1000€").

Para ello se adopta el paradigma de Líneas de Productos de Software (*Software Product Lines - SPL*): en lugar de tratar cada configuración como un producto aislado, el sistema modela la familia completa de configuraciones posibles (sus componentes, restricciones y dependencias) de forma que pueda derivar automáticamente configuraciones válidas y óptimas para cada usuario (Benavides, Segura, & Ruiz-Cortés, Automated analysis of feature models 20 years later: A literature review, 2010). Esto no solo asegura que toda configuración generada sea válida y coherente desde el primer momento, sino que traslada al sistema el conocimiento técnico que antes recaía en el usuario.

1.2 Objetivos

El objetivo general de este Trabajo Fin de Grado es desarrollar un sistema de configuración de equipos informáticos basado en el paradigma de Líneas de Productos de Software (SPL). Para alcanzar

¹ <https://pcpartpicker.com/>

² <https://www.coolmod.com/>

³ <https://www.pccomponentes.com/>

esta meta, se establecen los siguientes objetivos específicos, divididos según su naturaleza técnica y académica:

Objetivos técnicos

1. Construir un Modelo de Características (*Feature Model*) –representa la información de todos los productos posibles de una línea de productos de software en términos de características y relaciones entre ellas (Benavides, Segura, & Ruiz-Cortés, Automated analysis of feature models 20 years later: A literature review, 2010)– utilizando herramientas especializadas, como FlamaPy⁴, que sean de utilidad a la hora de trabajar con estos modelos de características.
2. Desarrollar una aplicación web intuitiva que facilite a cualquier perfil de usuario la obtención de un equipo a medida.

Objetivos académicos

1. Profundizar en el uso de tecnologías de desarrollo, así como su configuración e integración.
2. Profundizar en el análisis y resolución de problemas.
3. Demostrar autonomía a la hora de identificar problemas.
4. Demostrar capacidad de aprendizaje sobre los contextos de los problemas, sus soluciones y las técnicas y tecnologías usadas.
5. Elaborar la documentación técnica necesaria para darle soporte a la solución desarrollada.

1.3 Justificación

Se pretende con este proyecto integrar los conocimientos adquiridos a lo largo del grado con nuevas herramientas aprendidas durante la ejecución del Trabajo Fin de Grado. Se busca no solo consolidar metodologías y herramientas ya vistas, sino también profundizar en el aprendizaje autónomo mediante un análisis detallado y riguroso del problema. Todo ello se llevará a cabo bajo la supervisión de los tutores, manteniendo el equilibrio necesario entre el apoyo y la iniciativa propia que requiere un trabajo de estas características.

En el plano metodológico el proyecto retoma Scrum y la guía PMBOK estudiadas en asignaturas como Proyectos Software y Gestión (PSG1 y PSG2), y Planificación y Gestión de Proyectos Informáticos (PGPI), e incorpora Design Thinking como enfoque para la fase de investigación. En el plano técnico, se apoya en asignaturas como Ingeniería de Requisitos (IR) para la elicitación y el modelado de requisitos, en Diseño y Pruebas (DP1 y DP2) para la estrategia de pruebas, o en Evolución y Gestión de la Configuración (EGC) para la gestión del código y la integración continua. Junto a esta base, el TFG exige aprender por cuenta propia tecnologías que quedan fuera del plan de estudios. Todas estas metodologías, herramientas y tecnologías se describirán en detalle en los capítulos posteriores, proporcionando el contexto necesario para justificar las decisiones adoptadas a lo largo del proyecto.

⁴ <https://ide.flamapy.org/>

2. Gestión del proyecto

2.1 Planificación temporal del problema

Para la planificación del Trabajo de Fin de Grado, se ha decidido su división en 5 etapas fundamentales:

1. Fase de investigación: análisis profundo del problema y exploración de alternativas existentes en el mercado para identificar el valor diferencial de la solución.
2. Fase de planificación: tras analizar el problema en profundidad y conocer las alternativas existentes es posible estimar con rigor los recursos, los tiempos y los riesgos del proyecto.
3. Fase de desarrollo: diseño, implementación y pruebas de la solución software, desde el análisis de requisitos hasta la validación en un entorno de preproducción.
4. Fase de operaciones: puesta en marcha y despliegue de la solución en el entorno de producción real.
5. Finalización: consolidación de los resultados obtenidos, cierre de la documentación y preparación de la presentación del proyecto.

Conviene matizar el papel particular que ocupa la fase de investigación dentro de esta planificación. Por ser la primera etapa del proyecto y el punto de partida sobre el que se construye todo lo demás, su duración no se obtiene de una estimación previa, sino de la medición del tiempo realmente invertido en llevarla a cabo (16 horas, según se recoge en la Tabla 1: Planificación temporal de la fase de investigación). Difícilmente puede estimarse con rigor el esfuerzo de comprender un problema antes de haberlo estudiado. Es precisamente el resultado de esta fase el que permite estimar de forma fundamentada la duración del resto de etapas. Así, la fase de investigación queda igualmente planificada y acotada en el cronograma, si bien su duración responde al tiempo medido durante su ejecución y no a una previsión anterior a ella.

Teniendo claras las diferentes etapas, se procede a realizar distintas actividades asociadas a cada una de dichas etapas.

La **fase de investigación** es la primera fase asociada al proyecto. Se caracteriza por la realización de un análisis profundo del problema, su investigación y su repercusión en el proyecto. Buscando entender y dejar claras las ideas que se van a desarrollar en un futuro.

Aplicando las fases de la metodología Design Thinking (detalladas en el apartado 5. Fase de investigación), esta fase se dividirá en 3 actividades:

1. Descubrimiento del problema: esta actividad constituye un punto de inflexión en el proyecto, pues se centra en comprender las necesidades reales de los usuarios. Para ello, se realiza un proceso de investigación sobre los problemas habituales a los que se enfrenta un usuario al configurar un ordenador.
2. Comparación de alternativas: En esta parte del proceso se comparan las distintas alternativas que existen en el mercado. Para cada una de ellas se estudian sus funcionalidades principales y sus

limitaciones. De este modo, permite identificar las carencias comunes de las soluciones actuales y establece un punto de referencia sobre el que construir la propuesta diferencial del proyecto.

3. **Búsqueda de valor añadido:** Tras comparar las alternativas existentes se identifican las funcionalidades que ninguna de ellas ofrece de forma completa y que suponen una mejora real para el usuario.

Tarea Duración

<i>Fase de investigación</i>	16 horas
<i>Descubrimiento del problema</i>	10 horas
<i>Comparación de alternativas</i>	4 horas
<i>Búsqueda de valor añadido</i>	2 horas

Tabla 1: Planificación temporal de la fase de investigación

La **fase de planificación** es la segunda fase asociada al proyecto. Se caracteriza por la organización de los recursos, tiempos y tareas necesarios para alcanzar los objetivos, buscando establecer una hoja de ruta que garantice el correcto desarrollo. Su posición tras la fase de investigación no es casual. Solo una vez analizado el problema en profundidad y conocidas las alternativas existentes es posible estimar con rigor los plazos, los costes y los riesgos del proyecto.

Se dividirá en 6 actividades:

1. **Introducción:** En esta parte se describe la idea general del proyecto, su contexto y el problema que pretende resolver, sentando las bases sobre las que se articula toda la planificación posterior. Se redacta la motivación del proyecto, definen los objetivos técnicos y académicos, y se elabora la justificación que proporciona viabilidad e interés.
2. **Planificación temporal:** Se definen las fechas de inicio y fin de cada actividad, así como su duración estimada. Las horas asignadas a esta parte incluyen la identificación y secuenciación de todas las tareas del proyecto, la elaboración del Diagrama de Gantt y el establecimiento de los hitos principales que marcan el progreso entre fases.
3. **Estimación de costes:** Consiste en el cálculo detallado de los recursos financieros necesarios para llevar a cabo el proyecto, con el fin de establecer un presupuesto realista. Las horas destinadas a esta actividad comprenden la identificación de los perfiles profesionales involucrados, la consulta de tarifas de referencia del sector TIC, el desglose de costes y la elaboración del resumen económico final.
4. **Análisis de riesgos:** Identificación y evaluación de los posibles eventos negativos que podrían comprometer el desarrollo o el resultado del proyecto, diseñando planes de contingencia para

mitigar su impacto o probabilidad. Las horas asignadas se distribuyen entre la identificación de los riesgos más relevantes, su evaluación, la definición de las medidas de mitigación y el establecimiento del plan de monitoreo y control a lo largo del proyecto.

5. Metodologías: Definición del marco de trabajo y los procesos de gestión que se seguirán para organizar el flujo de trabajo a lo largo del proyecto. Se recoge el estudio y la justificación de las metodologías seleccionadas, así como la descripción de cómo se combinan y aplican en el contexto concreto de este TFG.
6. Herramientas: Selección y justificación del software y los recursos tecnológicos que se utilizarán durante el proyecto. Se investigan las alternativas disponibles para cada necesidad y la argumentación de por qué las herramientas elegidas se adecúan a los objetivos y restricciones del proyecto.

Tarea Duración

<i>Fase de planificación</i>	39 horas
<i>Introducción</i>	4 horas
<i>Planificación temporal</i>	8 horas
<i>Estimación de costes</i>	6 horas
<i>Análisis de riesgos</i>	8 horas
<i>Metodologías</i>	5 horas
<i>Herramientas</i>	8 horas

Tabla 2: Planificación temporal de la fase de planificación

La **fase de desarrollo** es la tercera fase asociada al proyecto. Se ejecutarán las tareas planificadas para materializar la solución. Se centra en la creación y la validación del producto.

Se dividirá en 4 actividades:

1. Análisis y diseño: Se describen los requisitos del sistema y se transforman en especificaciones técnicas que guían la implementación. Las horas asignadas a esta parte abarcan la definición de los requisitos, la elaboración de los diagramas y el diseño del modelo de características que representa las opciones de configuración del sistema y sus restricciones.

2. **Automatización:** Consiste en el diseño e implementación de procesos automáticos (como pipelines de CI/CD) para agilizar las tareas repetitivas, reducir riesgo de errores y optimizar el flujo de trabajo. Las horas dedicadas a esta actividad recogen la configuración del repositorio y las ramas de trabajo, la definición de los flujos de integración continua y la preparación del entorno de despliegue para facilitar la transición a producción.
3. **Construcción:** Desarrollo de los artefactos software del sistema. Esto incluye la programación del backend, la creación del frontend web interactivo y la implementación de la base de datos. Es la actividad más extensa, lo que refleja la complejidad de combinar todos estos aspectos.
4. **Pruebas y preproducción:** Consiste en la elaboración de pruebas software para verificar el correcto funcionamiento del sistema y en el despliegue del resultado en un entorno controlado previo a producción.

Tarea Duración

<i>Fase de desarrollo</i>	195 horas
<i>Análisis y diseño</i>	25 horas
<i>Automatización</i>	20 horas
<i>Construcción</i>	120 horas
<i>Pruebas y preproducción</i>	30 horas

Tabla 3: Planificación temporal de la fase de desarrollo

La **fase de operaciones** es la cuarta fase asociada al proyecto. Representa la etapa de puesta en marcha del proyecto, abarcando el proceso de despliegue de la solución tecnológica en un entorno de producción. Constará de la siguiente actividad:

- **Despliegue:** Consiste en el traslado de la solución desde el entorno de desarrollo al entorno de producción real. Las horas asignadas a esta actividad recogen la configuración del servidor de producción, el despliegue y la verificación del correcto funcionamiento del sistema en el entorno definitivo.

Tarea Duración

<i>Fase de operaciones</i>	10 horas
----------------------------	-----------------

Despliegue | 10 horas

Tabla 4: Planificación temporal de la fase de operaciones

La **finalización** es la última fase del proyecto. Su objetivo es consolidar la información y ultimar el trabajo.

Se dividirá en 3 actividades:

1. **Análisis de los resultados:** Se examinan los datos obtenidos comparándolos con los objetivos iniciales. Las horas asignadas a esta parte recogen la revisión de los costes y tiempos estimados frente a los reales, la extracción de conclusiones sobre las decisiones tomadas a lo largo del proyecto y la redacción de las mejoras que podrían abordarse en trabajos futuros.
2. **Finalización de documentación:** Asegura que toda la información esté actualizada, estructurada y sea comprensible. Las horas dedicadas a esta actividad comprenden la revisión y corrección de todos los apartados del documento.
3. **Presentación:** Plasma el proyecto en una presentación visual con el objetivo de su defensa frente al tribunal. Las horas asignadas a esta actividad incluyen la selección y síntesis de los contenidos más relevantes del proyecto, el diseño de las diapositivas, la preparación del discurso y los ensayos necesarios para lograr una exposición clara y fluida dentro del tiempo establecido para la defensa.

Tarea Duración

<i>Finalización</i>	40 horas
<i>Análisis de los resultados</i>	15 horas
<i>Finalización de documentación</i>	10 horas
<i>Presentación</i>	15 horas

Tabla 5: Planificación temporal de la finalización

Para más claridad, se ha realizado un Diagrama de Gantt en el que se representan visualmente las fases del proyecto:

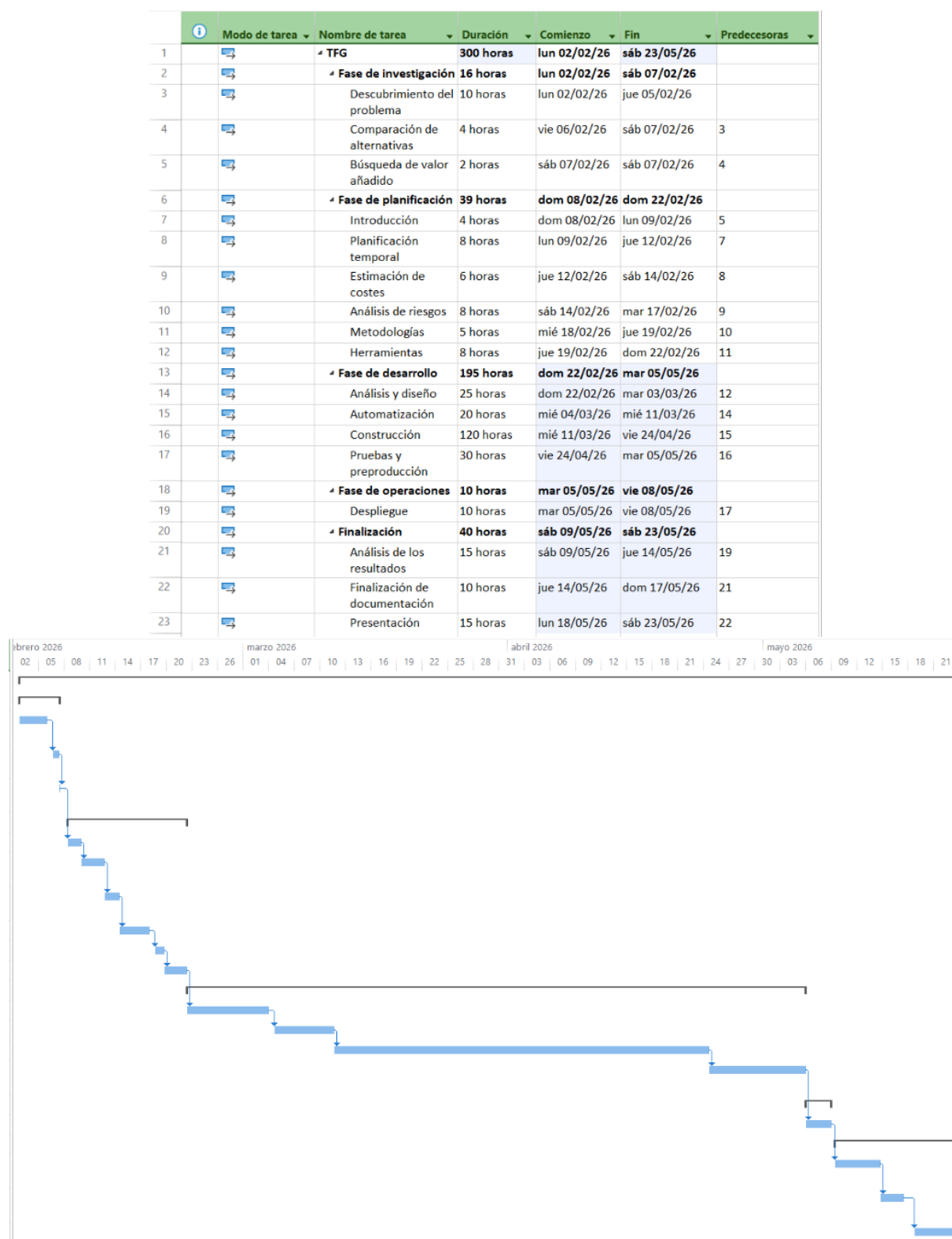


Ilustración 1: Diagrama de Gantt (Microsoft Project)

2.1.1 Seguimiento y análisis de desviaciones

Una vez finalizado el proyecto, se ha contrastado la planificación estimada con su ejecución real, con el fin de identificar las desviaciones producidas y analizar sus causas. A continuación, se exponen las diferencias observadas, tanto en el plazo de entrega como en la distribución del esfuerzo entre las distintas fases.

Cabe señalar que la planificación temporal inicial se elaboró tomando como referencia la entrega en la primera convocatoria, fijada para el 23 de mayo de 2026. Finalmente, el proyecto se presentó en la segunda convocatoria, el 17 de junio de 2026. Esta desviación se debió, principalmente, a la necesidad de compaginar el desarrollo del Trabajo de Fin de Grado con otras tareas y responsabilidades académicas, lo que redujo el ritmo de trabajo en determinadas fases y obligó a distribuir el esfuerzo a lo largo de un periodo de tiempo mayor al previsto. Es importante destacar que esta desviación afectó al plazo de entrega, pero no al esfuerzo total invertido, que se mantuvo en torno a las horas estimadas inicialmente. Por tanto, la diferencia se concentra en el tiempo natural de calendario y no en el número de horas de trabajo ni en el coste del proyecto.

		Modo de tarea	Nombre de tarea	Duración	Duración real	Comienzo	Fin	Predecesoras
1	✓		TFG	307 horas	307 horas	lun 02/02/26	mar 26/05/26	
2	✓		Fase de investigación	16 horas	16 horas	lun 02/02/26	sáb 07/02/26	
3	✓		Descubrimiento del problema	10 horas	10 horas	lun 02/02/26	jue 05/02/26	
4	✓		Comparación de alternativas	4 horas	4 horas	vie 06/02/26	sáb 07/02/26	3
5	✓		Búsqueda de valor añadido	2 horas	2 horas	sáb 07/02/26	sáb 07/02/26	4
6	✓		Fase de planificación	36 horas	36 horas	dom 08/02/26	sáb 21/02/26	
7	✓		Introducción	6 horas	6 horas	dom 08/02/26	mar 10/02/26	5
8	✓		Planificación temporal	10 horas	10 horas	mar 10/02/26	vie 13/02/26	7
9	✓		Estimación de costes	5 horas	5 horas	sáb 14/02/26	dom 15/02/26	8
10	✓		Análisis de riesgos	6 horas	6 horas	dom 15/02/26	mié 18/02/26	9
11	✓		Metodologías	5 horas	5 horas	mié 18/02/26	jue 19/02/26	10
12	✓		Herramientas	4 horas	4 horas	vie 20/02/26	sáb 21/02/26	11
13	✓		Fase de desarrollo	160 horas	160 horas	sáb 21/02/26	mar 21/04/26	
14	✓		Análisis y diseño	35 horas	35 horas	sáb 21/02/26	vie 06/03/26	12
15	✓		Automatización	10 horas	10 horas	vie 06/03/26	lun 09/03/26	14
16	✓		Construcción	100 horas	100 horas	mar 10/03/26	mié 15/04/26	15
17	✓		Pruebas y preproducción	15 horas	15 horas	jue 16/04/26	mar 21/04/26	16
18	✓		Fase de operaciones	15 horas	15 horas	mar 21/04/26	dom 26/04/26	
19	✓		Despliegue	15 horas	15 horas	mar 21/04/26	dom 26/04/26	17
20	✓		Finalización	80 horas	80 horas	dom 26/04/26	mar 26/05/26	
21	✓		Análisis de los resultados	25 horas	25 horas	dom 26/04/26	mié 06/05/26	19
22	✓		Finalización de documentación	31 horas	31 horas	mié 06/05/26	dom 17/05/26	21
23	✓		Presentación	24 horas	24 horas	dom 17/05/26	mar 26/05/26	22

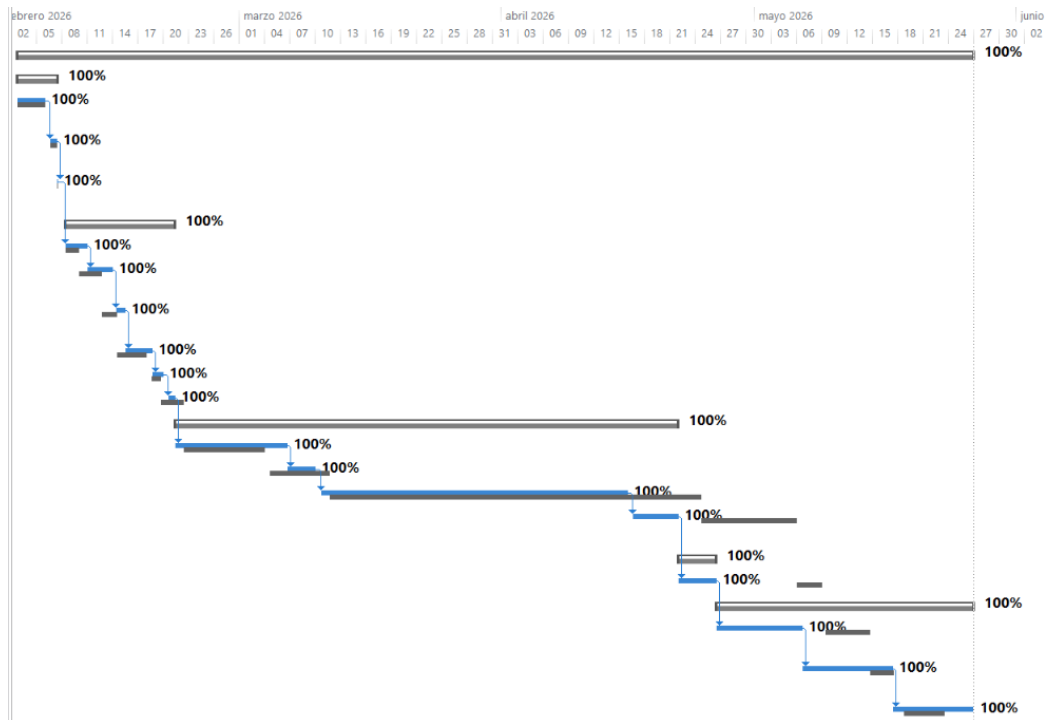


Ilustración 2: Planificación estimada vs. real (Microsoft Project)

En la Ilustración 2 se representa, para cada tarea, una doble barra: la barra inferior en gris corresponde a la planificación estimada, mientras que la barra superior en color refleja la ejecución real. La comparación entre ambas permite apreciar visualmente las desviaciones por fase: la fase de desarrollo se completó en menos tiempo del previsto, mientras que la fase de finalización se prolongó de forma notable respecto a la estimación inicial. En conjunto, el esfuerzo total apenas varía (307 horas reales frente a las 300 estimadas), si bien su distribución a lo largo del calendario se desplazó hacia los meses finales del proyecto, en coherencia con el cambio de la primera a la segunda convocatoria.

2.1.2 Política de versionado del documento

El control de versiones recogido al inicio del documento sigue un esquema de numeración de tres niveles (X.Y.Z), inspirado en el versionado semántico, en el que cada dígito refleja la naturaleza del cambio realizado.

Versión mayor (X): el primer dígito se incrementa, reiniciando los dígitos siguientes a cero (por ejemplo, de 1.13 a 2.0), únicamente ante un hito que cierra una versión completa y autocontenida del documento. En este proyecto, dicho salto se reserva para la entrega formal de la memoria de cara a su defensa, momento en el que el documento se considera consolidado en su totalidad.

Versión menor (Y): el segundo dígito se incrementa (por ejemplo, de 1.7 a 1.8) cuando se incorpora un nuevo contenido sustancial al documento, como la redacción de un apartado completo o la adición de una sección no existente hasta ese momento. Es la operación más habitual a lo largo del desarrollo de la memoria e indica un avance significativo, pero sin cerrar todavía una versión completa.

Versión de revisión (Z): el tercer dígito se incrementa (por ejemplo, de 1.7 a 1.7.1) cuando se realiza una corrección o refinamiento menor sobre un contenido ya existente, sin añadir apartados nuevos. Engloba la corrección de errores, la mejora de la redacción o pequeños ajustes derivados de la revisión de la memoria.

De este modo, la simple lectura del número de versión permite identificar de forma inmediata el alcance de cada cambio: un nuevo apartado, una corrección puntual o el cierre de una entrega completa del documento.

2.2 Estimación de costes y esfuerzo

La asignatura Trabajo de Fin de Grado equivale a 12 créditos ECTS en el grado Ingeniería Informática – Ingeniería del Software de la Universidad de Sevilla, por lo que se invertirán 300 horas para el desarrollo y realización del propio trabajo, así como tiempo en reuniones y procesos de estimación del mismo. También, se ha considerado que el tiempo empleado por los tutores será de aproximadamente 20 horas.

Las personas que van a intervenir en el proyecto son:

Nombre	Miguel Ángel Olivero González
Correo electrónico	molivero@us.es
Cargo	Tutor del Trabajo de Fin de Grado
Función	Dirigir y supervisar el proyecto

Tabla 6: Descripción Tutor 1

Nombre	José Manuel Sánchez Ruiz
Correo electrónico	jsanchez7@us.es
Cargo	Tutor del Trabajo de Fin de Grado
Función	Dirigir y supervisar el proyecto

Tabla 7: Descripción tutor 2

Nombre	Juan Ramírez Morales
Correo electrónico	juarammor2@alum.us.es
Cargo	Desarrollador del Trabajo de Fin de Grado
Función	Desarrollo y documentación del proyecto

Tabla 8: Descripción alumno

El coste del proyecto se ha calculado en función de las 300 horas, más las 20 empleadas por los tutores. Tras suponer que el proyecto se realiza en dicho período de tiempo, se ha procedido al cálculo del coste del proyecto en función de las horas reales necesitadas.

Para calcular el coste de la forma más cercana posible, se ha tenido en cuenta el coste medio por hora de trabajo de los distintos tipos de trabajadores dentro del sector de la informática. Según la información de la Junta de Andalucía, este es el coste por hora según el tipo de trabajador asociado a proyectos informáticos (Junta de Andalucía, 2023):

Director	93,80 euros
Jefe de proyecto	56,65 euros
Analista	45,30 euros
Desarrollador	35,80 euros
Desarrollador Junior	28,64 euros

Tabla 9: Precios por trabajador TI

Este Trabajo de Fin de Grado es un trabajo en el que para la realización y desarrollo del mismo va a intervenir una sola persona. En la fase inicial de Investigación, asumirá el rol de analista, encargado del descubrimiento del problema, la búsqueda de valor añadido y la comparación de alternativas. Durante la fase de planificación, el alumno asume el rol de jefe de proyecto para la gestión de riesgos, estimación de costes y planificación temporal. Durante las fases de desarrollo y operaciones, el alumno pasa al rol de desarrollador junior. Y al finalizar, vuelve a actuar como analista realizando el análisis de los resultados obtenidos, finalizando la documentación y preparando la presentación del proyecto.

A su vez, el proyecto cuenta con dos tutores, previamente mencionados, que desempeñarán de forma conjunta el rol único de director realizando tareas de supervisión, validación de hitos y apoyo estratégico⁵ (ver apartado 3.2 PMBOK).

Por ello, se han calculado los costes estimados para la realización del proyecto por parte de dichas personas y sus recursos, con el resultado:

Persona o Recurso	Tiempo (Horas)	Coste/hora (euros)	Coste total (euros)
Analista	56	45,30	2536,8
Jefe de proyecto	39	56,65	2209,35
Desarrollador Junior	205	28,64	5871,2
Director	20	93,80	1876
TOTAL			12493,35

Tabla 10: Coste estimado proyecto

2.2.1 Coste real y análisis de desviación

Finalizado el proyecto, se contrasta el coste estimado con el coste real derivado de las horas invertidas, con el objetivo de cuantificar la desviación económica y explicar sus causas.

El esfuerzo total real ascendió a 307 horas, muy próximo a las 300 horas previstas (una desviación de tan solo el 2,3 %). Sin embargo, esa cifra global esconde una redistribución relevante entre fases: la fase de desarrollo requirió menos horas de las estimadas (160 frente a 195, un 17,9 % menos), mientras que la fase de finalización duplicó su duración prevista (80 horas frente a 40), debido fundamentalmente al mayor esfuerzo dedicado a la documentación y a la preparación de la presentación. Aplicando las mismas tarifas y roles considerados en la estimación inicial, el coste real del proyecto asciende a 13.276,20 euros.

Persona o Recurso	Tiempo (Horas)	Coste/hora (euros)	Coste total (euros)
Analista	96	45,30	4348,80
Jefe de proyecto	36	56,65	2039,40
Desarrollador Junior	175	28,64	5012,00
Director	20	93,80	1876,00

⁵ Los perfiles profesionales empleados (analista, jefe de proyecto, desarrollador y director) corresponden a las categorías laborales del sector TIC recogidas en la tabla de tarifas de la Junta de Andalucía. Si bien algunos de estos perfiles guardan correspondencia con roles descritos en el estándar PMBOK (por ejemplo, el jefe de proyecto con el Project Manager), su denominación y tarifa proceden de dicha clasificación de la Junta de Andalucía y no de los roles de PMBOK. En particular, la categoría “Director” se aplica al rol que desempeñan los tutores en atención a su nivel de responsabilidad y a sus funciones de supervisión, validación de hitos y apoyo estratégico. La metodología PMBOK se emplea para estructurar la gestión de costes (estimación del presupuesto por perfiles profesionales y control de desviaciones), no para la asignación nominal de roles.

TOTAL			13276,20
--------------	--	--	-----------------

Tabla 11: Coste real proyecto

En consecuencia, la desviación de coste entre la planificación estimada (12.493,35 euros) y los valores reales (13.276,20 euros) es del 6,27 %. Resulta llamativo que esta desviación de coste sea superior a la desviación de esfuerzo (2,3 %): ello se debe a que las horas adicionales se concentraron en la fase de finalización, desempeñada bajo el rol de analista (45,30 euros/hora), mientras que las horas ahorradas correspondían a la fase de desarrollo, más económica al realizarse bajo el rol de desarrollador junior (28,64 euros/hora). Conviene señalar, además, que la principal desviación del proyecto no se produjo en el número total de horas, sino en el plazo de entrega: la planificación inicial se elaboró para la primera convocatoria y, finalmente, el proyecto se presentó en la segunda, tal y como se detalla en el apartado 2.1.1 Seguimiento y análisis de desviaciones.

Fase	Horas estimadas	Horas reales	Desviación
Investigación	16	16	0,0 %
Planificación	39	36	-7,7 %
Desarrollo	195	160	-17,9 %
Operaciones	10	15	+50,0 %
Finalización	40	80	+100,0 %
TOTAL	300	307	+2,3 %

Tabla 12: Comparativa de horas estimadas vs. reales por fase

2.3 Gestión de riesgos

En esta sección se tratará de identificar, analizar y evaluar eventos que puedan afectar al proyecto. Su objetivo es diseñar planes de respuesta y contingencia que reduzcan impactos negativos y aseguren que el plan se cumpla con éxito.

2.3.1 Identificación de riesgos

A continuación, se realizará la identificación de los factores que podrían comprometer el éxito del proyecto. Para ello, se presenta una matriz de riesgos, donde cada elemento se evalúa en función de su probabilidad de ocurrencia y su impacto estimado sobre el alcance, los costes y el tiempo (Project Management Institute, 2017).

De modo que tanto la probabilidad como el impacto podrán tomar tres valores: Alto, Medio o Bajo.

ID	Riesgo	Impacto			Probabilidad
		Alcance	Costes	Tiempo	
1	Desconocimiento de la dificultad del proyecto.	Alto	Alto	Alto	Alta
2	Entregas fuera del plazo definido.	Alto	Alto	Alto	Media
3	Desconocimiento de nuevas tecnologías.	Bajo	Bajo	Alto	Alta
4	Cambios en las herramientas tecnológicas utilizadas.	Bajo	Medio	Alto	Alta
5	Cambios en los requisitos del proyecto durante/tras la fase de desarrollo.	Alto	Medio	Alto	Media
6	Requisitos mal definidos.	Alto	Medio	Medio	Media

7	Falta de comunicación con los tutores del proyecto.	Medio	Bajo	Medio	Media
8	Falta de implicación por el alumno que lleva a cabo el proyecto.	Alto	Alto	Alto	Media
9	Falta de estandarización en los datos de los componentes.	Bajo	Bajo	Medio	Media
10	Alta complejidad en el modelado de características.	Bajo	Bajo	Alto	Media
11	Desviación del alcance por funcionalidades complejas.	Medio	Bajo	Bajo	Alta
12	Problemas de rendimiento debido al algoritmo de selección.	Bajo	Bajo	Alto	Media
13	Insuficiencia de pruebas en el código.	Bajo	Medio	Alto	Media
14	Documentación mal estructurada o poco comprensible.	Bajo	Bajo	Alto	Baja
15	Documentación con insuficientes apartados.	Bajo	Bajo	Alto	Baja

Tabla 13: Identificación de riesgos

2.3.2 Evaluación de riesgos

Tras identificar los posibles riesgos, se procede a su evaluación cuantitativa para determinar su nivel de prioridad. Siguiendo el modelo de matriz de probabilidad e impacto propuesto por el PMBOK (Project Management Institute, 2017), se ha definido una escala numérica propia que traduce las valoraciones cualitativas a valores cuantitativos comparables. La siguiente tabla presenta la matriz de evaluación, donde se cruzan los valores asignados al impacto (donde Bajo es igual a 2, Medio a 5 y Alto a 8) y la probabilidad (donde Baja es igual al 20%, Media a 50% y Alta a 80%) para obtener un índice de riesgo que permite clasificar la prioridad de estos en baja (verde), media (amarillo) o alta (rojo).

La prioridad se calculará como:

$$Prioridad = Probabilidad \times \left(\frac{Impacto_{alcance} + Impacto_{costes} + Impacto_{tiempo}}{3} \right)$$

Impacto	8	1,6	4,0	6,4
	5	1	2,5	4,0
	2	0,4	1,0	1,6
		20%	50%	80%
		Probabilidad		

Tabla 14: Matriz de evaluación de riesgos

Se considera que un riesgo tiene una prioridad alta si su valor es mayor o igual que 4. Su prioridad es media si su valor se encuentra entre 1 y 4 (sin incluirlos), y su prioridad es baja si su valor es menor o igual que 1. De este modo, los riesgos se clasificarían de la siguiente forma según su prioridad:

ID	Riesgo	Prioridad
1	Desconocimiento de la dificultad del proyecto.	6,4
2	Entregas fuera del plazo definido.	4
3	Desconocimiento de nuevas tecnologías.	3,2
4	Cambios en las herramientas tecnológicas utilizadas.	4
5	Cambios en los requisitos del proyecto durante/tras la fase de desarrollo.	3,5
6	Requisitos mal definidos.	3
7	Falta de comunicación con los tutores del proyecto.	2
8	Falta de implicación por el alumno que lleva a cabo el proyecto.	4
9	Falta de estandarización en los datos de los componentes.	1,5
10	Alta complejidad en el modelado de características.	2
11	Desviación del alcance por funcionalidades complejas.	2,4
12	Problemas de rendimiento debido al algoritmo de selección.	2
13	Insuficiencia de pruebas en el código.	2,5
14	Documentación mal estructurada o poco comprensible.	0,8
15	Documentación con insuficientes apartados.	0,8

Tabla 15: Identificación de riesgos (cuantitativa)

Destacan especialmente el ID 1 con un valor de 6,4 y los ID 2, 4 y 8 con un valor de 4. Al superar el umbral de prioridad alta (≥ 4), estos riesgos se clasifican en zona roja y requieren una respuesta activa e inmediata. Los cuatro comparten un denominador común: afectan directamente a la gestión del tiempo y a la curva de aprendizaje tecnológica, que son los factores más críticos en un proyecto de estas características. Las acciones de mitigación y el ciclo de seguimiento asociado se detallan en las secciones 2.3.3 Mitigación de riesgos y 2.3.4 Monitoreo y control respectivamente.

Gran parte de los riesgos (ID 3, 5, 6, 7, 9, 10, 11, 12 y 13) se encuentran en la zona amarilla, con valores comprendidos entre 1 y 4 (exclusive). Este rango indica una prioridad media: el riesgo existe y podría materializarse, pero su probabilidad o su impacto actuales no justifican una intervención urgente. Para estos riesgos se adopta una postura de vigilancia activa: no requieren intervención inmediata, pero sí un seguimiento periódico.

Los riesgos relacionados con la documentación (ID 14 y 15) presentan los valores más bajos (0,8), iguales o inferiores al umbral de prioridad baja (≤ 1), lo que los sitúa en zona verde. Su impacto es reducido porque la probabilidad asignada es baja y los daños potenciales se limitan a una revisión del alcance sin repercusión significativa en costes ni en los plazos críticos del proyecto. La estrategia de respuesta concreta se recoge en la sección 2.3.3 Mitigación de riesgos.

2.3.3 Mitigación de riesgos

A continuación, se definen las respuestas para reducir la probabilidad de aparición de los riesgos en el proyecto. La siguiente tabla detalla las acciones diseñadas para gestionar cada riesgo, priorizando aquellos con niveles de severidad alto y medio.

ID	Estrategia de Mitigación
1	Realizar un estudio previo para validar la viabilidad de las herramientas seleccionadas.
2	Implementar un cronograma con márgenes de tiempo teniendo en cuenta posibles desviaciones sin afectar a la entrega final.
3	Realizar un estudio previo de las tecnologías a usar, buscando asesoramiento de tutores si fuese necesario.
4	Realizar un estudio previo para validar la viabilidad de las herramientas seleccionadas.
5	Mantener reuniones periódicas con los tutores para validar los requisitos y el progreso, evitando así cambios drásticos.
6	Mantener reuniones periódicas con los tutores para validar los requisitos y el progreso, evitando así cambios drásticos.
7	Mantener reuniones periódicas con los tutores para validar los requisitos y el progreso, evitando así cambios drásticos.
8	Implementar un cronograma con márgenes de tiempo teniendo en cuenta posibles desviaciones sin afectar a la entrega final.
9	Usar datasets estáticos (CSV/JSON) o limitar el alcance a un proveedor con datos estructurados.
10	Acotar el dominio. Descartar hardware muy específico.
11	Priorizar el desarrollo de funcionalidades esenciales. Si una funcionalidad compleja bloquea el avance, se simplificará para asegurar el Producto Mínimo Viable.
12	Aplicar buenas prácticas y optimizar el código.
13	Aplicar buenas prácticas, como la ejecución de pruebas sistemáticas durante la construcción.
14	Utilizar plantillas proporcionadas por los tutores y herramientas que faciliten la documentación y todo lo incluido en ella para evitar la acumulación de trabajo al final.
15	Utilizar plantillas proporcionadas por los tutores y herramientas que faciliten la documentación y todo lo incluido en ella para evitar la acumulación de trabajo al final.

Tabla 16: Estrategias de mitigación de riesgos

2.3.4 Monitoreo y control

En esta parte se establecen los mecanismos para supervisar el proyecto y la efectividad de las estrategias de mitigación.

Se dividirá en 2 actividades:

1. Primeramente, se realiza la revisión periódica, tras cada Sprint⁶ (ver apartado 3.1 Scrum para más detalle), de las tareas completadas frente al cronograma (diagrama de Gantt). En esta revisión se compara el avance real con el planificado. Si se detecta una desviación significativa, por ejemplo, una tarea que debería haberse completado, pero sigue activa, se redistribuyen las tareas para no comprometer la entrega final.
2. Por último, se realiza la reevaluación continua, también tras cada Sprint, de la matriz de riesgos. Esta actividad consiste en recalcular la puntuación de prioridad de cada riesgo aplicando la misma fórmula definida en la sección 2.3.2 Evaluación de riesgos, pero actualizando los valores de probabilidad e impacto en función de lo observado durante la iteración. Si un riesgo de zona amarilla ha aumentado su puntuación hasta igualar o superar el umbral de 4, se eleva a zona roja y refuerza la estrategia de mitigación correspondiente descrita en la sección 2.3.3 Mitigación de riesgos. Del mismo modo, si un riesgo de zona roja ha disminuido gracias a las medidas aplicadas, puede rebajarse a zona amarilla, liberando así la carga de seguimiento intensivo. Además, en cada

⁶ Un Sprint es el ciclo de iteración básico definido en la metodología ágil Scrum.

revisión se valora si han aparecido nuevos riesgos no contemplados inicialmente que deban incorporarse a la matriz.

En la práctica, esta revisión se llevó a cabo al cierre de cada Sprint, si bien los riesgos identificados se mantuvieron estables a lo largo del proyecto. No fue necesario reclasificar riesgos entre zonas ni incorporar riesgos nuevos a la matriz inicial.

3. Metodología

Se ha definido una estrategia metodológica híbrida adaptada a las necesidades de cada fase. Esta elección responde al contexto de desarrollo individual del proyecto. Dado que es ejecutado por una única persona, la aplicación ortodoxa de las metodologías resultaría ineficiente e irreal, al requerir de un equipo y distintos roles. De este modo, se integra Design Thinking para la ideación y el análisis centrado en el usuario, el marco ágil Scrum para estructurar el proyecto, y los estándares de PMBOK para la gestión formal de la planificación, los costes y los riesgos del proyecto.

3.1 Scrum

Para la gestión y desarrollo del proyecto se ha optado por utilizar Scrum. La elección de esta metodología frente a modelos tradicionales en cascada se debe a la necesidad de gestionar la incertidumbre, permitir la adaptación a cambios durante el desarrollo y asegurar entregas incrementales de valor (Schwaber & Sutherland, 2020).

Dado que este proyecto cuenta con características específicas (es un trabajo académico individual con una fecha de entrega inamovible), se ha realizado una adaptación de Scrum.

Aunque Scrum es concebido con equipos especializados y Sprints de duración fija, para este TFG se ha adoptado un enfoque híbrido. Se mantienen los eventos y la filosofía de trabajo continuo, pero la duración y el objetivo de los Sprints se han adaptado a las fases ya definidas de investigación, planificación, desarrollo, operaciones y finalización.

Planificación de Sprints

El ciclo de vida del proyecto se ha estructurado en 6 Sprints. La duración de estos es variable (entre 2 y 4 semanas) según la complejidad de cada fase y el calendario escolar (incluyendo periodos vacacionales como Semana Santa o Feria).

La distribución es la siguiente:

Sprint	Periodo	Foco Principal
Sprint 1	02/02 – 22/02	Fases de Investigación y Planificación
Sprint 2	22/02 – 11/03	Análisis y diseño, y Automatización
Sprint 3	11/03 – 06/04	Construcción (Fase I)
Sprint 4	06/04 – 24/04	Construcción (Fase II)
Sprint 5	24/04 – 14/05	Pruebas y preproducción, Despliegue, Análisis de los resultados
Sprint 6	14/05 – 23/05	Finalización y Presentación

Tabla 17: Planificación de Sprints

Se utilizará un Tablero Kanban digital (GitHub Projects) para visualizar el estado de cada tarea (*To Do*, *In Progress*, *Done*) dentro de cada Sprint (véase sección 11.2 Issues y tablero Kanban).

3.2 PMBOK

Para la planificación de este trabajo también se ha tomado como referencia el estándar PMBOK (Project Management Institute, 2017), centrándose en tres de sus áreas de conocimiento: la Gestión del Cronograma, la Gestión de los Costes y la Gestión de los Riesgos. PMBOK no constituye una metodología en sí mismo, sino un marco de buenas prácticas que aporta el rigor y el vocabulario común con los que se han abordado dichas áreas en este proyecto.

La aplicación de esta metodología se refleja en los siguientes puntos:

- **Gestión de los Costes:** PMBOK propone estimar el presupuesto a partir de los recursos que ejecutan el proyecto, imputando a cada actividad las horas de los perfiles profesionales que la realizan y su coste por hora. Siguiendo esta técnica, el trabajo se ha descompuesto en los perfiles del sector TIC que intervienen en cada fase (analista, jefe de proyecto, desarrollador y director), valorados según las tarifas de referencia de la Junta de Andalucía. Aunque el proyecto lo desarrolla una sola persona, repartir su dedicación entre los distintos roles que asume a lo largo del ciclo de vida e imputar a los tutores el perfil de Director de dicha tabla de tarifas, en atención a sus funciones de supervisión, permite una estimación presupuestaria más realista que aplicar una única tarifa. El desarrollo completo de esta estimación se recoge en la sección 2.2 Estimación de costes y esfuerzo.
- **Gestión de los Riesgos:** la gestión sigue el ciclo propuesto por el PMBOK, que comprende la identificación de los riesgos, su evaluación cualitativa y cuantitativa según probabilidad e impacto, la planificación de respuestas ante las amenazas detectadas y el seguimiento y control de dichos riesgos a lo largo del proyecto. Este proceso se desarrolla en detalle en la sección 2.3 Gestión de riesgos.
- **Gestión del Cronograma:** conforme al PMBOK, se han secuenciado las actividades, estimado su duración y elaborado un cronograma que sirve de línea base para el seguimiento del avance. El control del cronograma permite detectar desviaciones respecto a los plazos previstos para el TFG. La planificación temporal resultante se detalla en la sección 2.1 Planificación temporal del problema.

3.3 Design Thinking

Mientras que Scrum y PMBOK estructuran la forma en que se desarrolla y gestiona el proyecto, Design Thinking se aplica con un propósito distinto: entender en profundidad el problema del usuario antes de comenzar a diseñar la solución (Brown, Design thinking, 2008). A diferencia de los métodos de desarrollo de software convencionales, que parten de una especificación de requisitos ya definida, Design Thinking incorpora una fase de investigación previa que permite descubrir necesidades reales que el usuario no siempre sabe expresar (Brown, Change by design: How design thinking transforms organizations and inspires innovation, 2009).

En este proyecto, su uso está justificado por la naturaleza del problema: la configuración de un PC a medida es una tarea que combina alta complejidad técnica con una gran barrera de conocimiento para el usuario no especializado. Esta combinación (problema complejo y usuario sin experiencia) es precisamente el escenario para el que Design Thinking está diseñado (Buchanan, 1992): estructurar la investigación para identificar los puntos de fricción reales y traducirlos en requisitos concretos del sistema.

4. Herramientas para la realización de actividades

A continuación, se detallan las diferentes herramientas y plataformas de software empleadas a lo largo del proyecto, ya sean para gestión y planificación, modelado, desarrollo técnico y elaboración documental o visual:

Microsoft Project⁷: Herramienta usada para la planificación temporal y la creación del diagrama de Gantt.

Clockify⁸: Aplicación utilizada para la gestión del tiempo y el registro de las horas dedicadas a cada actividad. Este registro se ha dividido en seis partes para facilitar la visualización del tiempo real invertido en cada fase de la planificación temporal: investigación, planificación, desarrollo, operaciones, finalización y reunión.

Mural⁹: Plataforma de apoyo visual empleada en las fases iniciales para el modelado de ideas y plasmar la aplicación de la metodología Design Thinking.

Visual Studio Code¹⁰: Entorno de desarrollo integrado (IDE) para la implementación del código.

Git¹¹: Sistema de control de versiones distribuido para la gestión del repositorio y el historial de cambios.

GitHub¹²+ GitHub Actions¹³: Plataforma de alojamiento de código que, junto con GitHub Actions, permite implementar flujos de integración continua (CI) y despliegue continuo (CD).

FlamaPy¹⁴: Framework de código abierto basado en Python diseñado para el análisis automatizado, modelado y gestión de la variabilidad en líneas de productos de software.

Microsoft Word¹⁵: Software utilizado para la redacción de la memoria del TFG.

Canva¹⁶: Herramienta de diseño gráfico empleada para la elaboración de la presentación final del proyecto.

draw.io¹⁷: Herramienta para crear diagramas, diagramas de flujo, mapas conceptuales, organigramas y esquemas técnicos.

⁷ <https://support.microsoft.com/es-es/office/instalar-project-7059249b-d9fe-4d61-ab96-5c5bf435f281>

⁸ <https://clockify.me/>

⁹ <https://app.mural.co/>

¹⁰ <https://code.visualstudio.com/>

¹¹ <https://git-scm.com/>

¹² <https://github.com/>

¹³ <https://github.com/features/actions>

¹⁴ <https://ide.flamapy.org/>

¹⁵ <https://www.microsoft.com/es-es/microsoft-365/word>

¹⁶ <https://www.canva.com/>

¹⁷ <https://app.diagrams.net/>

Docker¹⁸: Plataforma que permite empaquetar y ejecutar aplicaciones dentro de contenedores ligeros y portátiles, garantizando que funcionen de forma idéntica en cualquier entorno de desarrollo, pruebas y producción.

Render¹⁹: Plataforma de alojamiento en la nube (Platform as a Service) que permite desplegar aplicaciones web, servicios backend, bases de datos y tareas programadas.

cron-job.org²⁰: Servicio en línea que permite programar y ejecutar tareas automatizadas (peticiones HTTP) a intervalos de tiempo definidos. En este proyecto se emplea para realizar pings periódicos al backend desplegado en Render y evitar que el servicio entre en modo inactivo por falta de uso.

UptimeRobot²¹: Servicio de monitorización de disponibilidad que comprueba periódicamente si un sitio web o servicio está en línea y envía alertas inmediatas por correo electrónico u otros canales en caso de caída.

Codacy²²: Plataforma de análisis estático de código que revisa automáticamente la calidad del código, detectando problemas de estilo, complejidad, duplicación y posibles errores.

A modo de justificación, se comparan a continuación las principales alternativas evaluadas para cada categoría de herramienta y los criterios que motivaron la elección final.

Necesidad	Opción elegida	Alternativas consideradas	Motivo de la elección
Planificación temporal y diagrama de Gantt	Microsoft Project	GanttProject, ProjectLibre	Licencia disponible vía Microsoft 365 de la Universidad y familiarización tras uso en asignatura Planificación y Gestión de Proyectos Informáticos (PGPI).
Registro de tiempo	Clockify	Toggl, RescueTime	Plan gratuito sin límite de proyectos, etiquetas para desglosar por fases e informes exportables.
Repositorio y CI/CD	GitHub + GitHub Actions	GitLab	Familiarizado tras uso durante la carrera.

¹⁸ <https://www.docker.com/>

¹⁹ <https://render.com/>

²⁰ <https://cron-job.org/>

²¹ <https://uptimerobot.com/>

²² <https://www.codacy.com/>

Modelado visual y Design Thinking	Mural	Miro, FigJam	Plantillas listas para Design Thinking y exportación a imagen para incluir en la memoria.
Diagramas	draw.io	PlantUML	Personalización completa, gratuita y sin registro, integración con Google Drive y exporta a SVG/PNG.
IDE de desarrollo	Visual Studio Code	Eclipse	Más fácil e intuitivo de usar y contiene ecosistema de extensiones.
Control de versiones	Git	Mercurial, SVN	Familiarizado tras uso durante toda la carrera.
Modelo de características	FlamaPy	FeatureIDE	Framework alineado con el stack del backend.
Redacción de la memoria	Microsoft Word	LaTeX (Overleaf), Google Docs	Licencia disponible vía Microsoft 365 de la Universidad, plantilla institucional del TFG ya en formato .docx y revisión asistida con control de cambios y comentarios.
Presentación final	Canva	PowerPoint, Google Slides	Gran variedad de plantillas profesionales listas para usar.
Contenerización	Docker	Podman	Estándar en la industria, documentación exhaustiva y familiarización tras uso durante la carrera.
Despliegue en la nube (PaaS)	Render	Railway, Fly.io, Heroku	Plan gratuito suficiente para el proyecto, despliegue automático desde GitHub y soporte nativo de Docker.

Mantenimiento del servicio activo	cron-job.org	GitHub Actions	Servicio gratuito, sin configuración adicional en el repositorio y orientado específicamente a peticiones HTTP periódicas.
Monitorización de disponibilidad	UptimeRobot	Freshping, StatusCake	Plan gratuito, verificación cada 5 minutos y alertas por correo electrónico sin necesidad de configuración adicional.
Calidad y cobertura de código	Codacy	SonarCloud	Integración nativa con GitHub, plan gratuito para repositorios públicos y generación automática de informes de cobertura sin configuración adicional.

Tabla 18: Comparativa y justificación de herramientas

5. Fase de investigación

A continuación, se explica cómo se ha realizado la fase de investigación utilizando la metodología Design Thinking. La siguiente figura muestra el mural elaborado al inicio del proyecto. Cada cuadrito representa una idea o concepto generado durante las distintas etapas del método: Empatizar, Definir, Idear, Prototipar y Testear. Dentro de cada columna, los post-its de colores agrupan reflexiones, problemas detectados, posibles soluciones e hipótesis surgidas en esa etapa. Los conceptos sin tachar son aquellos que se priorizaron por su viabilidad y virtudes pedagógicas, mientras que los tachados quedaron descartados tras la reflexión. El objetivo de este ejercicio previo es entender los problemas reales de los usuarios antes de comenzar a programar, garantizando así que el software desarrollado aporte valor real al proyecto.

A las cinco fases clásicas de Design Thinking (Empatizar, Definir, Idear, Prototipar y Testear) se han añadido dos etapas adicionales, que se desarrollan a lo largo del apartado: una de Decidir, tomada del Design Sprint, situada entre la ideación y el prototipado, y una Retrospectiva final, propia de las metodologías ágiles.

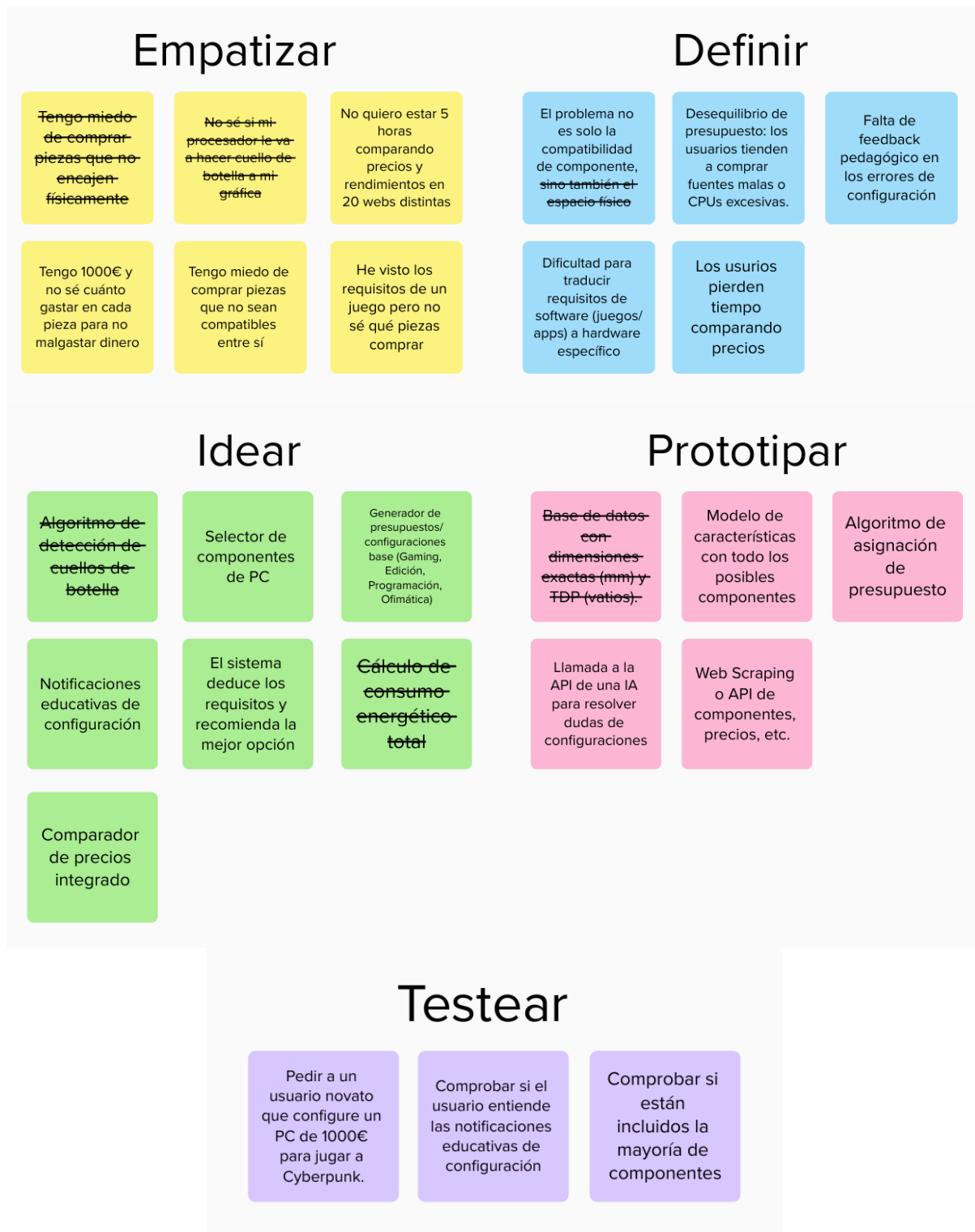


Ilustración 3: Mural Design Thinking

5.1 Entendimiento y empatía

En esta fase, se ha intentado adoptar el punto de vista del usuario para entender sus miedos a la hora de configurar un ordenador:

1. Pérdida de tiempo al tener que comparar precios y rendimiento en webs distintas.

2. Dificultad para saber cómo repartir el presupuesto según el uso que se le va a dar, por ejemplo, tener 1000€ y no saber si gastar más en el procesador o en la tarjeta gráfica para no malgastar el dinero, si se va a usar para jugar a videojuegos.
3. Temor a comprar piezas que no funcionen juntas o que no encajen.
4. Frustración al ver los requisitos técnicos de un videojuego o programa y no saber a qué piezas reales equivalen.

Para completar esta fase se empleó la técnica del Story Board, una representación visual en forma de viñetas que narra, paso a paso, la experiencia de un usuario al enfrentarse al problema. Esta técnica permite empatizar con sus emociones y frustraciones a lo largo del recorrido:



Ilustración 4: Story Board

5.2 Definir

Una vez comprendidos los miedos del usuario, en esta fase se definen exactamente qué problemas reales va a solucionar la aplicación, a partir de la investigación anterior:

1. Los usuarios tienden a distribuir el presupuesto de forma desequilibrada según el uso previsto del equipo, por ejemplo, comprando procesadores muy caros junto a gráficas muy básicas y siendo usados tal vez para actividades que requieren de una gráfica de alto rendimiento.
2. Los configuradores actuales informan de si dos componentes son compatibles o no, sin explicar con precisión el motivo.
3. El usuario necesita una forma de traducir requisitos técnicos abstractos a componentes reales sin conocimientos previos de hardware.
4. Existe una gran variedad de precios y stock en internet que obliga al usuario a comparar manualmente.

A partir de estos problemas se elaboró un Impact Map, una técnica que enlaza el objetivo principal del proyecto (Por qué) con los actores implicados (Quién), los cambios de comportamiento buscados (Cómo) y las funcionalidades concretas que los hacen posibles (Qué):

POR QUÉ FACILITAR A CUALQUIER USUARIO LA CONFIGURACIÓN DE UN PC COMPATIBLE Y EQUILIBRADO, AHORRANDO TIEMPO Y DINERO.

QUIÉN	Usuario con conocimientos técnicos	Usuario sin conocimientos técnicos
CÓMO	Evitando que compre piezas incompatibles por error.	Permitiéndole describir sus necesidades sin necesidad de conocer terminología técnica.
QUÉ	Propagación de restricciones en tiempo real + mensajes educativos sobre incompatibilidades.	Consulta por IA: genera una configuración completa a partir de una descripción libre. Generación automática: crea una configuración equilibrada a partir de un perfil de uso y un presupuesto máximo.

Tabla 19: Impact Map

Posteriormente se empleó un Story Map, técnica que organiza las funcionalidades del sistema según el flujo de actividades que el usuario realiza, ordenadas de forma horizontal por la secuencia de uso. Esto ayuda a visualizar el recorrido completo del usuario dentro de la aplicación y a priorizar qué funcionalidades son esenciales:

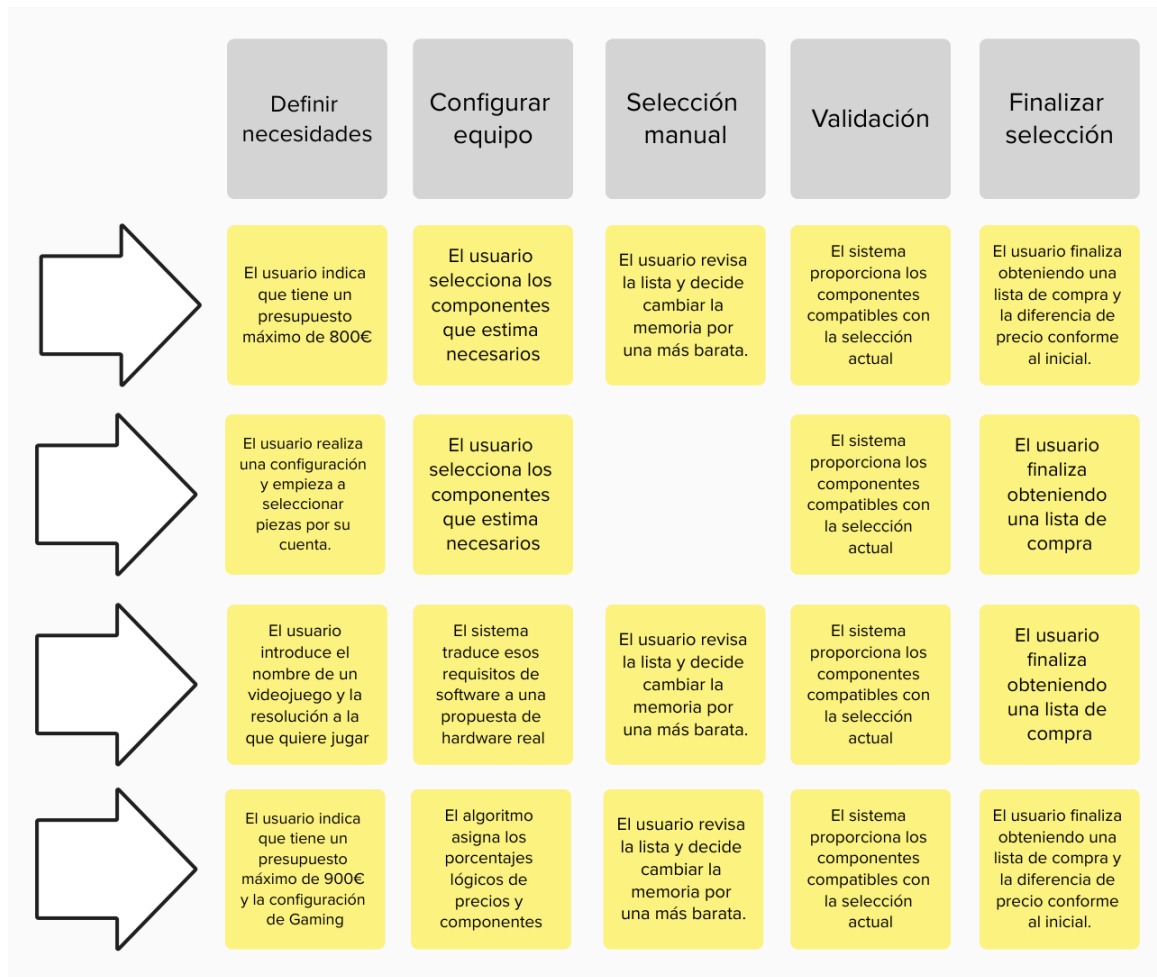


Ilustración 5: Story Map

5.3 Idear

En esta etapa, el objetivo principal es generar soluciones para los problemas definidos anteriormente. En primer lugar, se realizó un análisis de la competencia (benchmarking), técnica que consiste en estudiar productos similares ya existentes en el mercado para identificar sus puntos fuertes y, sobre todo, las carencias que el proyecto puede cubrir. Para ello se compararon los configuradores de PcComponentes, Coolmod y PCPartPicker frente a la propuesta de este TFG:

	TFG	PcComponentes	PCPartPicker	Coolmod
Selector de componentes de PC	SÍ	SÍ	SÍ	SÍ
Generador de presupuestos/configuraciones base (Gaming, Edición, Programación, Ofimática)	SÍ	NO	NO	SÍ
Notificaciones educativas de configuración	SÍ	SÍ	SÍ	SÍ
El sistema deduce los requisitos y recomienda la mejor opción	SÍ	SÍ	NO	SÍ

Comparador de precios	SÍ	NO	SÍ	NO
Exportación de la configuración en PDF y JSON	SÍ	NO	NO	NO

Tabla 20: Comparativas alternativas del mercado

Se observó que estas herramientas son muy buenas validando la compatibilidad, pero carecen de ciertas funciones que le aportan gran valor. Para cubrir estas carencias, se realizó una lluvia de ideas (Brainstorming) orientada a aportar este valor extra. Las ideas principales que surgieron fueron:

- Generador de presupuestos base: plantillas inteligentes (Gaming, Edición, Programación, Ofimática) que seleccionan los componentes óptimos y distribuyen el presupuesto de forma equilibrada entre categorías.
- Propagación de restricciones con mensajes educativos: en lugar de limitarse a señalar incompatibilidades, el sistema explica al usuario por qué dos componentes no pueden combinarse.
- Consulta en lenguaje natural: el usuario describe sus necesidades con sus propias palabras ("quiero jugar a 1080p sin gastarme mucho") y el sistema genera automáticamente una configuración, eliminando la necesidad de conocer terminología técnica.
- Enlaces a tiendas online para cada componente, evitando que el usuario tenga que buscar manualmente.
- Posibilidad de guardar el resultado de la configuración en PDF o JSON para compartirlo o consultarlo posteriormente.

5.4 Decidir

Tras la generación de ideas, se tomaron decisiones para definir el Producto Mínimo Viable (MVP). Para priorizar qué funcionalidades desarrollar se aplicó una matriz de Impacto frente a Esfuerzo, técnica que clasifica cada idea según el valor que aporta al usuario y la complejidad de implementarla, de modo que se priorizan las de alto impacto y esfuerzo asumible y se descartan o posponen las de bajo impacto y esfuerzo elevado. Las decisiones tomadas fueron las siguientes:

Ideas seleccionadas (Alto Impacto / Esfuerzo Asumible):

- Generador de configuraciones por perfil de uso: Seleccionado por ser la funcionalidad central del sistema y por resolver directamente el problema del reparto desequilibrado del presupuesto.
- Propagación de restricciones con mensajes educativos: Seleccionado porque resuelve el miedo a equivocarse.
- Consulta en lenguaje natural: Seleccionado por su alto impacto en la experiencia de usuario, al eliminar la barrera de conocimiento técnico previo.
- Comparador de precios: Seleccionado por su utilidad práctica y bajo coste de implementación.
- Exportación de la configuración: Seleccionado por ser útil para el usuario y técnicamente sencillo de implementar.

Ideas descartadas o simplificadas. Son aquellas que se muestran en la Ilustración 3: Mural Design Thinking como tachadas (Bajo Impacto / Esfuerzo Muy Alto):

- Validación de espacio físico: Se descartó esta idea, presente en fases iniciales, debido a la dificultad de mantener una base de datos actualizada con las dimensiones de cada pieza.
- Cálculo del consumo energético: Del mismo modo que la idea anterior, se descartó esta idea debido a la dificultad de mantener una base de datos actualizada con el consumo energético de cada componente.
- Cálculo de cuellos de botella: Se descartó esta idea, también presente en fases iniciales, por la dificultad supuesta a la hora de calcular los rendimientos de los componentes.

5.5 Prototipar

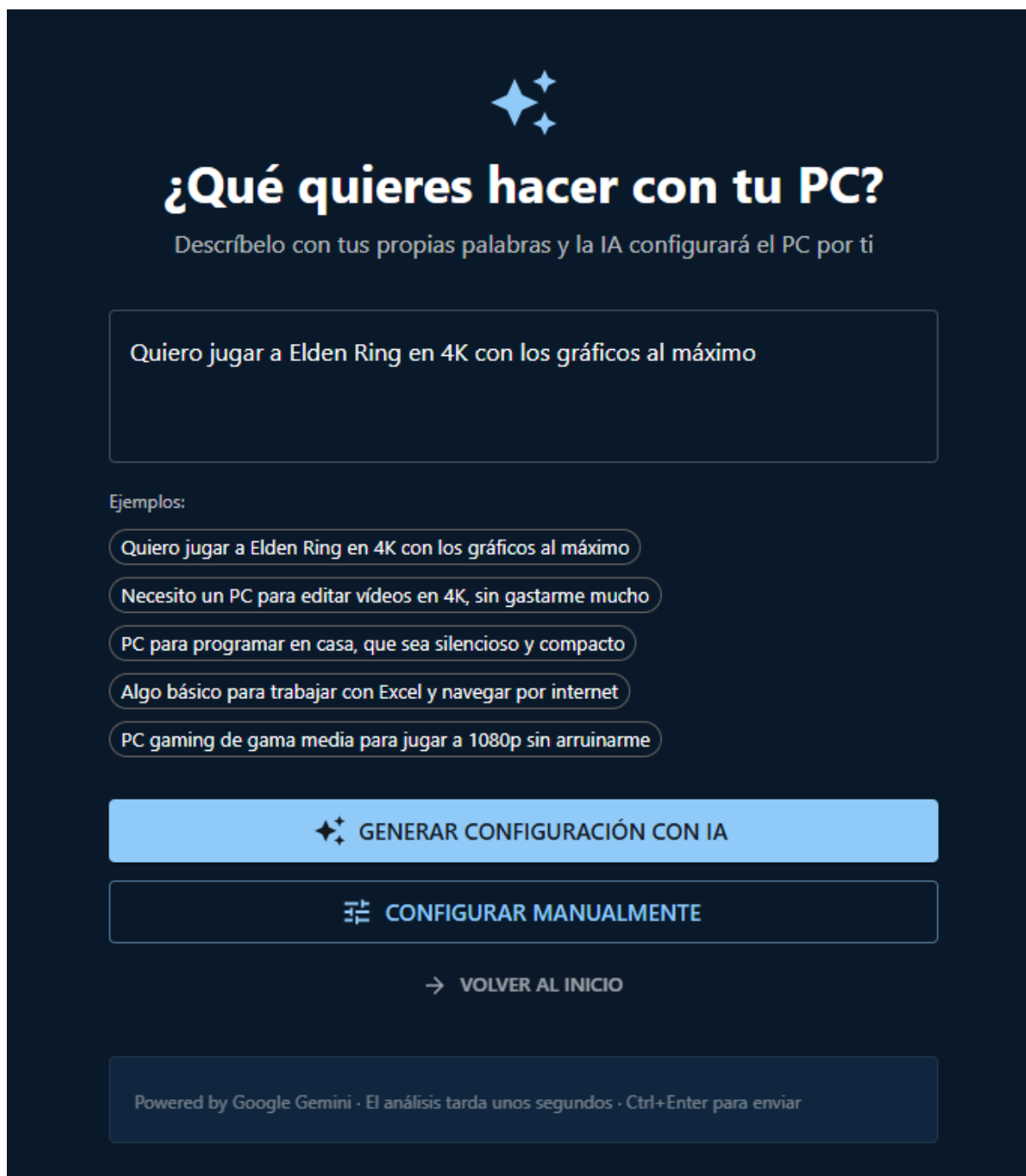
En lugar de elaborar mockups desechables, se optó por un enfoque de prototipado evolutivo: las primeras versiones del código de la aplicación hicieron las veces de prototipo. El objetivo de un prototipo es reducir la incertidumbre sobre lo que se va a desarrollar sin asumir todavía el esfuerzo completo de la versión final (Sommerville, 2011). Eso es precisamente lo que permitieron estas versiones tempranas. En concreto, sirvieron para comprobar la viabilidad de dos de las funcionalidades nucleares del proyecto: la propagación de restricciones, verificando que el motor filtraba correctamente los componentes incompatibles, y la generación automática, comprobando que el reparto por perfil de uso producía configuraciones equilibradas. Una vez validadas estas decisiones de diseño sobre código en ejecución, dicho prototipo se fue refinando y ampliando de forma incremental hasta convertirse en la versión final que se presenta en este trabajo. Dado que el equipo de desarrollo era unipersonal, este enfoque resultó más eficiente que mantener prototipos intermedios independientes. Las capturas siguientes corresponden a esa versión final, cuyo desarrollo técnico detallado se aborda en el apartado 8. Implementación.

La página de inicio presenta las dos vías de configuración disponibles: manual mediante el configurador o asistida por inteligencia artificial.



Ilustración 6: Página de inicio

La pantalla de consulta por IA permite al usuario describir sus necesidades en lenguaje natural. El sistema interpreta la consulta mediante Google Gemini y genera automáticamente una configuración completa.



¿Qué quieres hacer con tu PC?

Descríbelo con tus propias palabras y la IA configurará el PC por ti

Quiero jugar a Elden Ring en 4K con los gráficos al máximo

Ejemplos:

- Quiero jugar a Elden Ring en 4K con los gráficos al máximo
- Necesito un PC para editar vídeos en 4K, sin gastarme mucho
- PC para programar en casa, que sea silencioso y compacto
- Algo básico para trabajar con Excel y navegar por internet
- PC gaming de gama media para jugar a 1080p sin arruinarme

✦ ✦ ✦ GENERAR CONFIGURACIÓN CON IA

⚙ CONFIGURAR MANUALMENTE

→ VOLVER AL INICIO

Powered by Google Gemini · El análisis tarda unos segundos · Ctrl+Enter para enviar

Ilustración 7: Consulta por IA

El configurador permite también generar una configuración automática seleccionando un perfil de uso y un presupuesto máximo.

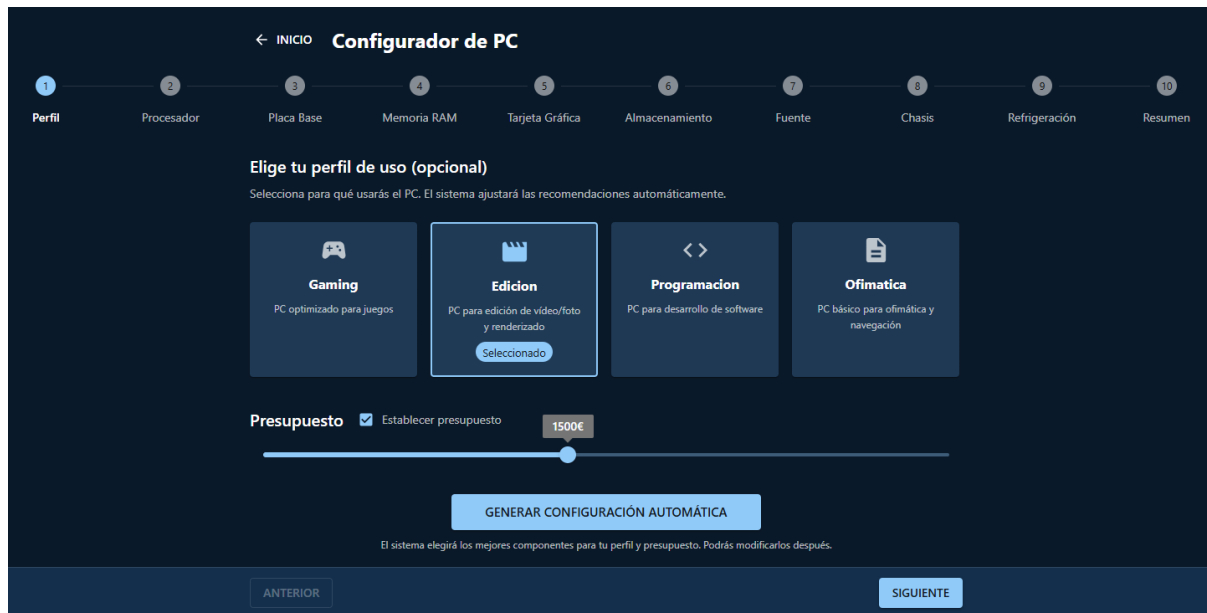


Ilustración 8: Configuración automática por perfil de uso y presupuesto

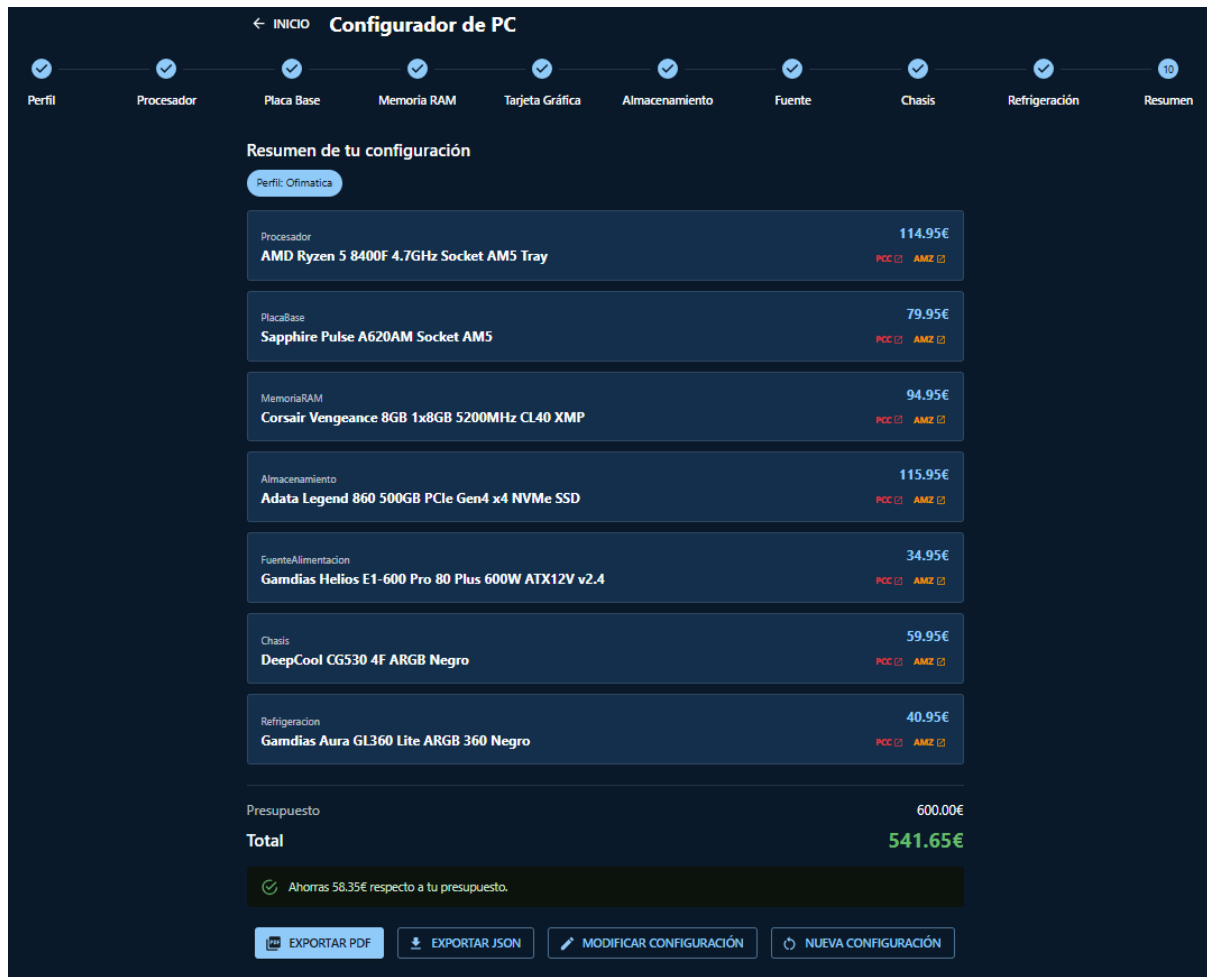
En el modo manual, el configurador guía al usuario paso a paso por cada categoría de componentes, mostrando únicamente los productos compatibles con las elecciones previas. Durante el proceso, se muestra una barra de presupuesto que se actualiza en tiempo real con el coste acumulado de los componentes seleccionados. Cada producto incluye enlaces directos a PcComponentes y Amazon²³ para facilitar su adquisición. Además, a lo largo del proceso se muestran mensajes educativos contextuales que explican al usuario el motivo técnico por el que ciertas opciones están o no disponibles.



Ilustración 9: Configurador paso a paso

El resumen final muestra la configuración completa con el precio de cada componente, la comparativa con el presupuesto, los botones de exportación y el de modificar la configuración creada. Además, se incluyen enlaces directos a PcComponentes y Amazon para cada componente, permitiendo al usuario acceder fácilmente a la compra.

²³ Multinacional tecnológica estadounidense y el mayor minorista en línea del mundo



← INICIO **Configurador de PC**

Perfil Procesador Placa Base Memoria RAM Tarjeta Gráfica Almacenamiento Fuente Chasis Refrigeración Resumen

Resumen de tu configuración

Perfil: Ofimática

Procesador	AMD Ryzen 5 8400F 4.7GHz Socket AM5 Tray	114.95€
PlacaBase	Sapphire Pulse A620AM Socket AM5	79.95€
MemoriaRAM	Corsair Vengeance 8GB 1x8GB 5200MHz CL40 XMP	94.95€
Almacenamiento	Adata Legend 860 500GB PCIe Gen4 x4 NVMe SSD	115.95€
FuenteAlimentación	Gamdias Helios E1-600 Pro 80 Plus 600W ATX12V v2.4	34.95€
Chasis	DeepCool CG530 4F ARGB Negro	59.95€
Refrigeración	Gamdias Aura GL360 Lite ARGB 360 Negro	40.95€
Presupuesto		600.00€
Total		541.65€

✓ Ahorras 58.35€ respecto a tu presupuesto.

EXPORTAR PDF EXPORTAR JSON MODIFICAR CONFIGURACIÓN NUEVA CONFIGURACIÓN

Ilustración 10: Resumen

5.6 Validar

La validación no se limitó a un único momento: durante la fase de prototipado se comprobaron de forma temprana las funcionalidades nucleares sobre las primeras versiones del código, y, como cierre del ciclo, una vez integradas todas las funcionalidades se llevó a cabo una validación final con usuarios reales sobre la versión completa de la aplicación. Estas pruebas se realizaron con usuarios de distintos perfiles, con el objetivo de comprobar que la aplicación cumplía con las expectativas y necesidades de diferentes tipos de usuarios.

En primer lugar, se contó con la participación de varias personas de unos 60 años sin experiencia técnica avanzada. Durante la prueba, las funcionalidades que resultaron más útiles fueron la consulta mediante inteligencia artificial y la configuración automática a partir de un perfil de uso y un presupuesto máximo, que valoraron muy positivamente por su sencillez. En cambio, la configuración manual del PC les resultó compleja debido a la tecnicidad de los componentes y la terminología específica del hardware, no por la dificultad de manejar la interfaz en sí, cuya navegación resultó intuitiva. En este aspecto, los mensajes educativos integrados les proporcionaron una ayuda moderada para comprender algunos de estos conceptos técnicos. Asimismo, los enlaces de compra a Amazon y PcComponentes les fueron de gran utilidad.

En segundo lugar, se realizaron pruebas con usuarios de entre 20 y 40 años con mayor familiaridad tecnológica. En general, este grupo fue capaz de utilizar la totalidad de las funcionalidades del software sin mayores dificultades. Al igual que en el caso anterior, la consulta por inteligencia artificial

y la configuración automática mediante perfil de uso y presupuesto máximo fueron las funcionalidades mejor valoradas por su comodidad y eficacia.

5.7 Retrospectiva

En general, la fase de investigación transcurrió sin grandes contratiempos. No obstante, surgieron algunas dificultades que conviene destacar.

En primer lugar, durante la etapa de Idear resultó complicado delimitar el alcance del proyecto: la lluvia de ideas generó un número elevado de funcionalidades potenciales, lo que dificultó la toma de decisiones sobre qué incluir en el MVP. Esta situación se resolvió en la etapa de Decidir, aplicando un criterio de valoración basado en el impacto para el usuario frente al esfuerzo de implementación, lo que permitió descartar ideas como la validación de espacio físico o el cálculo del consumo energético, cuyo mantenimiento resultaría inviable.

En segundo lugar, en la etapa de Prototipar se adoptó un enfoque de prototipado evolutivo: las primeras versiones del código hicieron de prototipo en lugar de elaborar mockups previos. Al tratarse de un equipo unipersonal, resultó más eficiente y permitió reducir la incertidumbre sobre las funcionalidades críticas, si bien prescindir de una validación visual previa obligó a cuidar especialmente las decisiones de diseño de la interfaz.

Por último, durante la etapa de Validar se constató que los usuarios sin experiencia técnica encontraban dificultades con la configuración manual, no por la interfaz en sí, sino por la terminología técnica del hardware. Este hallazgo reforzó la decisión de haber priorizado la consulta por inteligencia artificial y la configuración automática por perfil de uso, que resultaron las funcionalidades mejor valoradas por ambos grupos de usuarios.

6. Análisis y Diseño

Una vez delimitado el problema y planificado el proyecto, esta fase tiene como objetivo traducir las necesidades detectadas durante la etapa anterior de investigación en especificaciones técnicas concretas.

6.1 Requisitos del sistema

Los requisitos se clasifican en tres categorías: funcionales (RF), no funcionales (RNF) y de información (RI).

La siguiente figura muestra el diagrama de casos de uso del sistema. Se identifica un único actor, el Usuario, que interactúa con los trece casos de uso correspondientes a los requisitos funcionales del sistema. La relación <<include>> entre "Consultar en lenguaje natural (IA)" y "Generar configuración automática" refleja que el flujo de consulta por IA siempre culmina invocando el motor de generación automática.

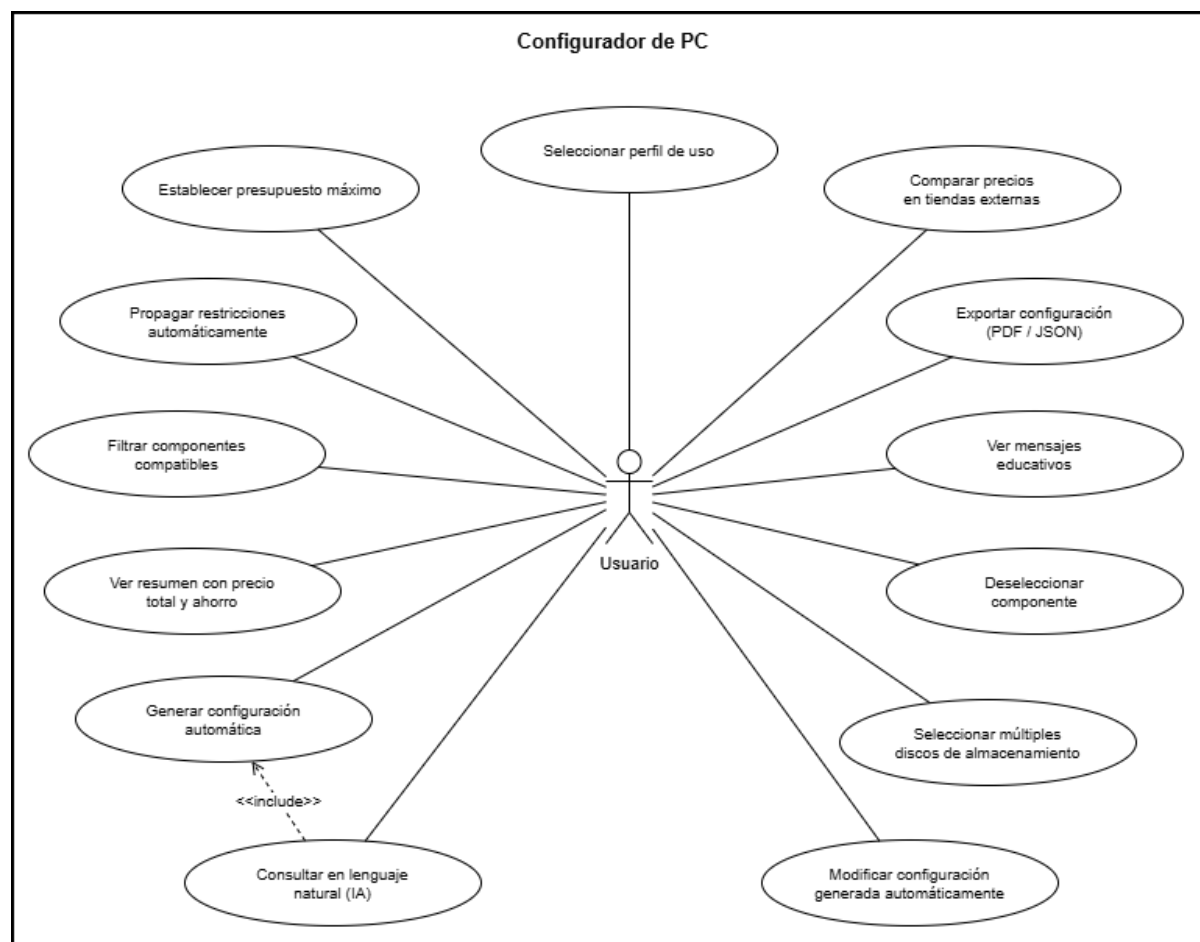


Ilustración 11: Casos de uso

6.1.1 Requisitos Funcionales

ID	Descripción
RF-01	El sistema permitirá al usuario seleccionar un perfil de uso (Gaming, Edición, Programación, Ofimática).

RF-02	El sistema permitirá al usuario introducir un presupuesto máximo en euros.
RF-03	El sistema propagará automáticamente las restricciones del modelo de características al seleccionar o descartar cualquier componente.
RF-04	El sistema mostrará únicamente los componentes compatibles con las elecciones del usuario en cada paso del configurador.
RF-05	El sistema generará automáticamente una configuración completa de PC a partir del perfil de uso y el presupuesto indicado.
RF-06	El sistema mostrará un resumen de la configuración con el precio total, el ahorro respecto al presupuesto o excedente respecto al mismo.
RF-07	El sistema permitirá al usuario describir sus necesidades en lenguaje natural y generará automáticamente una configuración interpretando la consulta mediante un modelo de lenguaje (Google Gemini).
RF-08	El sistema mostrará enlaces de búsqueda de cada componente en PcComponentes y Amazon para comparar precios reales.
RF-09	El sistema permitirá exportar la configuración final en formato PDF y JSON.
RF-10	El sistema mostrará mensajes educativos explicando por qué ciertos componentes son incompatibles en cada paso del asistente.
RF-11	El sistema permitirá deseleccionar un componente ya elegido haciendo clic de nuevo sobre él.
RF-12	El sistema permitirá seleccionar múltiples discos de almacenamiento a la vez.
RF-13	El sistema permitirá al usuario volver al configurador desde el resumen para modificar cualquier componente de una configuración generada automáticamente.

Tabla 21: Requisitos funcionales

6.1.2 Requisitos No Funcionales

ID	Descripción
RNF-01	El tiempo de respuesta de la API de propagación no superará los 3 segundos.
RNF-02	El sistema proporcionará fallback automático al catálogo JSON local si MongoDB Atlas no está disponible.
RNF-03	El frontend será accesible desde cualquier navegador moderno sin instalación de software adicional.
RNF-04	El backend contará con una suite de pruebas automatizadas con una cobertura mínima del 80%.
RNF-05	El código fuente estará gestionado con Git y alojado en GitHub, con integración y despliegue continuos mediante GitHub Actions.
RNF-06	El sistema utilizará una arquitectura desacoplada (API REST + SPA) que permita sustituir el frontend o el backend de forma independiente.
RNF-07	El sistema deberá poder desplegarse en cualquier entorno mediante contenedores Docker.

Tabla 22: Requisitos no funcionales

6.1.3 Requisitos de Información

ID	Descripción
----	-------------

RI-01	El sistema almacenará el catálogo de componentes con los campos: nombre, categoría, precio, marca, lista de características del modelo UVL (features) y especificaciones (specs).
RI-02	El modelo de características estará almacenado en formato UVL y será cargado en memoria al iniciar el backend.
RI-03	Las configuraciones generadas se calculan en tiempo de ejecución bajo demanda.

Tabla 23: Requisitos de información

6.2 Arquitectura del sistema

El sistema sigue una arquitectura con tres capas desacopladas:

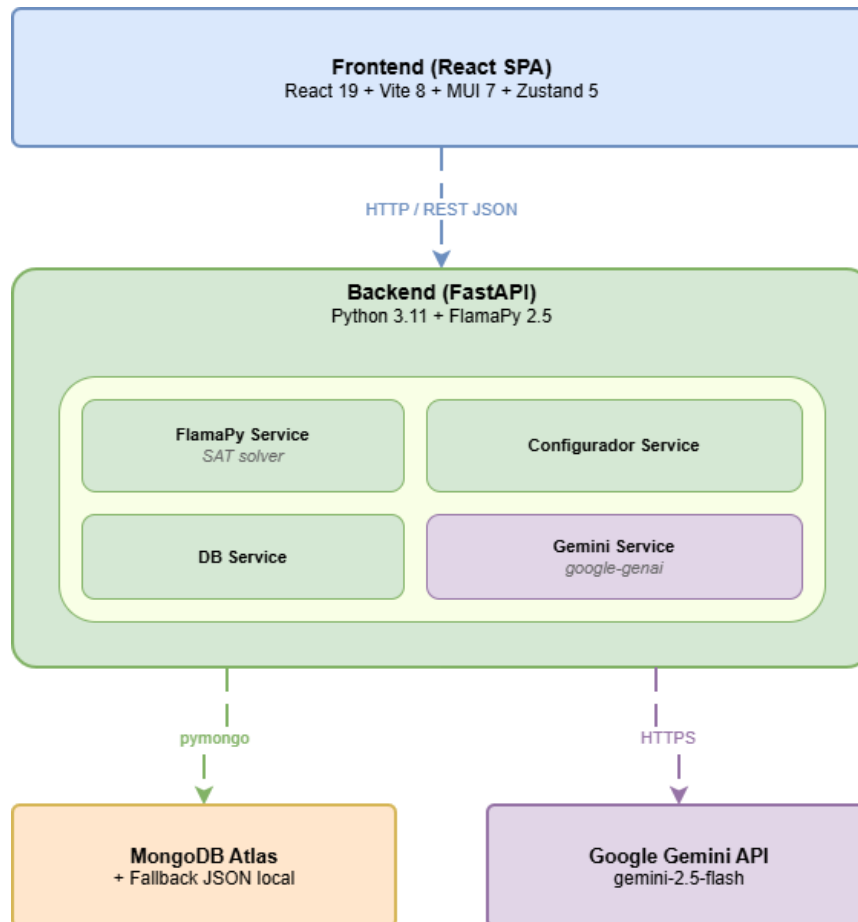


Ilustración 12: Arquitectura del sistema

Frontend: aplicación de página única (Single Page Application) desarrollada con React 19²⁴ y Vite (detallado en el apartado 8.2 Frontend) como herramienta de build y servidor de desarrollo. Gestiona el estado global de la configuración con Zustand (ver 8.2.2 Estado global con Zustand) y utiliza Material UI para los componentes visuales. Se comunica con el backend a través de API REST.

Backend: servidor FastAPI²⁵ que expone tres grupos de endpoints (/api/features, /api/productos, /api/configuracion). Internamente se organiza en cuatro servicios principales: Integra FlamaPy como servicio singleton cargado en el arranque, evitando recargas innecesarias del modelo UVL. Por otro lado, el motor de configuración se encarga de la propagación

²⁴ React es una biblioteca de JavaScript de código abierto, creada por Meta en 2013, diseñada para construir interfaces de usuario (UI) interactivas, rápidas y escalables para aplicaciones web y móviles.

²⁵ Framework web para construir APIs con Python.

de restricciones con la selección de productos del catálogo. También, el servicio de base de datos gestiona la conexión a MongoDB Atlas con fallback automático al catálogo JSON local si la conexión falla. Y adicionalmente, el servicio Gemini maneja la comunicación con la API de Google Gemini para interpretar consultas en lenguaje natural y adaptarlas al modelo.

Capa de datos: MongoDB Atlas como base de datos para el catálogo de productos, accedida a través del servicio de base de datos descrito anteriormente.

6.3 Diagrama de clases

El diagrama de clases completa la vista del sistema mostrando las responsabilidades de cada módulo y las relaciones entre ellos. A diferencia del diagrama de arquitectura anterior (sección 6.2 Arquitectura del sistema), que describe las capas y tecnologías, el diagrama de clases desciende al nivel de código: qué atributos gestiona cada componente y qué operaciones expone.

El diagrama recoge siete elementos principales agrupados por capa:

- **useConfigStore** (azul): store Zustand del frontend. Centraliza el estado de la sesión de configuración (componentesElegidos, perfil, presupuesto, activeStep) y maneja todas las llamadas al backend.
- **ConfiguradorService** (verde): motor de configuración del backend. Expone la estrategia de generación (por perfil/presupuesto) y contiene la lógica de selección de productos y asignación de presupuesto por categoría.
- **FlamapyService** (verde): encapsula el modelo UVL y el solver SAT (ver apartado 7. Modelo de Características para más detalle). Proporciona operaciones de validación, propagación de restricciones y análisis del modelo de características.
- **GeminiService** (verde): gestiona la comunicación con la API de Google Gemini. La función interpretar_consulta() traduce texto libre a un JSON estructurado con perfil, presupuesto, features, explicación y entendida (el campo entendida será false cuando la consulta sea texto sin sentido, spam o completamente irrelevante para PCs.).
- **DBService** (verde): capa de acceso a datos. Abstrae la fuente (MongoDB Atlas o JSON local) y expone una única operación de consulta filtrada por features.
- **Producto** (amarillo): entidad de dominio que representa un componente del catálogo.
- **ConfiguracionResult** (amarillo, <<response>>): estructura de respuesta devuelta por el motor al frontend, incluyendo los componentes seleccionados, el precio total, avisos de restricciones y la propagación final.

Las relaciones modeladas son: useConfigStore llama al backend mediante HTTP REST. ConfiguradorService usa FlamapyService para propagar restricciones, DBService para consultar el catálogo y GeminiService para interpretar consultas en lenguaje natural. DBService devuelve peticiones de Producto y ConfiguradorService produce un ConfiguracionResult.

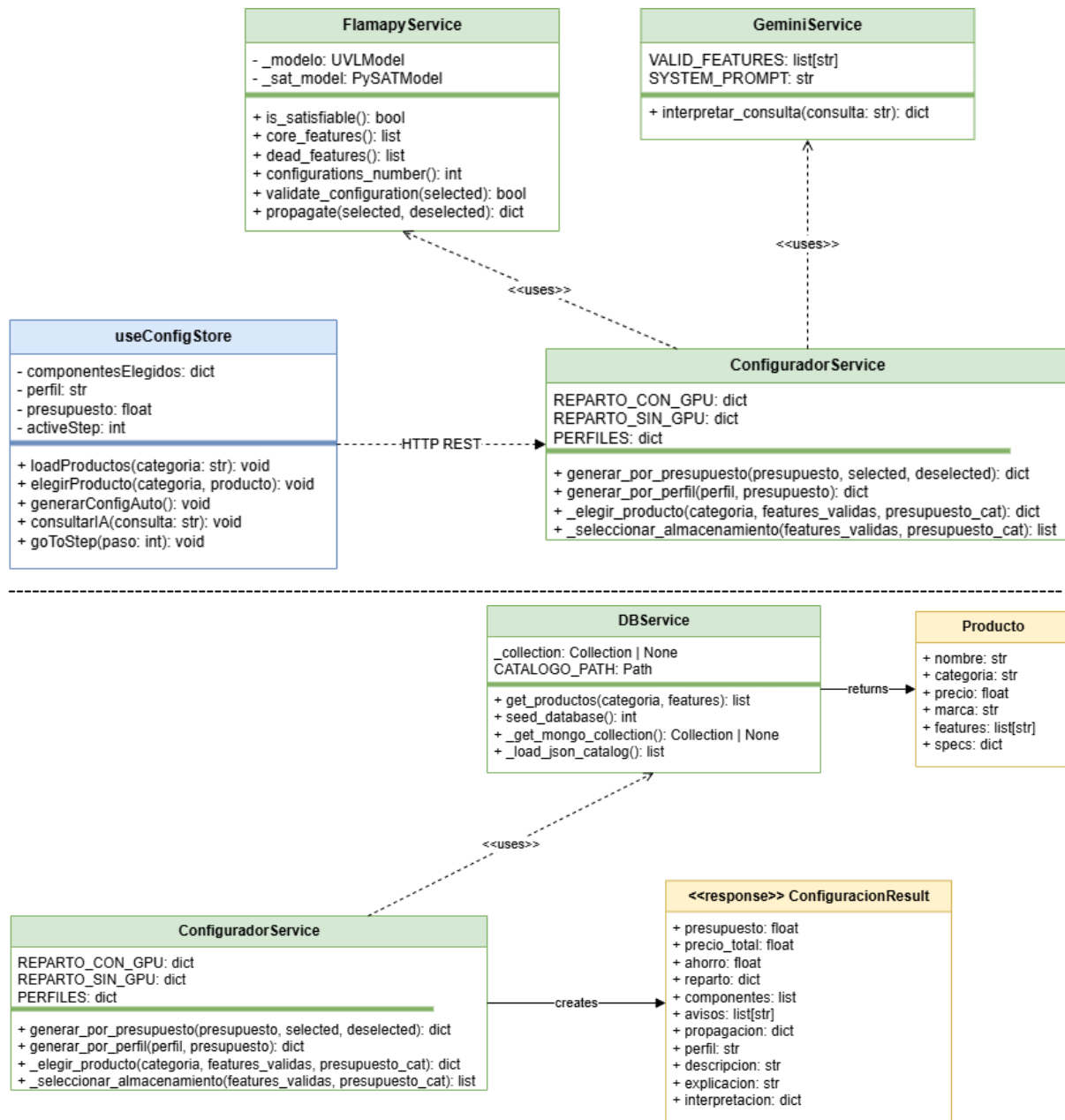


Ilustración 13: Diagrama de clases

6.4 Diagrama de secuencia

Para complementar las vistas de la arquitectura descritas en las secciones 6.2 Arquitectura del sistema y 6.3 Diagrama de clases, se presentan a continuación los diagramas de secuencia de los tres flujos principales del sistema. Mientras las secciones anteriores describen qué componentes existen, cómo se organizan y qué operaciones exponen, estos diagramas muestran cómo interactúan esos componentes en tiempo de ejecución para dar respuesta a cada caso de uso.

6.4.1 Generación automática de configuración

El flujo principal del caso de uso de generación automática es el siguiente:

1. El usuario selecciona perfil de uso y presupuesto en el frontend.
2. El frontend llama a POST /api/configuracion/perfil.

3. El backend obtiene las features del perfil (p.ej. Gaming → selected: ["Gaming"]).
4. Invoca `FlamapyService.propagate()`²⁶ (detallado en 7.2 Propagación de restricciones) para determinar features forzadas y seleccionables.
5. Para cada categoría de componente, calcula el presupuesto asignado según los porcentajes de reparto ya establecidos en el Configurador Service.
6. Filtra el catálogo con las features válidas y elige el producto de mayor precio dentro del presupuesto.
7. El frontend navega directamente al paso de resumen mostrando los componentes seleccionados.
8. Devuelve la configuración completa con precio total, ahorro o excedentes, botones de exportación (PDF/JSON) y modificar la configuración generada.

6.4.2 Configuración manual paso a paso

El flujo del asistente manual se repite en cada paso de selección de componente:

1. El usuario avanza al paso de una categoría (p.ej. Procesador).
2. El frontend recoge las features de todos los componentes ya elegidos en categorías previas.
3. Llama a `POST /api/features/propagate` con las features seleccionadas y descartadas acumuladas.
4. FlamaPy ejecuta comprobaciones para cada feature restante y devuelve la clasificación: forzadas, excluidas y seleccionables.
5. El frontend filtra el catálogo de la categoría actual mostrando solo los productos cuyas features son forzadas o seleccionables.
6. Si alguna feature queda excluida, `utils/educationalMessages.js`²⁷ (véase sección 8.2.4 Notificaciones educativas) evalúa las reglas y muestra mensajes explicativos al usuario.
7. El usuario hace clic en un producto: si ya estaba seleccionado se deselectiona, si no sustituye al anterior.
8. El estado global de Zustand se actualiza y el usuario puede avanzar al siguiente paso.
9. Este ciclo (pasos 2–8) se repite hasta completar todas las categorías de componentes.
10. En el paso de resumen el frontend muestra el precio total, ahorro respecto al presupuesto o excedente, botones de exportación (PDF/JSON) y modificar la configuración generada.

²⁶ `Propagate()` consiste en clasificar cada feature del modelo como forzada (debe seleccionarse), excluida (no puede seleccionarse) o seleccionable (libre elección), a partir de las features ya fijadas.

²⁷ Módulo que evalúa las reglas de incompatibilidad y genera los mensajes explicativos.

6.4.3 Consulta en lenguaje natural (Gemini)

El flujo del caso de uso de consulta con IA es el siguiente:

1. El usuario escribe una descripción en lenguaje natural en /consulta (p.ej. "quiero jugar en 4K sin arruinarme").
2. El frontend llama a consultarIA(consulta) del store Zustand.
3. El store hace POST /api/configuracion/consulta con el texto.
4. El backend valida que la consulta no esté vacía (HTTP 400 en caso contrario) y llama a GeminiService.interpretar_consulta()²⁸ (descrito en detalle en el apartado 8.1.4 Servicio Gemini).
5. El servicio Gemini construye la petición al modelo gemini-2.5-flash.
6. Google Gemini devuelve un JSON con perfil, presupuesto, selected, deselected, explicación y entendida.
7. El backend sintetiza la respuesta eliminando features desconocidas, fusiona las features del perfil detectado con las extraídas por Gemini, y aplica el presupuesto por defecto del perfil si Gemini no proporcionó uno.
8. Delega en generar_por_presupuesto() (véase sección 8.1.3 Motor de configuración) para producir la configuración completa (mismo algoritmo que la generación automática).
9. Añade perfil, explicacion e interpretacion al resultado y lo devuelve al frontend.
10. El frontend rellena el store con los componentes, perfil y presupuesto recibidos, y redirige al paso de resumen mostrando el precio total, el ahorro respecto al presupuesto o excedentes, botones de exportación (PDF/JSON) y modificar la configuración generada.

El flujo elegido para ser representado mediante un diagrama de secuencia es la consulta en lenguaje natural porque es el escenario que activa la totalidad de los componentes del sistema: el frontend, el backend, el motor de interpretación con Google Gemini, el solucionador SAT FlamaPy y el catálogo real en MongoDB. Los otros dos flujos (generación automática y configuración manual) están incluidos en este: La generación automática omite la fase de interpretación de lenguaje natural y el asistente manual omite además la generación automática.

²⁸ Función del servicio Gemini que construye la petición al modelo gemini-2.5-flash.

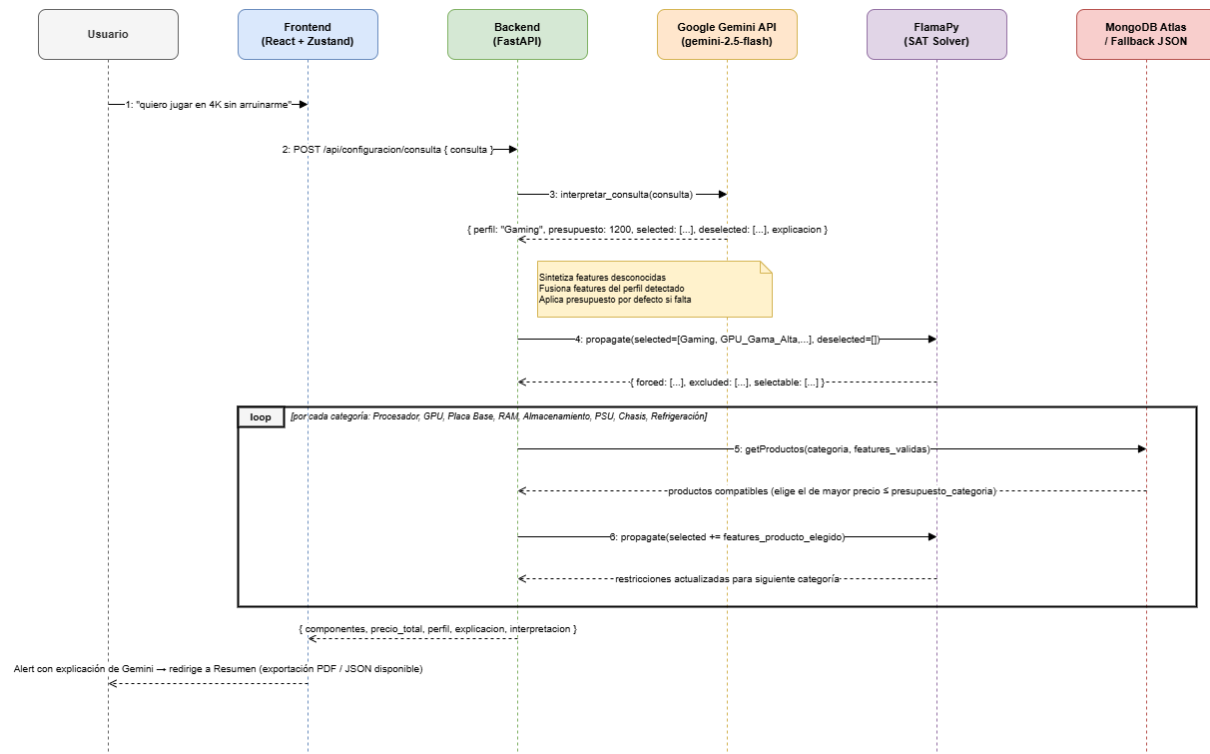


Ilustración 14: Diagrama de secuencia (consulta en lenguaje natural)

6.5 Diagrama de actividades

El diagrama de actividades modela el flujo del sistema desde la perspectiva del usuario, complementando las vistas (diagrama de clases y de secuencia) con una visión completa del proceso de configuración. Muestra los tres caminos alternativos de uso y las decisiones clave que los diferencian.

El diagrama parte de un único punto de inicio y se bifurca en tres caminos según el modo de configuración elegido:

- **Consulta en lenguaje natural (IA):** el usuario escribe una descripción libre. Gemini extrae perfil, presupuesto, features, explicación y entendida, y el backend genera la configuración completa de forma automática redirigiendo al Resumen.
- **Generación automática:** el usuario selecciona un perfil predefinido e introduce un presupuesto. El backend asigna porcentajes por categoría y elige el mejor producto disponible para cada una.
- **Asistente manual paso a paso:** el usuario recorre el configurador categoría a categoría. En cada paso, FlamaPy propaga las restricciones del modelo UVL para mostrar solo los productos compatibles. El ciclo se repite hasta completar las 8 categorías de componentes.

Los tres caminos convergen en el paso de Resumen, donde el usuario visualiza los componentes seleccionados y el precio total. Desde ahí puede volver al asistente para modificar cualquier componente, o bien exportar la configuración en PDF o JSON antes de finalizar.

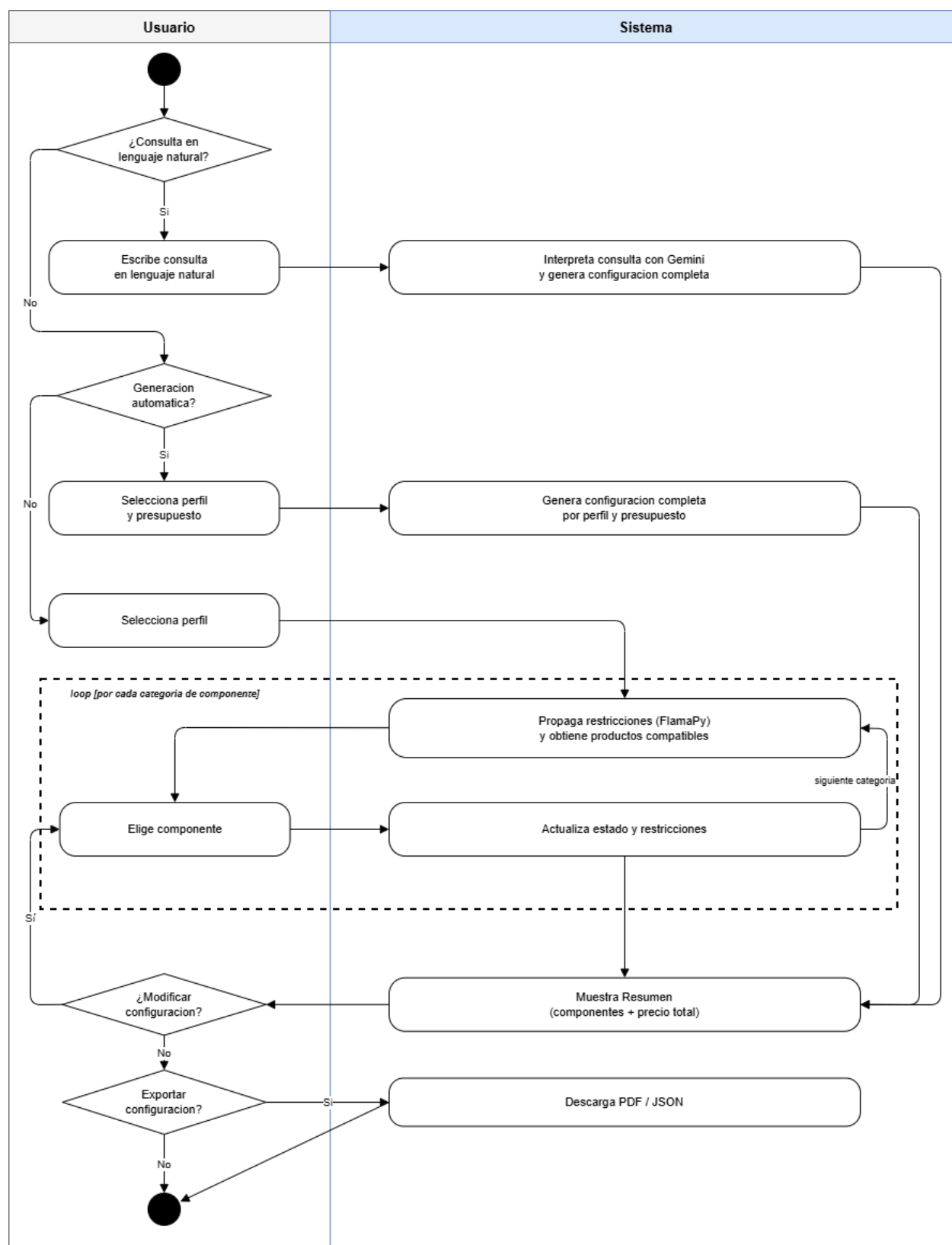


Ilustración 15: Diagrama de actividades

7. Modelo de Características

El Modelo de Características de este proyecto está formalizado en UVL (Universal Variability Language), un lenguaje estándar de propósito general para la especificación de modelos de variabilidad, propuesto por (Sundermann, Feichtinger, Engelhardt, Rabiser, & Thüm, 2021).

El análisis del modelo se realiza con FlamaPy v2.5.0, un framework de código abierto desarrollado por el grupo DiversoLab de la Universidad de Sevilla. FlamaPy transforma el modelo UVL en una codificación SAT (Boolean Satisfiability) mediante el solucionador PySAT y expone operaciones de razonamiento como: comprobar si el modelo es satisfacible, calcular el número de configuraciones válidas, identificar características núcleo (core features, siempre presentes) y características muertas (dead features, nunca seleccionables), y verificar si una configuración parcial puede completarse de forma válida.

7.1 Diseño del modelo

El modelo resultante, denominado PC_Custom, contiene 54 características (40 hojas seleccionables) y 23 restricciones entre características. El formato UVL utilizado para representar el modelo es un lenguaje textual estándar diseñado para ser legible por humanos y compatible con diversas herramientas de análisis (Benavides, y otros, 2025).

7.1.1 Estructura jerárquica

En los Feature Models, el tipo de grupo define cuántas características hijas pueden seleccionarse simultáneamente:

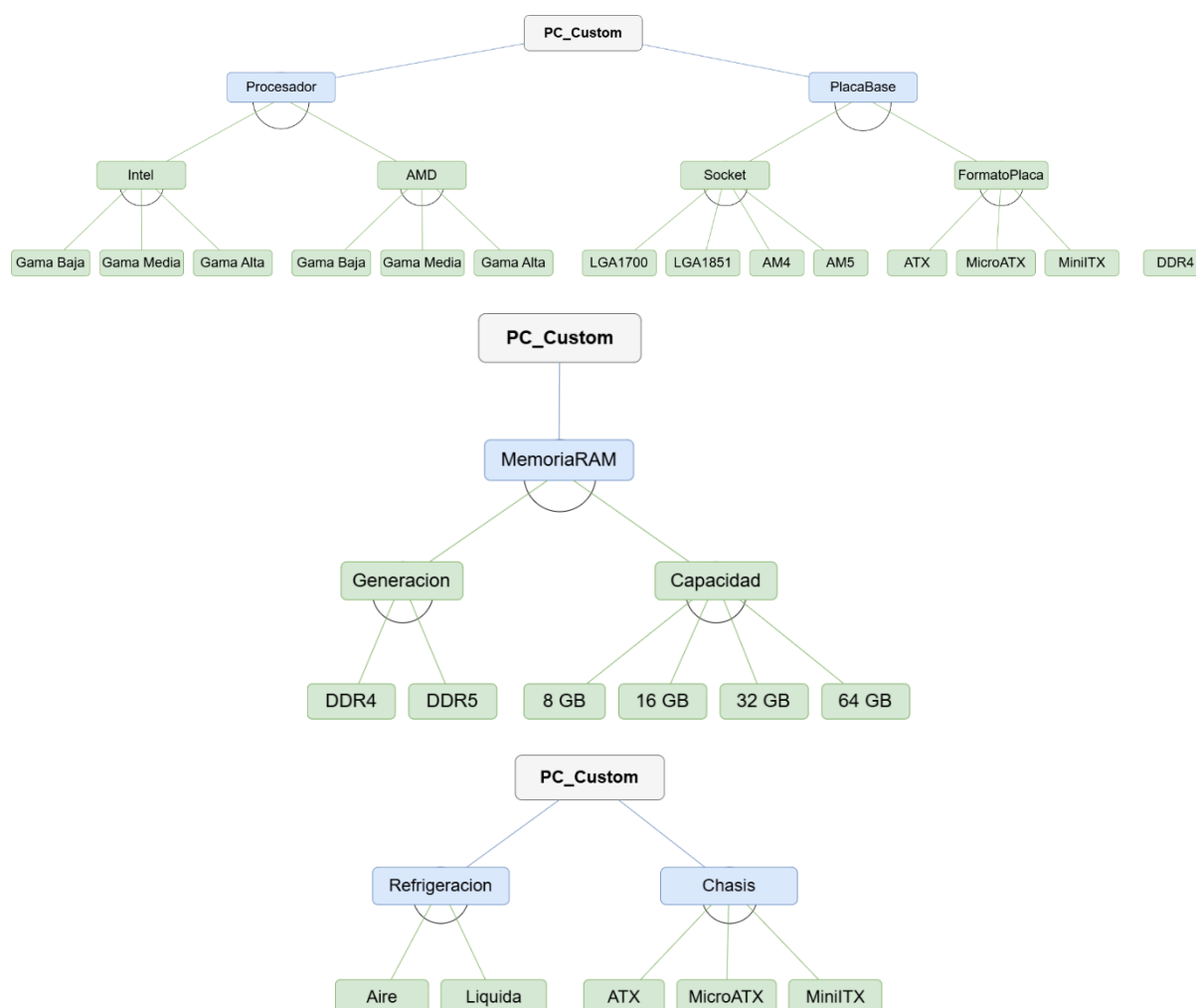
- **alternative:** exactamente una de las hijas debe seleccionarse (elección exclusiva).
- **or:** una o más hijas pueden seleccionarse a la vez.
- **mandatory:** la hija siempre está presente en toda configuración válida.
- **optional:** la hija puede o no seleccionarse libremente.

Grupo	Tipo en PC_Custom	Tipo interno	Características hijas
Procesador	mandatory	alternative	Intel, AMD
Intel	—	alternative	Intel_Gama_Baja, Intel_Gama_Media, Intel_Gama_Alta
AMD	—	alternative	AMD_Gama_Baja, AMD_Gama_Media, AMD_Gama_Alta
PlacaBase > Socket	mandatory	alternative	LGA1700, LGA1851, AM4, AM5
PlacaBase > FormatoPlaca	mandatory	alternative	Placa_ATX, Placa_MicroATX, Placa_MiniITX

MemoriaRAM > GeneracionRAM	mandatory	alternative	DDR4, DDR5
MemoriaRAM > CapacidadRAM	mandatory	alternative	RAM_8GB, RAM_16GB, RAM_32GB, RAM_64GB
Refrigeracion	mandatory	alternative	Refrigeracion_Aire, Refrigeracion_Liquida
Chasis	mandatory	alternative	Caja_ATX, Caja_MicroATX, Caja_MinilTX
FuenteAlimentacion	mandatory	alternative	PSU_500W, PSU_650W, PSU_750W, PSU_850W
Almacenamiento	mandatory	or	SSD_NVMe, SSD_SATA, Disco_HDD
TarjetaGrafica	optional	alternative	GPU_Gama_Baja, GPU_Gama_Media, GPU_Gama_Alta
PerfilUso	optional	alternative	Gaming, Ofimatica, Edicion, Programacion

Tabla 24: Modelo de características

La siguiente figura complementa la tabla con una representación visual del árbol completo:



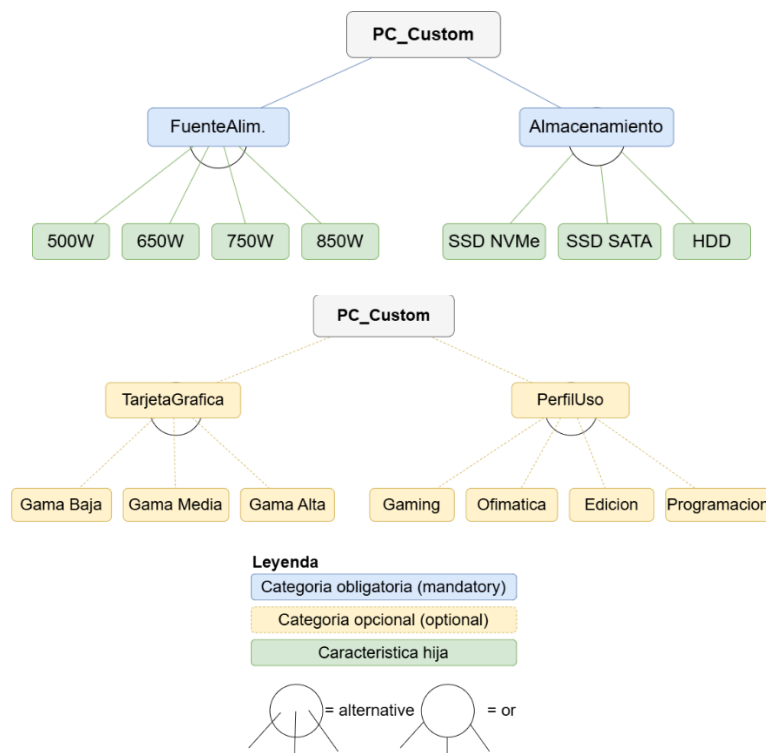


Ilustración 16: Modelo de características (representación visual)

7.1.2 Restricciones entre características

Las restricciones del modelo codifican incompatibilidades y requisitos de rendimiento reales. Conviene distinguir dos tipos: las de compatibilidad física y eléctrica (socket–procesador, socket–RAM, formato placa–chasis y potencia de la fuente) reflejan hechos verificables recogidos en las especificaciones de los fabricantes y en los estándares de la industria, y las basadas en umbrales de rendimiento (TDP y refrigeración, dimensiones de la GPU y requisitos por perfil) parten de valores de referencia escogidos como aproximación, es decir, reflejan el comportamiento habitual del mercado, pero no pretenden ser umbrales exactos ni de aplicación universal.

A continuación, se describe la explicación de cada una junto con su expresión lógica, empleando las conectivas de la lógica proposicional: $\Rightarrow$ (implicación), $\vee$ (disyunción, “o”), $\wedge$ (conjunción, “y”) y $\neg$ (negación).

Compatibilidad socket–procesador:

Los procesadores Intel usan zócalos LGA mientras que los AMD usan zócalos AM. Un procesador Intel no puede instalarse en una placa AMD ni viceversa. La restricción es bidireccional, es decir, elegir marca fuerza el socket, y elegir socket fuerza la marca (AMD, s.f.; AMD, s.f.; Intel Corporation, s.f.).

- Intel $\Rightarrow$ (LGA1700 $\vee$ LGA1851) y recíprocamente
- AMD $\Rightarrow$ (AM4 $\vee$ AM5) y recíprocamente

Compatibilidad socket–RAM:

Cada placa base solo admite una generación de memoria DDR. AM4 es una plataforma de generación anterior que únicamente soporta DDR4. AM5 y LGA1851 son plataformas modernas que exigen DDR5 (AMD, s.f.; AMD, s.f.; Intel Corporation, s.f.).

- AM4 $\Rightarrow$ DDR4 (solo soporta DDR4)
- AM5 $\Rightarrow$ DDR5 (solo soporta DDR5)
- LGA1851 $\Rightarrow$ DDR5 (Intel de última generación, solo DDR5)
- El socket LGA1700 no aparece en estas restricciones porque admite tanto DDR4 como DDR5, por lo que no impone ninguna limitación sobre la generación de memoria.

Compatibilidad formato placa–chasis:

La placa base debe encajar físicamente dentro del chasis. Una placa ATX (305 × 244 mm) solo cabe en una caja ATX, una placa MicroATX es más pequeña y cabe también en cajas ATX. Las placas Mini-ITX no imponen restricción sobre la caja porque son las más pequeñas. Pero el modelo sí impide montar placas grandes en cajas pequeñas (Intel Corporation, 1997).

- Placa_ATX $\Rightarrow$ Caja_ATX
- Placa_MicroATX $\Rightarrow$ (Caja_ATX V Caja_MicroATX)

Requisitos de PSU por consumo de GPU:

Las tarjetas gráficas de gama alta pueden superar los 300 W de consumo, por lo que requieren fuentes de alimentación de mayor potencia para garantizar la estabilidad del sistema. Las fuentes de menor potencia (500 W) no proporcionan margen suficiente para GPUs exigentes (NVIDIA, s.f.).

- GPU_Gama_Alta $\Rightarrow$ (PSU_750W V PSU_850W)
- GPU_Gama_Media $\Rightarrow$ (PSU_650W V PSU_750W V PSU_850W)

Refrigeración por TDP del procesador:

Los procesadores de gama alta suelen tener un TDP (Thermal Design Power) superior a 100 W, lo que desborda la capacidad de disipación de los disipadores de aire convencionales, por lo que se recomienda el uso de refrigeración líquida. A su vez, un radiador para refrigeración líquida requiere espacio para montarse en el panel frontal o superior, lo que puede no ser viable en cajas Mini-ITX compactas.

- Intel_Gama_Alta $\Rightarrow$ Refrigeracion_Liquida
- AMD_Gama_Alta $\Rightarrow$ Refrigeracion_Liquida
- Refrigeracion_Liquida $\Rightarrow$ (Caja_ATX V Caja_MicroATX)

Restricción de espacio físico:

Las cajas Mini-ITX son extremadamente compactas (longitud interior típica inferior a 300 mm) y no suelen tener capacidad para alojar tarjetas gráficas de gama alta, que habitualmente superan los 330 mm de longitud.

- Caja_MiniiTX $\Rightarrow$ -GPU_Gama_Alta

Requisitos por perfil de uso:

Cada perfil de uso implica unas necesidades mínimas de hardware: el perfil Gaming exige GPU de gama media o alta con al menos 16 GB de RAM para ejecutar videojuegos modernos con fluidez. El perfil Edición requiere GPU para acelerar el renderizado y al menos 32 GB de RAM para manejar proyectos de vídeo o imágenes de alta resolución (Adobe, s.f.). El perfil Programación no necesita GPU dedicada, pero sí al menos 16 GB de RAM para entornos de desarrollo, contenedores y compilaciones paralelas.

- Gaming $\Rightarrow$ TarjetaGrafica $\wedge$ (GPU_Gama_Media $\vee$ GPU_Gama_Alta) $\wedge$ (RAM_16GB $\vee$ RAM_32GB $\vee$ RAM_64GB)
- Edicion $\Rightarrow$ TarjetaGrafica $\wedge$ (RAM_32GB $\vee$ RAM_64GB)
- Programacion $\Rightarrow$ (RAM_16GB $\vee$ RAM_32GB $\vee$ RAM_64GB)

7.2 Propagación de restricciones

El algoritmo de propagación, implementado en `FlamapyService.propagate()`, recibe dos listas: las características ya seleccionadas por el usuario y las que ha descartado explícitamente. Para cada característica aún no decidida ejecuta dos comprobaciones de satisfacibilidad contra el solver SAT:

1. ¿Puede seleccionarse?: comprueba si añadirla a las ya seleccionadas produce una combinación válida según las restricciones.
2. ¿Puede descartarse?: comprueba si excluirla sigue dejando una configuración válida.

Con los resultados de ambas comprobaciones, clasifica la característica como:

can_select	can_deselect	Clasificación
✓	X	Forzada — descartarla hace el modelo insatisfacible
X	✓	Excluida — seleccionarla hace el modelo insatisfacible
X	X	Excluida — incompatible en ambos sentidos
✓	✓	Seleccionable — libre elección del usuario

Tabla 25: Clasificación de características

Esta clasificación tiene dos usos en el sistema: en el frontend determina qué productos del catálogo se muestran, ocultan o deshabilitan en tiempo real según las elecciones del usuario. Y en el backend, `ConfiguradorService` la utiliza durante la generación automática de configuraciones para filtrar en cada categoría los productos cuyas características son compatibles con las ya seleccionadas.

8. Implementación

Este capítulo describe los detalles de implementación del sistema, organizado en dos bloques principales: el backend, que contiene la lógica de negocio, la integración con FlamaPy y Google Gemini y la gestión de la base de datos, y el frontend, que implementa la interfaz de usuario y la comunicación con la API REST. El código fuente completo del proyecto está disponible en el repositorio público de GitHub: <https://github.com/Juanramire/tfg>.

8.1 Backend

El backend está desarrollado con FastAPI y Python 3.11. El punto de entrada es `main.py`, que configura la aplicación y registra tres routers:

- `/api/features` — operaciones sobre el modelo de características.
- `/api/productos` — consulta del catálogo.
- `/api/configuracion` — motor de configuración.

8.1.1 Servicio FlamaPy

FlamaPy es un framework de análisis automatizado de modelos de características (Galindo, Horcas, Felfernig, Fernandez-Amoros, & Benavides, 2023). `FlamapyService` es un singleton²⁹ instanciado al arrancar el servidor. Carga el modelo UVL desde `backend/data/modelo.uvl` y lo convierte a una codificación PySAT. Expone los métodos:

Método	Descripción
<code>is_satisfiable()</code>	Comprueba que el modelo no contiene contradicciones.
<code>core_features()</code>	Devuelve las características presentes en toda configuración válida.
<code>dead_features()</code>	Devuelve las características que nunca pueden seleccionarse.
<code>configurations_number()</code>	Cuenta el número total de configuraciones válidas (resultado cacheado).
<code>validate_configuration(selected)</code>	Verifica si una configuración parcial puede completarse.
<code>propagate(selected, deselected)</code>	Clasifica el resto de features en forzadas, excluidas y seleccionables.

²⁹ Diseño que garantiza que una clase tenga una única instancia durante todo el ciclo de vida de la aplicación

Tabla 26: Métodos de FlamapyService

El método más relevante para el sistema es `propagate()`. Para cada característica que no está ni seleccionada ni descartada, realiza dos comprobaciones SAT: si puede seleccionarse y si puede descartarse. Según el resultado, la clasifica como forzada, excluida o seleccionable:

```
def propagate(self, selected: list[str], deselected: list[str]) -> dict:
    forced = set(selected)
    excluded = set(deselected)
    selectable = set()

    remaining = self._feature_names - forced - excluded

    for fname in remaining:
        can_select = self.validate_configuration(list(forced | {fname}))

        test_deselected = {name: True for name in forced}
        test_deselected[fname] = False
        config = Configuration(test_deselected)
        op = PySATsatisfiableConfiguration()
        op.set_configuration(config)
        op.execute(self._sat)
        can_deselect = op.get_result()

        if can_select and not can_deselect:
            forced.add(fname)
        elif not can_select:
            excluded.add(fname)
        else:
            selectable.add(fname)

    return {"forced": sorted(forced), "excluded": sorted(excluded),
            "selectable": sorted(selectable)}
```

Código 1: Método `propagate()` de propagación de restricciones

8.1.2 Catálogo y base de datos

El catálogo de productos reales se genera mediante un proceso de adquisición en dos fases implementado en `backend/scrapper_coolmod.py` y `backend/import_catalogo.py`.

Scraping de Coolmod: el scraper recorre las páginas de cada una de las 8 categorías extrayendo nombre, precio y URL de detalle. A continuación, visita la ficha de cada producto para obtener sus especificaciones técnicas en bruto (pares clave–valor de los `<li>` de la página).

Clasificación con Gemini: los productos se envían en lotes de 5 al modelo `gemini-2.5-flash`, que determina para cada uno la marca, las features UVL correspondientes, las especificaciones y si el producto es un componente estándar de sobremesa (descartando portátiles, workstations, etc.). El resultado se guarda en `catalogo_real.json`.

Importación: `import_catalogo.py` valida el catálogo scrapeado (categorías, features, precios), guarda una copia de seguridad del catálogo activo (`catalogo.json`) en `catalogo_backup.json` y lo sustituye por `catalogo_real.json` como nueva fuente de datos. Opcionalmente importa el catálogo a MongoDB Atlas mediante `seed_database()`

Cada producto almacena entre otros atributos, una lista de features que corresponden directamente a características del modelo UVL, lo que permite al motor de configuración filtrar y propagar restricciones sobre datos reales de mercado.

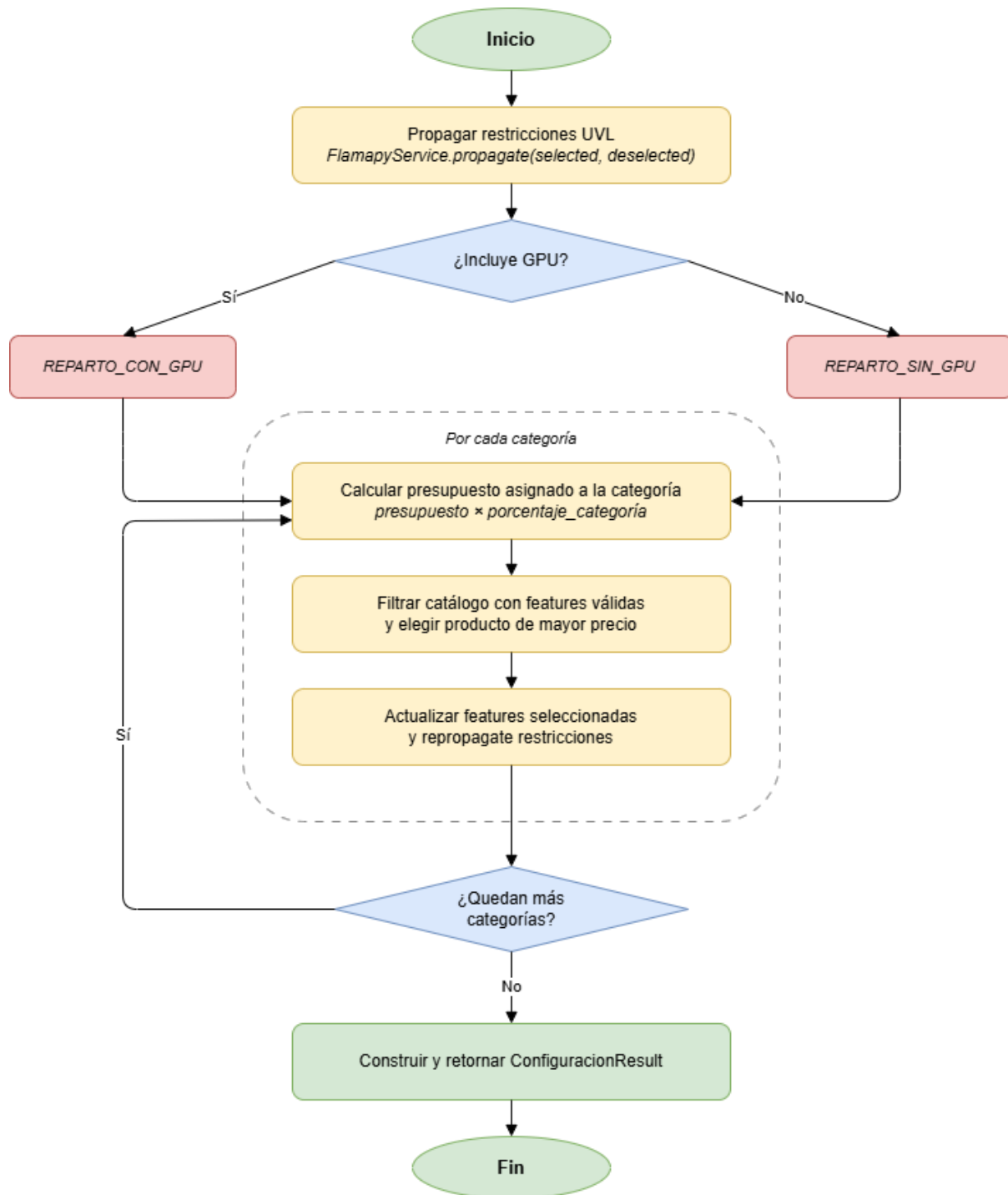
El servicio db.py intenta conectarse a MongoDB Atlas al arrancar. Si la conexión falla, el sistema cae automáticamente en modo fallback sirviendo los productos desde el fichero catalogo.json local. Esto garantiza que el sistema sea funcional en cualquier entorno de desarrollo sin dependencia de red.

8.1.3 Motor de configuración

El fichero configurador_service.py implementa el motor de generación automática. El algoritmo generar_por_presupuesto() sigue estos pasos:

1. Propaga restricciones con FlamaPy para obtener las features válidas.
2. Determina si la configuración incluye GPU.
3. Para cada categoría, calcula el presupuesto asignado según el porcentaje correspondiente.
4. Filtra el catálogo con las features válidas y elige el producto de mayor precio dentro del presupuesto (estrategia de máximo rendimiento).
5. Tras seleccionar cada producto, actualiza las features seleccionadas y repropaga para que las elecciones siguientes sean coherentes.

El diagrama de flujo siguiente ilustra el recorrido completo del algoritmo:

Ilustración 17: Diagrama de flujo del algoritmo `generar_por_presupuesto()`

El reparto del presupuesto entre categorías constituye una decisión de diseño del sistema. Los porcentajes no proceden de un estudio de mercado propio, pero son coherentes con las recomendaciones habituales de las guías de montaje de PC, que en equipos orientados a juego priorizan la tarjeta gráfica en torno al 35-40% del presupuesto, seguida del procesador (20-25%), la placa base, la memoria y el almacenamiento (8-15% cada uno) y, con menor peso, la fuente de alimentación, el chasis y la refrigeración (5-8% cada uno) (BuiltToFrag, s.f.; PerfGamer, s.f.).

Categoría	Con GPU	Sin GPU
Procesador	20%	30%
Tarjeta Gráfica	35%	—
Placa Base	10%	15%
Memoria RAM	8%	15%
Almacenamiento	7%	12%
Fuente de Alimentación	6%	10%
Chasis	7%	10%
Refrigeración	7%	8%

Tabla 27: Porcentajes de reparto en componentes

El siguiente extracto muestra el bucle central del algoritmo, donde se combina la asignación de presupuesto por categoría con la repropagación de restricciones tras cada elección:

```
def generar_por_presupuesto(presupuesto, selected=None, deselected=None):
    # 1. Propagar restricciones iniciales
    prop = flamapy_service.propagate(selected, deselected)
    features_validas = set(prop["forced"] + prop["selectable"])

    # 2. Seleccionar tabla de porcentajes según presencia de GPU
    incluye_gpu = "TarjetaGrafica" in features_validas
    reparto = REPARTO_CON_GPU if incluye_gpu else REPARTO_SIN_GPU

    # 3. Iterar por categoría
    componentes = []
    for categoria, porcentaje in reparto.items():
        presupuesto_cat = presupuesto * porcentaje

        # 4. Elegir el producto de mayor precio dentro del presupuesto
        producto = _elegir_producto(categoria, features_validas,
        presupuesto_cat)

        if producto:
            componentes.append({"categoria": categoria, "producto":
            producto})

    # 5. Repropagate con las features del producto elegido
```

```
nuevas_selected = list(set(selected) |
set(producto["features"]))
prop = flamapy_service.propagate(nuevas_selected, deselected)
features_validas = set(prop["forced"] + prop["selectable"])
selected = nuevas_selected

return {"componentes": componentes, ...}
```

Código 2: Algoritmo generar_por_presupuesto

8.1.4 Servicio Gemini

El servicio `gemini_service.py` encapsula la integración con la API de Google Gemini (Google, 2026) usando la librería oficial `google-genai`. La función `interpretar_consulta(consulta)` construye una petición al modelo `gemini-2.5-flash` con las features UVL válidas y reglas de interpretación (p.ej. "jugar" → perfil Gaming, "4K ultra" → GPU_Gama_Alta).

Gemini devuelve un JSON con los campos `perfil`, `presupuesto`, `selected`, `deselected`, explicación y entendida (el campo `entendida` será `false` cuando la consulta sea texto sin sentido, spam o completamente irrelevante para PCs.). Antes de usar la respuesta, el servicio la sintetiza eliminando cualquier feature que no pertenezca al conjunto de features válidas.

La ruta POST `/api/configuracion/consulta` combina la respuesta de Gemini con los presupuestos por defecto por perfil (Gaming: 1200 €, Edición: 1500 €, Programación: 1000 €, Ofimática: 600 €) cuando el usuario no especifica cantidad. Después, fusiona las features del perfil detectado con las extraídas por Gemini y delega en `generar_por_presupuesto()` para producir la configuración final.

El siguiente extracto muestra cómo se llama a Gemini y cómo se sintetiza su respuesta antes de usarla:

```
def interpretar_consulta(consulta: str) -> dict:
    client = genai.Client(api_key=api_key)
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=consulta,
        config=types.GenerateContentConfig(
            system_instruction=SYSTEM_PROMPT,
            temperature=0.2,
        ),
    )
    result = json.loads(response.text.strip())

    # Eliminar cualquier feature que no exista en el modelo UVL (se
sintetiza la respuesta)
    valid_set = set(VALID_FEATURES)
    result["selected"] = [f for f in result.get("selected", []) if f in
valid_set]
    result["deselected"] = [f for f in result.get("deselected", []) if f
in valid_set]

    return result
```

Código 3: Llamada a API de Gemini con interpretar_consulta

8.2 Frontend

El frontend está desarrollado con React 19, Vite como bundler³⁰ y servidor de desarrollo, Material UI 7 para los componentes visuales y Zustand 5 para la gestión del estado global.

8.2.1 Flujo del asistente

La aplicación dispone de dos puntos de entrada desde la página de inicio:

- "Configurar con IA": redirige a la pantalla de consulta por IA, donde el usuario describe sus necesidades en lenguaje natural.
- "Configurar manualmente": redirige directamente al asistente de 10 pasos.



Ilustración 18: Puntos de entrada de la aplicación desde la página de inicio

El asistente de 10 pasos se conforma de la siguiente manera:

- 1. Perfil: selección de perfil de uso y presupuesto.
- 2–9. Componentes: un paso por cada categoría (Procesador, Placa Base, RAM, GPU, Almacenamiento, Fuente, Chasis, Refrigeración).
- 10. Resumen: configuración completa con precio total, comparativa con el presupuesto, posibilidad de modificar la configuración y botones de exportación.

8.2.2 Estado global con Zustand

El store `useConfigStore` centraliza el estado de la configuración e implementa la lógica de negocio del frontend. La razón de usar un store global en lugar de estado local de React es que el configurador consta de 10 pasos independientes que comparten la misma configuración en construcción: sin un store, cada paso tendría que recibir toda la selección acumulada o realizar peticiones redundantes al backend. Con Zustand, cualquier paso puede leer o modificar el estado directamente:

- **`loadProductos(categoria)`**: llama al endpoint de propagación usando las features de los componentes ya elegidos en otras categorías (no la actual), evitando que la selección previa restrinja innecesariamente las opciones de esa misma categoría.

³⁰ Un bundler es una herramienta que empaqueta los módulos JavaScript, CSS y otros recursos de la aplicación en archivos listos para ser servidos en el navegador. Vite destaca por su servidor de desarrollo con recarga instantánea (Hot Module Replacement) y su rapidez en el proceso de build.

- **elegirProducto(categoria, producto):** soporta desección (clic sobre el mismo producto para quitarlo) y selección múltiple para Almacenamiento, en esta categoría, el estado almacena una lista de productos, de forma que el usuario puede combinar varios discos (p.ej. NVMe + HDD).
- **generarConfigAuto():** llama a POST /api/configuracion/perfil, rellena el estado con el resultado y salta directamente al paso de resumen.
- **consultarIA(consulta):** llama a POST /api/configuracion/consulta, interpreta la respuesta de Gemini, rellena el estado completo (componentes, perfil, presupuesto) y salta al resumen.

El siguiente extracto muestra la lógica de elegirProducto(), que gestiona tanto la desección (clic sobre el mismo producto) como la selección múltiple en la categoría Almacenamiento:

```
elegirProducto: async (categoria, producto) => {
  if (categoria === "Almacenamiento") {
    // Almacenamiento admite varios productos simultáneamente
    const current = state.componentesElegidos["Almacenamiento"] ?? [];
    const yaElegido = current.some((p) => p.nombre === producto.nombre);
    // Clic sobre el mismo producto lo elimina; clic sobre uno nuevo lo
    añade
    const newArray = yaElegido
      ? current.filter((p) => p.nombre !== producto.nombre)
      : [...current, producto];
    newComponentes = { ...state.componentesElegidos, Almacenamiento:
    newArray };
  } else {
    // Para el resto de categorías: clic sobre el mismo producto lo
    deselecciona
    const yaElegido = state.componentesElegidos[categoria]?.nombre ===
    producto.nombre;
    if (yaElegido) {
      newComponentes = Object.fromEntries(
        Object.entries(state.componentesElegidos).filter(([cat]) => cat
        !== categoria)
      );
    } else {
      newComponentes = { ...state.componentesElegidos, [categoria]:
      producto };
    }
  }
  // Repropagate con las features de todos los componentes elegidos
  const newSelected = Object.values(newComponentes).flat().flatMap((p) =>
  p.features);
  set({ componentesElegidos: newComponentes });
  await get().loadProductos(categoria);
},
```

Código 4: Lógica de elegirProducto() en Frontend

8.2.3 Indicador de gasto

Cada vez que el usuario avanza a un paso de selección, el sistema muestra un indicador de gasto acumulado con el coste de los componentes ya elegidos frente al presupuesto escogido. Si no hay presupuesto, se muestra únicamente el gasto acumulado de los componentes elegidos.

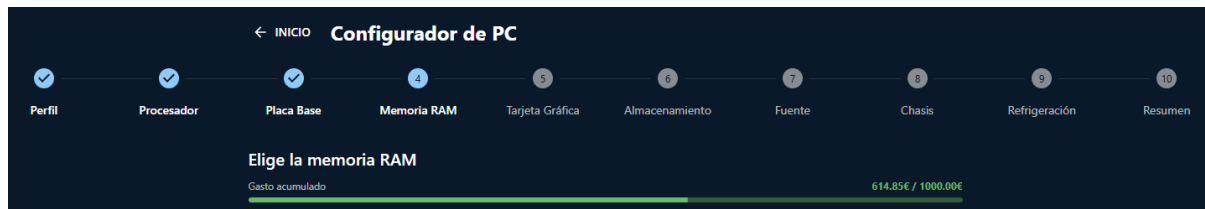


Ilustración 19: Gasto acumulado de los componentes elegidos

8.2.4 Notificaciones educativas

El módulo `utils/educationalMessages.js` implementa un sistema de reglas que genera mensajes explicativos en cada paso del configurador. Cuando FlamaPy excluye una feature, el sistema comprueba si la causa es una elección del usuario, para mostrar así solo mensajes relevantes y evitar avisos fuera de contexto.

Por ejemplo, cuando el usuario elige una CPU AMD de gama alta, FlamaPy excluye `Refrigeracion_Aire`. El sistema garantiza que el mensaje "Los procesadores de gama alta generan mucho calor. La refrigeración líquida es necesaria para mantener temperaturas seguras y estables." solo aparezca si el usuario eligió efectivamente esa CPU, y no en otros contextos.

Cada regla especifica qué features deben estar excluidas (`anyOf/allOf`), cuáles no deben estarlo (`noneOf`) y qué features deben estar seleccionadas para que el mensaje sea relevante (`requiresAnySelected`). El siguiente extracto muestra algunas reglas y la función que las evalúa:

```
const RULES = {
  Refrigeracion: [
    {
      anyOf: ["Refrigeracion_Aire"],
      requiresAnySelected: ["Intel_Gama_Alta", "AMD_Gama_Alta"],
      message: "Los procesadores de gama alta generan mucho calor. " +
        "La refrigeración líquida es necesaria para mantener"
    },
  ],
  MemoriaRAM: [
    {
      anyOf: ["DDR4"],
      requiresAnySelected: ["AM5", "LGA1851"],
      message: "Tu procesador y placa base solo admiten memoria DDR5.",
    },
  ],
};

export function getMessages(categoria, excluded, selected = []) {
  const excludedSet = new Set(excluded);
  const selectedSet = new Set(selected);

  return (RULES[categoria] ?? []).filter((rule) => {
    const triggered = rule.allOf
      ? rule.allOf.every((f) => excludedSet.has(f))
      : rule.anyOf.some((f) => excludedSet.has(f));
    const blocked = rule.noneOf?.some((f) => excludedSet.has(f)) ?? false;
    const requirementMet = !rule.requiresAnySelected ||
      rule.requiresAnySelected.some((f) => selectedSet.has(f));
  });
}
```

```
return triggered && !blocked && requirementMet;
}).map((rule) => rule.message);
}
```

Código 5: Reglas y función que evalúa si un mensaje debe aparecer o no

 Los procesadores de gama alta generan mucho calor. La refrigeración líquida es necesaria para mantener temperaturas seguras y estables.

Ilustración 20: Ejemplo de notificación educativa

8.2.5 Comparador de precios

El componente PriceLinks.jsx muestra en cada tarjeta de producto dos iconos (PcComponentes y Amazon), con color distintivo y etiqueta (PCC / AMZ). Las tiendas se definen en una estructura de datos "STORES" en la que cada entrada aporta su etiqueta, su color y una función que construye la URL de búsqueda, así el comparador es extensible (añadir una tienda nueva solo requiere agregar un objeto al array, sin tocar el resto de la lógica).

Cada enlace es una búsqueda por nombre, no un enlace directo a la ficha del producto: como el catálogo procede del scraping de Coolmod, no existe una correspondencia garantizada con una página concreta de cada tienda, por lo que la búsqueda por el nombre resulta más robusta. Al hacer clic, abre la búsqueda en una pestaña nueva. Además, e.stopPropagation() evita que el clic active también la selección del producto.



Ilustración 21: Iconos de navegación a PcComponentes o Amazon

8.2.6 Modificación de configuración generada

El paso de resumen StepResumen.jsx presenta cada componente elegido en una tarjeta, calcula el precio total y lo contrasta con el presupuesto, mostrándolo en verde o rojo junto a un aviso con el ahorro o el exceso. Desde aquí el usuario dispone de cuatro acciones: exportar a PDF, exportar a JSON, Modificar configuración y Nueva configuración.

La diferencia clave está entre las dos últimas: "Nueva configuración" llama a reset() y vacía todo. "Modificar configuración" invoca goToStep(1), que devuelve al primer paso de componentes sin reiniciar el estado, por lo que los componentes ya elegidos se conservan. Gracias a ello, una configuración generada automáticamente puede ajustarse componente a componente. El usuario sustituye cualquier pieza y como el asistente repropaga las restricciones en cada paso (sección 7.2 Propagación de restricciones), el resto de la configuración permanece coherente. Se combinan así la comodidad de la generación automática y el control de la configuración manual.



Ilustración 22: Acciones del paso resumen

8.2.7 Exportación de configuraciones

El módulo utils/exportConfig.js ofrece dos funciones de exportación accesibles desde el paso de resumen:

- **exportJSON:** genera un archivo .json con la configuración completa (fecha, perfil, presupuesto, lista de componentes con nombre, precio y features).
- **exportPDF:** genera un archivo .pdf usando la librería **jsPDF** con el resumen de la configuración, incluyendo la lista de componentes y el precio total.



Configuración de PC

Generado el 9/5/2026

Perfil: Gaming

Presupuesto: 1200.00 €

Componentes seleccionados

PROCESADOR	AMD Ryzen 9 9950X3D 5.7GHz Socket AM5 Tray	649.95 €
PLACABASE	Asus Prime B650M-A II-CSM Socket AM5	116.95 €
MEMORIARAM	Corsair Vengeance 16GB 2x8GB 5200MHz CL40	199.95 €

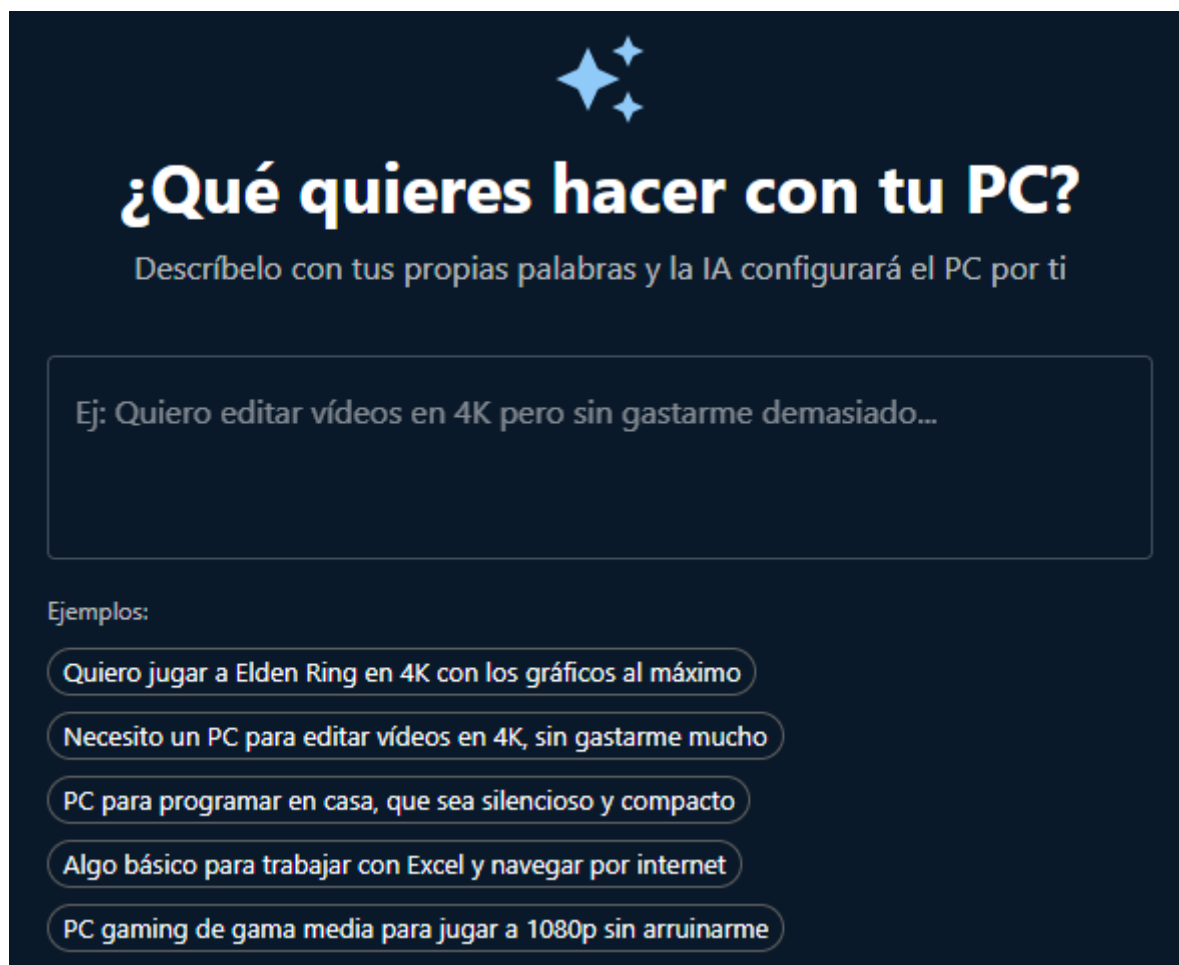
Ilustración 23: PDF generado tras una configuración

8.2.8 Integración con Google Gemini

La página ConsultarIA.jsx proporciona una interfaz de texto libre donde el usuario puede describir sus necesidades. Al enviar la consulta:

1. El frontend llama a consultarIA(consulta) del store.
2. El store hace POST /api/configuracion/consulta.
3. El backend llama a GeminiService.interpretar_consulta(), que envía la consulta a gemini-2.5-flash.
4. Gemini devuelve un JSON con perfil, presupuesto, selected, deselected, explicación y entendida.
5. El backend fusiona las features del perfil detectado con las extraídas por Gemini y llama al motor de configuración.
6. El frontend rellena el store y redirige al resumen, mostrando la explicación generada por Gemini.

La integración maneja el caso en que la API de Gemini no está disponible devolviendo HTTP 503, que el frontend muestra como un Alert de error.



¿Qué quieres hacer con tu PC?

Descríbelo con tus propias palabras y la IA configurará el PC por ti

Ej: Quiero editar vídeos en 4K pero sin gastarme demasiado...

Ejemplos:

- Quiero jugar a Elden Ring en 4K con los gráficos al máximo
- Necesito un PC para editar vídeos en 4K, sin gastarme mucho
- PC para programar en casa, que sea silencioso y compacto
- Algo básico para trabajar con Excel y navegar por internet
- PC gaming de gama media para jugar a 1080p sin arruinarme

Ilustración 24: Vista del apartado de la aplicación "Consulta por IA"

9. Pruebas

El objetivo de las pruebas es verificar que el sistema cumple los requisitos funcionales y no funcionales definidos, con especial atención al requisito RNF-04, que establece una cobertura de código mínima del 80% sobre el backend.

La estrategia se organiza en dos niveles:

- **Unitario e integración (backend):** pruebas automatizadas sobre los servicios y endpoints de la API REST, verificando la lógica de negocio, la integración con FlamaPy y el comportamiento ante dependencias externas (MongoDB, Gemini).
- **Nivel end-to-end:** pruebas sobre la interfaz de usuario que simulan la interacción real de un usuario con la aplicación, verificando los flujos completos de configuración.

Las herramientas seleccionadas para cada nivel son:

Nivel	Herramienta	Propósito
Backend	pytest ³¹ + httpx ³²	Ejecución de tests y cliente HTTP en memoria.
Backend	pytest-cov ³³ + .coveragerc ³⁴	Medición de cobertura de código.
Backend	unittest.mock.patch ³⁵	Simulación de servicios externos (MongoDB, Gemini).
Sistema	Playwright ³⁶ + Chromium ³⁷	Tests E2E con navegador real.
Sistema	page.route() ³⁸	Intercepción de llamadas al backend.

Tabla 28: Herramientas para la realización de pruebas

³¹ pytest es el framework de pruebas automatizadas más utilizado en el ecosistema Python.

³² httpx es un cliente HTTP para Python que, en el contexto de tests con FastAPI, permite realizar peticiones a la aplicación sin levantar un servidor real, ejecutando todo en memoria.

³³ pytest-cov es un plugin de pytest que mide la cobertura de código durante la ejecución de los tests, indicando qué líneas y ramas del código fuente han sido ejercitadas.

³⁴ .coveragerc es el archivo de configuración de la herramienta de cobertura. Permite especificar qué ficheros incluir o excluir del análisis.

³⁵ unittest.mock.patch es una utilidad de la biblioteca estándar de Python que permite sustituir temporalmente cualquier objeto o función por un simulacro (*mock*) durante la ejecución de un test, sin modificar el código de producción.

³⁶ Playwright es un framework de automatización de navegadores que permite controlarlos mediante código, siendo ampliamente utilizado para tests end-to-end.

³⁷ Chromium es el motor de navegador de código abierto en el que se basan Google Chrome y Microsoft Edge, entre otros.

³⁸ page.route() es una función de Playwright que intercepta peticiones de red realizadas por el navegador durante el test, permitiendo devolver respuestas simuladas sin que la petición llegue al servidor real.

El ámbito de las pruebas cubre toda la lógica de aplicación del backend (servicios, rutas y modelos) y los flujos principales del frontend. Se excluyen expresamente los scripts de utilidad `scraper_coolmod.py` e `import_catalogo.py`, que no forman parte de la API y dependen de recursos externos (web scraping, sistema de ficheros).

9.1 Pruebas implementadas y resultados

Se implementaron un total de 71 tests (56 de backend y 15 E2E), todos superados satisfactoriamente. La cobertura del código de aplicación del backend alcanza el 97%, superando el requisito RNF-04. La siguiente tabla resume los módulos de prueba, el número de tests de cada uno y el resultado obtenido:

Módulo	Tests	Resultado
test_features.py — Modelo de Características	12	✓ Todos superados
test_productos.py — Catálogo	9	✓ Todos superados
test_configuracion.py — Motor de configuración	20	✓ Todos superados
test_db.py — Servicio de base de datos	7	✓ Todos superados
test_gemini.py — Servicio Gemini	7	✓ Todos superados
test_integration.py — Integración FlamaPy + Catálogo	1	✓ Superado
landing.spec.js — Página de inicio (E2E)	4	✓ Todos superados
consulta-ia.spec.js — Consulta por IA (E2E)	5	✓ Todos superados
configurador.spec.js — Wizard de configuración (E2E)	6	✓ Todos superados
Total	71	✓ 71/71

Tabla 29: Número de pruebas realizadas

Name	Stmts	Miss	Cover	Missing
-----	-----	-----	-----	-----
main.py	15	0	100%	
models__init__.py	0	0	100%	
models\producto.py	25	0	100%	
models\schemas.py	24	0	100%	
routes__init__.py	0	0	100%	
routes\configuracion.py	50	2	96%	99-104
routes\features.py	28	0	100%	
routes\productos.py	14	0	100%	
services__init__.py	0	0	100%	
services\configurador_service.py	93	7	92%	80, 103, 134-135, 179, 183, 197
services\db.py	48	0	100%	
services\flamapy_service.py	87	2	98%	105, 141
services\gemini_service.py	34	0	100%	
-----	-----	-----	-----	-----
TOTAL	418	11	97%	
===== 56 passed in 133.12s (0:02:13) =====				

Ilustración 25: Porcentaje de cobertura en backend

Running 15 tests using 4 workers	
✓ 1	...r manual > seleccionar el perfil Gaming lo marca como seleccionado (1.8s)
✓ 2	...Configurador manual > muestra el stepper con el paso Perfil activo (1.5s)
✓ 3	...Configurador manual > el checkbox de presupuesto muestra el slider (1.7s)
✓ 4	...urador manual > generar configuración automática navega al resumen (2.3s)
✓ 5	...nual > el resumen muestra el componente generado y el precio total (1.6s)
✓ 6	...figurador manual > el botón Siguiente avanza al paso de Procesador (1.2s)
✓ 7	...js:29:3 > Consulta por IA > renderiza el formulario correctamente (628ms)
✓ 8	...3 > Consulta por IA > el botón está deshabilitado si no hay texto (675ms)
✓ 9	... > Consulta por IA > clic en un ejemplo rellena el campo de texto (948ms)
✓ 10	...consulta exitosa muestra la explicación y redirige al configurador (2.9s)
✓ 11	...spec.js:61:3 > Consulta por IA > error de API muestra una alerta (908ms)
✓ 12	...ng.spec.js:8:3 > Página de inicio > muestra el título y subtítulo (620ms)
✓ 13	... Página de inicio > muestra las cinco tarjetas de características (558ms)
✓ 14	...ágina de inicio > el botón "Configurar con IA" navega a /consulta (604ms)
✓ 15	...e inicio > el botón "Configurar manualmente" navega a /configurar (533ms)
15 passed (10.1s)	

Ilustración 26: Pruebas E2E

A continuación, se describe el contenido de cada módulo de prueba.

9.1.1 Tests del Modelo de Características (test_features.py)

Este módulo cubre el núcleo del sistema: la integración con FlamaPy y la correcta propagación de restricciones UVL. Es el conjunto de tests más crítico, ya que cualquier error en la lógica SAT afectaría a todos los demás componentes que dependen de ella. Se organiza en cuatro clases:

Clase	Tests	Qué verifica
TestFeatureTree	2	Estructura del árbol UVL (raíz y tipos de relación).

TestValidate	4	Validación de configuraciones parciales.
TestPropagate	5	Propagación de restricciones SAT.
TestInfo	1	Satisfacibilidad e integridad del modelo.

Tabla 30: Pruebas por clase en test_features.py

TestFeatureTree verifica que el endpoint `/api/features/tree` devuelve el árbol de características con `PC_Custom` como raíz y que contiene los tipos de relación esperados (mandatory y optional).

TestValidate comprueba el endpoint de validación de configuraciones parciales: una combinación válida como AMD, AM5, DDR5 es aceptada, una combinación inválida como AMD, LGA1700 (socket Intel con procesador AMD) es rechazada, y una feature desconocida devuelve HTTP 400 con un mensaje descriptivo.

TestPropagate verifica la propagación de restricciones: seleccionar AMD excluye Intel, LGA1700 y LGA1851. Seleccionar AMD, AM5 fuerza DDR5 y excluye DDR4. Seleccionar Gaming fuerza TarjetaGrafica y excluye GPU_Gama_Baja y RAM_8GB. Features desconocidas o en ambas listas simultáneamente devuelven HTTP 400.

TestInfo verifica que el endpoint de información del modelo devuelve el modelo como satisfacible, sin características muertas y con `PC_Custom` entre las características obligatorias.

El siguiente extracto muestra el test de propagación más representativo, que verifica que seleccionar el perfil Gaming fuerza la inclusión de GPU y excluye las opciones de gama baja y 8GB de RAM:

```
def test_propagate_gaming_forces_gpu(self):
    r = client.post(
        "/api/features/propagate",
        json={"selected": ["Gaming"], "deselected": []},
    )
    data = r.json()
    assert "TarjetaGrafica" in data["forced"]
    assert "GPU_Gama_Baja" in data["excluded"]
    assert "RAM_8GB" in data["excluded"]
```

Código 6: Ejemplo de test de propagación

9.1.2 Tests del Catálogo (test_productos.py)

Este módulo garantiza que el catálogo real de productos, obtenido mediante scraping de Coolmod, es accesible y filtrable correctamente a través de la API. A diferencia de otros módulos que usan datos simulados, estos tests trabajan directamente con los productos reales, lo que permite detectar inconsistencias entre el catálogo y el modelo UVL en condiciones de producción. Se organiza en tres clases:

Clase	Tests	Qué verifica
TestListarProductos	6	Filtrado del catálogo por categoría, features y combinaciones.

TestCategorias	1	El endpoint devuelve las 8 categorías del sistema.
TestIntegracionFlamapyProductos	2	Combinación de propagación FlamaPy y filtrado de catálogo.

Tabla 31: Pruebas por clase en test_productos.py

TestListarProductos cubre los casos de filtrado del endpoint `/api/productos/`: consulta del catálogo completo, filtrado por categoría (verificando que todos los productos devueltos pertenecen a la categoría solicitada), filtrado por features (p.ej. AMD,AM5 devuelve solo productos que tienen ambas features), combinación de categoría y features, categoría inexistente devuelve lista vacía, y features incompatibles como AMD, LGA1700 devuelven cero resultados.

TestCategorias verifica que el endpoint de categorías devuelve las 8 categorías del sistema (Procesador, PlacaBase, MemoriaRAM, TarjetaGrafica, Almacenamiento, FuenteAlimentacion, Chasis y Refrigeracion).

TestIntegracionFlamapyProductos combina propagación FlamaPy con filtrado de catálogo: dado un perfil Gaming con AMD y AM5, comprueba que las GPUs de gama baja quedan excluidas por el modelo y que el filtrado por socket y tipo de RAM devuelve únicamente productos compatibles.

9.1.3 Tests del Motor de Configuración (test_configuracion.py)

Es el módulo más extenso, organizado en cuatro clases que cubren los distintos endpoints y escenarios del motor de configuración.

Clase	Tests	Qué verifica
TestPresupuesto	5	Generación de configuraciones por presupuesto.
TestPerfil	5	Generación de configuraciones por perfil de uso.
TestConsultaIA	8	Endpoint de consulta en lenguaje natural con IA.
TestCoherencia	2	Validez FlamaPy de las configuraciones generadas.

Tabla 32: Pruebas por clase en test_configuracion.py

TestPresupuesto verifica el endpoint `/api/configuracion/presupuesto`: que con 1000 € se genera una configuración con al menos 7 componentes, que un presupuesto inferior al mínimo devuelve HTTP 400, que al especificar `selected: ["AMD", "AM5"]` todos los componentes generados son compatibles con ese socket, que un perfil Gaming siempre incluye GPU de gama media o alta, y que cuando el presupuesto es insuficiente la respuesta incluye un aviso descriptivo en lugar de fallar.

TestPerfil verifica el endpoint `/api/configuracion/perfil`: Gaming genera configuración con GPU, Ofimática la genera sin GPU, se puede especificar un presupuesto personalizado, un perfil

desconocido devuelve HTTP 400, y el endpoint de listado devuelve los cuatro perfiles disponibles (Gaming, Edición, Programación, Ofimática).

TestConsultaIA verifica el endpoint `/api/configuracion/consulta` utilizando `unittest.mock.patch` para evitar llamadas reales a Gemini. Cubre tres de los cuatro perfiles principales (Gaming con GPU, Edición con RAM ≥ 32 GB, Ofimática sin GPU), consulta vacía (HTTP 400), servicio Gemini no disponible (HTTP 503), presencia del campo explicación en la respuesta, y la validación final de que las features de los componentes generados son aceptadas por FlamaPy.

```
with patch("services.gemini_service.interpretar_consulta",
return_value=MOCK_GAMING):
    r = client.post("/api/configuracion/consulta", json={"consulta":
"Gaming 1080p"})
assert r.status_code == 200
assert r.json()["perfil"] == "Gaming"
```

Código 7: Ejemplo de test `interpretar_consulta`

El caso más relevante es el de restricciones contradictorias: cuando Gemini devuelve `selected: ["Gaming"]`, `deselected: ["TarjetaGrafica"]`, la sintetización UVL del backend detecta que TarjetaGrafica es obligatoria por la constraint Gaming $\Rightarrow$ TarjetaGrafica, la elimina de `deselected` y la configuración resultante siempre incluye GPU.

TestCoherencia comprueba, para los perfiles Gaming y Ofimática, que las features de todos los componentes generados forman una configuración válida según FlamaPy, enviándolas al endpoint `/api/features/validate` y verificando que el resultado es válido.

9.1.4 Tests del servicio de base de datos (`test_db.py`)

Pruebas unitarias de `services/db.py` organizadas en tres clases:

Clase	Tests	Qué verifica
TestFallbackJSON	3	Fallback al catálogo local cuando MongoDB no está disponible.
TestGetMongoCollection	2	Ramas de la función de conexión a MongoDB.
TestSeedDatabase	2	Inserción y mantenimiento del catálogo en MongoDB.

Tabla 33: Pruebas por clase en `test_db.py`

```
def test_listar_todos_sin_mongodb(self):
    with patch("services.db._collection", None):
        r = client.get("/api/productos/")
        assert r.status_code == 200
        assert r.json()["total"] > 0
```

Código 8: Verifica que funciona usando el catálogo JSON cuando MongoDB no está disponible

TestFallbackJSON simula que MongoDB no está disponible, se verifica que la API devuelve resultados correctos (listado completo, filtrado por categoría y por features) usando el catálogo local.

TestGetMongoCollection verifica las ramas de la función de conexión: que sin URI configurada devuelve None y que ante un fallo de conexión (timeout) también devuelve None sin lanzar excepción.

TestSeedDatabase verifica que seed_database borra los datos existentes e inserta el catálogo correctamente usando una colección mockeada, y que lanza RuntimeError cuando MongoDB no está disponible.

9.1.5 Tests del servicio Gemini (test_gemini.py)

Pruebas unitarias de services/gemini_service.py agrupadas en la clase TestInterpretarConsulta, que mockea el cliente de Google Gemini para cubrir todas las ramas de interpretar_consulta() sin realizar llamadas reales a la API:

Test	Qué verifica
Sin API key	Lanza RuntimeError sin contactar con Gemini.
Respuesta válida	Parsea correctamente la respuesta.
Features inventadas	Son eliminadas en la sintetización ("FeatureInventada" no aparece en el resultado).
Markdown con json	La respuesta envuelta en json se procesa correctamente.
Markdown sin indicador	La respuesta envuelta en un bloque de código sin indicador también se procesa correctamente.
JSON inválido	Lanza RuntimeError con mensaje descriptivo.
Fallo de red	Lanza RuntimeError con el error original incluido.

Tabla 34: Pruebas de test_gemini.py

9.1.6 Tests E2E de interfaz (e2e/)

Tests end-to-end desarrollados con **Playwright** que verifican los flujos completos de la interfaz lanzando un navegador Chromium real. Las llamadas al backend se interceptan con page.route() para que los tests sean autocontenidos:

landing.spec.js (4 tests): verifican que la página de inicio muestra el título, las cinco tarjetas de características y que los botones de navegación redirigen correctamente a la pantalla de consulta por IA o la pantalla de configuración manual.

consulta-ia.spec.js (5 tests): verifican el formulario de consulta en lenguaje natural. Que no puede enviarse vacío, que los ejemplos de consulta predefinidos rellenan el campo, que una consulta exitosa

muestra la explicación de Gemini y redirige al resumen, y que un error de la API se muestra correctamente al usuario.

configurador.spec.js (6 tests): verifican el configurador manual. Que se muestra correctamente al entrar, que se puede seleccionar un perfil, que el campo de presupuesto se activa al marcarlo, que la generación automática produce un resumen con componentes y precio total, y que el botón Siguiente avanza al paso siguiente.

10. Despliegue

El sistema se pone en producción mediante una infraestructura basada en contenedores Docker desplegados sobre Render, con un proceso de integración y despliegue continuos (CI/CD) automatizado con GitHub Actions. Se definen dos entornos independientes (preproducción y producción) que permiten validar los cambios en un entorno real antes de exponerlos al usuario final. El entorno de producción es accesible públicamente en las siguientes URLs:

- **Frontend (aplicación web):** <https://configurador-de-pc.onrender.com>
- **Backend (API REST):** <https://configurador-de-pc-backend.onrender.com>

10.1 Estrategia de entornos

Cada entorno está vinculado a una rama del repositorio, de forma que el flujo de cambios (sección 11.1 Estrategia de ramas) se traduce directamente en despliegues:

Entorno	Rama	Propósito
Preproducción	develop	Validar cambios integrados antes de promocionarlos a producción
Producción	main	Versión estable accesible al usuario final

Tabla 35: Entornos existentes

Cada entorno se compone de dos servicios desplegados por separado en Render (el backend y el frontend), lo que suma cuatro servicios en total. Esta separación es consecuencia directa de la arquitectura desacoplada del sistema (sección 6.2 Arquitectura del sistema). Al exponerse backend y frontend como servicios independientes, cada uno se construye, escala y redespiega sin afectar al otro, cumpliendo el requisito RNF-06.

10.2 Contenerización con Docker

Ambos componentes se empaquetan en imágenes Docker, lo que garantiza que el entorno de ejecución sea idéntico en local y en Render y elimina las dependencias del sistema.

El backend (backend/Dockerfile) parte de la imagen oficial python:3.11-slim, instala las dependencias declaradas en requirements.txt y arranca el servidor con uvicorn:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Código 9: Dockerfile del Backend

El frontend (frontend/Dockerfile) emplea un build multietapa (multi-stage build)³⁹: una primera etapa basada en node:20-alpine compila la aplicación React con “npm run build”, y una segunda etapa basada en nginx:alpine sirve los archivos estáticos resultantes (/dist) por el puerto 80. De este modo, la imagen final no contiene Node ni las dependencias de desarrollo, sino únicamente un servidor web ligero con los archivos ya compilados.

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=http://localhost:8000
RUN VITE_API_URL=$VITE_API_URL npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Código 10: Dockerfile del Frontend

Un detalle relevante es el tratamiento de VITE_API_URL, la variable que indica al frontend dónde encontrar el backend. Al ejecutar docker build, Docker arranca la primera etapa del Dockerfile y ejecuta npm run build. Es en ese paso donde Vite sustituye VITE_API_URL por su valor real dentro del código JavaScript generado. Una vez que la compilación termina, el valor queda fijo en los archivos estáticos y no se puede cambiar sin reconstruir la imagen. Por eso VITE_API_URL se declara como argumento de docker build (ARG VITE_API_URL) y no como variable de entorno del contenedor: si se pasase como variable de entorno, el servidor (nginx) la recibiría al arrancar, pero en ese momento el código ya está compilado y no puede leerla. Cada entorno construye su propia imagen de frontend pasando la URL del backend correspondiente en el momento del docker build.

10.3 Servicios en Render

Cada uno de los cuatro servicios se configura en Render como Web Service de tipo Docker, indicando como “root directory” la carpeta backend o frontend según corresponda. La configuración por servicio es:

- **Backend:** variables de entorno MONGO_URI (conexión a MongoDB Atlas), GEMINI_API_KEY (clave de la API de Google Gemini) y FRONTEND_URL (dominio del frontend, usado para CORS, sección 10.5 Configuración de CORS).
- **Frontend:** argumento de construcción VITE_API_URL apuntando a la URL pública del backend.

El Auto-Deploy de Render se mantiene desactivado. En su lugar, el redespigie se dispara mediante deploy hooks⁴⁰ invocados desde los workflows de GitHub Actions (sección 10.4 Integración y despliegue continuos (CI/CD)).

³⁹ Un build multietapa (multi-stage build) de Docker emplea varias imágenes base dentro de un mismo Dockerfile: en una etapa se compila la aplicación y en otra, más ligera, solo se copia el resultado. Así la imagen final no incluye las herramientas de compilación, reduciendo su tamaño.

⁴⁰ Un deploy hook es una URL secreta proporcionada por Render que, al recibir una petición HTTP, dispara un nuevo despliegue del servicio asociado. Permite controlar el momento del despliegue desde sistemas externos como GitHub Actions.

Adicionalmente, el control de acceso de red de MongoDB Atlas se configura “0.0.0.0/0”, ya que las IPs de salida de los contenedores de Render no son fijas y deben poder conectarse a la base de datos.

10.4 Integración y despliegue continuos (CI/CD)

El proyecto cuenta con cinco workflows de GitHub Actions que automatizan la verificación de calidad y el despliegue, cumpliendo el requisito RNF-05:

Workflow	Disparador	Función
test-ci.yml	push (excepto en rama main)	Job test (pytest con cobertura) y job e2e (Playwright sobre Chromium)
lint-ci.yml	push (excepto en rama main)	Ejecuta pre-commit sobre todos los ficheros
codacy-ci.yml	push (excepto en rama main)	Análisis estático de seguridad y calidad con Codacy
deploy-preproduction.yml	test-ci.yml exitoso en develop	Despliega a preproducción solo si la CI fue satisfactoria
deploy-production.yml	push a main	Despliega a producción

Tabla 36: Workflows existentes

El despliegue a preproducción se encadena al workflow de pruebas mediante el disparador `workflow_run`: se ejecuta cuando `test-ci` finaliza sobre la rama `develop`, y solo despliega si la CI fue exitosa. De este modo, ningún cambio que rompa las pruebas llega siquiera al entorno de preproducción.

El despliegue a producción se dispara directamente con cada push a `main`. No repite la batería de pruebas porque estas ya se ejecutaron sobre `develop` antes de promocionar los cambios mediante el merge a `main` (sección 11.1 Estrategia de ramas). Se ha implementado un disparador adicional `workflow_dispatch` que permite además lanzar un despliegue manualmente desde la interfaz de GitHub sin necesidad de realizar un nuevo commit:

```
name: Deploy production (Render)

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Deploy backend
    run: curl -fsS "${{ secrets.RENDER_BACKEND_PROD_DEPLOY_HOOK }}"
  - name: Deploy frontend
    run: curl -fsS "${{ secrets.RENDER_FRONTEND_PROD_DEPLOY_HOOK }}"
```

Código 11: Workflow del disparador del despliegue de producción en Render

Las URLs de los deploy hooks se almacenan como secrets del repositorio en GitHub (RENDER_*_DEPLOY_HOOK), de forma que no quedan expuestas en el código fuente.

10.5 Configuración de CORS

Para que el navegador permita las peticiones del frontend al backend, el backend habilita CORS⁴¹. En lugar de codificar los dominios permitidos, backend/main.py construye la lista de orígenes combinando los orígenes locales de desarrollo con el valor de la variable de entorno FRONTEND_URL, que cada entorno define con su propio dominio:

```
_origins = ["http://localhost:5173", "http://localhost",
"http://localhost:80"]
_frontend_url = os.getenv("FRONTEND_URL", "")
if _frontend_url:
    _origins.append(_frontend_url)

app.add_middleware(
    CORSMiddleware,
    allow_origins=_origins,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Código 12: Configuración de CORS en el archivo main.py

Así, el mismo código fuente funciona sin modificaciones tanto en local como en preproducción y producción, bastando con definir FRONTEND_URL en cada servicio de Render.

10.6 Mecanismo de keep-alive

El plan gratuito de Render suspende automáticamente los servicios tras unos 15 minutos sin recibir tráfico. Cuando un usuario vuelve a acceder, el servicio debe rearrancar, provocando un arranque en frío (cold start)⁴² durante el cual el servidor responde con un error hasta quedar operativo. Para evitar esta degradación de la experiencia se establece un mecanismo de keep-alive consistente en realizar peticiones HTTP periódicas a las URLs de producción, manteniendo así los servicios siempre activos.

La solución definitiva se alcanzó tras evaluar dos alternativas previas para el backend, una de las cuales (UptimeRobot) sí resultó válida para el frontend:

⁴¹ CORS (*Cross-Origin Resource Sharing*) es un mecanismo de seguridad de los navegadores que restringe las peticiones HTTP entre orígenes (dominios) distintos. El servidor debe declarar explícitamente qué orígenes tienen permitido acceder a sus recursos.

⁴² El arranque en frío (cold start) es el retardo que se produce cuando un servicio previamente suspendido por inactividad debe rearrancar antes de poder atender una petición.

1. GitHub Actions: se descartó por su escasa fiabilidad. GitHub no garantiza que los workflows se ejecuten a la hora exacta. En repositorios con poca actividad, los disparos pueden retrasarse decenas de minutos o no producirse, lo que invalida su uso como mecanismo de ping periódico.
2. UptimeRobot: monitorizaba correctamente el frontend, pero marcaba reiteradamente el backend como caído pese a estar operativo, al no poder alcanzarlo desde sus servidores de comprobación.
3. Solución final: se emplea cron-job.org (ping cada 5 minutos) para el backend de producción y se mantiene UptimeRobot para el frontend, que sí funciona de forma estable sobre este.

11. Gestión del proyecto

El desarrollo del proyecto se ha gestionado con ayuda de las herramientas integradas en el ecosistema de GitHub (Issues, Projects y Actions). Este capítulo describe la estrategia de ramas, la gestión de tareas y los mecanismos de aseguramiento de la calidad.

11.1 Estrategia de ramas

El código se gestiona con Git y se aloja en GitHub (RNF-05). Al tratarse de un proyecto individual, no tiene sentido emplear flujos de revisión como Pull Requests. En su lugar se aplica una estrategia de ramas estructurada (estudiada en la asignatura Evolución y Gestión de la Configuración) que mantiene el historial ordenado y permite vincular cada tarea a su rama correspondiente. Se distinguen tres tipos de rama:

- **main**: rama de producción. Contiene siempre una versión estable y desplegable. Cada push a esta rama dispara el despliegue a producción (sección 10.4 Integración y despliegue continuos (CI/CD)).
- **develop**: rama de integración y preproducción. Recoge los cambios ya terminados y validados, y alimenta el entorno de preproducción.
- **feature/***: una rama por cada tarea o funcionalidad. Se crea a partir de develop, se desarrolla de forma aislada y, una vez completada, se fusiona en develop con git merge y se elimina.

Esta estructura se ilustra en la siguiente figura: develop acumula los merges de las ramas de tarea y alimenta el entorno de preproducción. Cuando el conjunto de cambios es estable, se fusiona en main, lo que dispara el despliegue a producción.

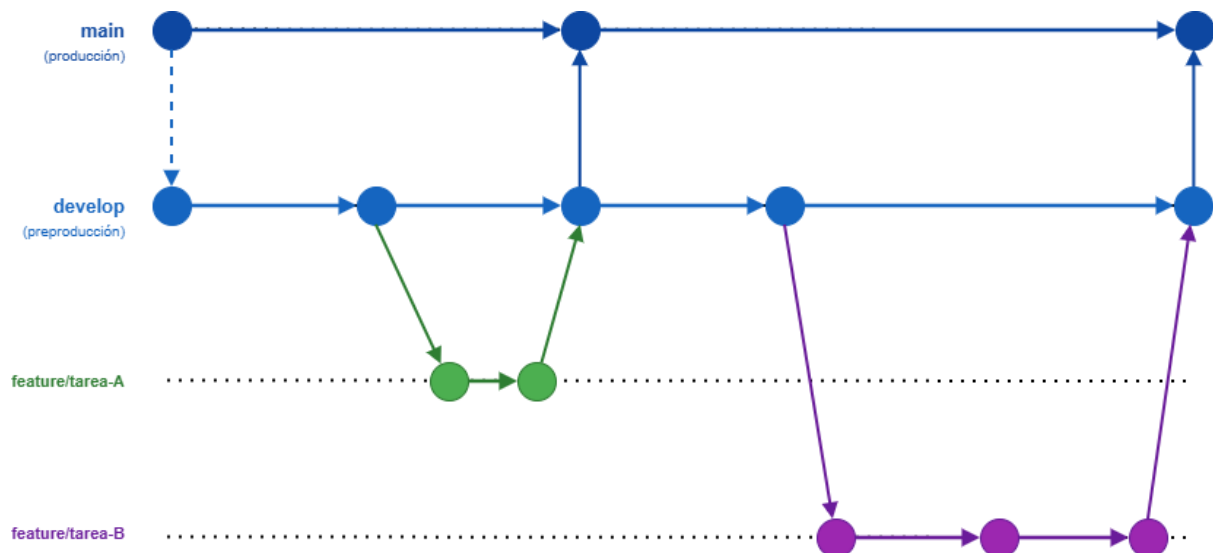


Ilustración 27: Estrategia de ramas

Esta estructura encaja directamente con la estrategia de entornos del apartado 10.1 Estrategia de entornos. Cada rama de larga vida mapea a un entorno desplegado: develop → preproducción, main → producción.

11.2 Issues y tablero Kanban

Cada tarea, mejora o corrección se registra como una GitHub Issue con un título descriptivo y una descripción. Las issues se organizan visualmente en un tablero Kanban gestionado con GitHub Projects, estructurado en tres columnas que reflejan el estado de cada tarea:

Todo → In Progress → Done

Las tareas avanzan por el tablero a medida que progresa su desarrollo. Cada issue se asocia a su rama `feature/*` correspondiente y se cierra manualmente al completar el merge, manteniendo así sincronizados el tablero y el estado real del repositorio.

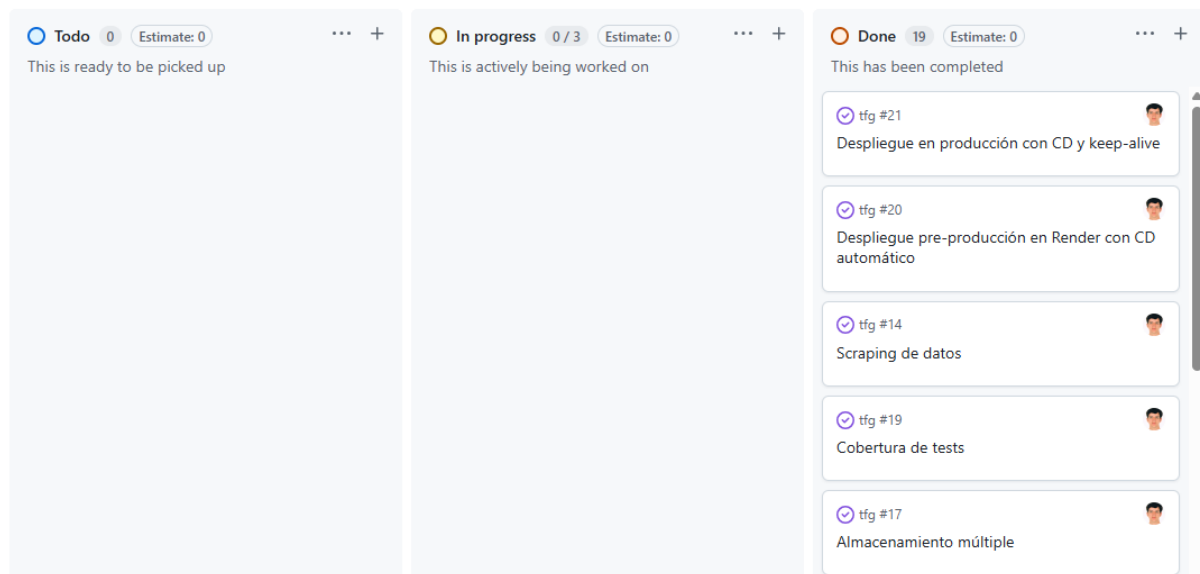


Ilustración 28: Tablero Kanban en GitHub project

11.3 Aseguramiento de la calidad

Más allá de la batería de pruebas descrita en el apartado 9. Pruebas, el proyecto incorpora varios mecanismos automáticos de aseguramiento de la calidad, integrados en el flujo de CI/CD (sección 10.4 Integración y despliegue continuos (CI/CD)):

- **Hooks de pre-commit** (`lint-ci.yml`): aplican comprobaciones de estilo y formateo automático sobre el código en cada push, garantizando un estilo homogéneo.
- **Codacy** (`codacy-ci.yml`): realiza análisis estático de seguridad y calidad del código en cada push, detectando posibles vulnerabilidades y code smells⁴³. La métrica principal de Codacy es el número de issues por cada mil líneas de código (`issues/kLoC`), que permite comparar la densidad de problemas independientemente del tamaño del proyecto. El análisis de la rama `develop` obtiene 5,457 `issues/kLoC`, por debajo del objetivo de calidad de 20 `issues/kLoC` establecido por la herramienta. Los 72 issues detectados se distribuyen entre complejidad de código (24), documentación (23), seguridad (13), buenas prácticas (6), estilo de código (4) y código no utilizado (2). La duplicación de código se sitúa en el 6%.

⁴³ Un code smell es un indicio en el código fuente que, sin ser necesariamente un error, sugiere un problema de diseño o mantenibilidad: nombres poco descriptivos, funciones demasiado largas, duplicación de lógica, etc.



Ilustración 29: Análisis de la rama develop (issues/kLoC)

72 total issues

Some of the issues can be fixed right now.

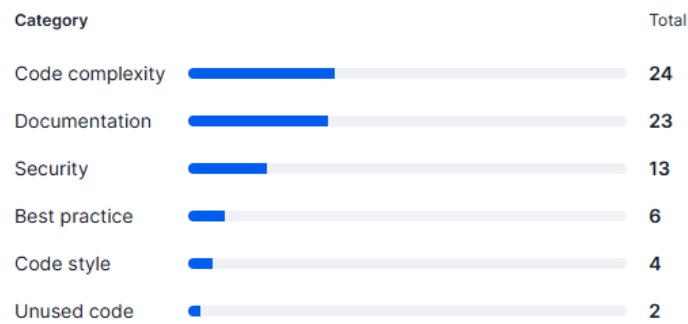


Ilustración 30: Número de issues detectados

12. Uso de la inteligencia artificial

Por transparencia, este capítulo declara el uso que se ha hecho de herramientas de inteligencia artificial generativa como apoyo durante la realización del proyecto. Conviene matizar desde el principio que la IA tratada aquí se ha empleado como una herramienta de trabajo más (al mismo nivel que un IDE, un buscador o la documentación oficial de una tecnología), y que es distinta de Gemini en su papel de componente del producto (la consulta en lenguaje natural, documentada en la sección 8.1.4 Servicio Gemini).

En el desarrollo se ha utilizado Claude Code, un asistente de programación que opera sobre el repositorio desde el editor. Su empleo se ha centrado en tareas mecánicas a partir de objetivos definidos por el alumno: generación de código, esqueletos iniciales de las pruebas, apoyo en refactorizaciones y ayuda en la depuración de errores concretos. En todos los casos el alumno ha revisado, adaptado e integrado el código resultante, descartando lo que no encajaba con la arquitectura del sistema.

En la documentación de la memoria se ha contado con la ayuda de Claude y Google Gemini, empleados como apoyo a la redacción: organización de la estructura de los capítulos, elaboración de borradores a partir de las ideas y notas del alumno, y revisión de estilo y claridad.

De cara a la presentación y defensa del trabajo se ha apoyado igualmente en Claude y Gemini para estructurar el guion, sintetizar los contenidos más relevantes y ayudar en el diseño de las diapositivas, manteniendo el mismo criterio de revisión.

Por último, conviene una reflexión sobre los límites de estas herramientas. La IA acelera el trabajo, pero no sustituye el criterio del autor: a lo largo del proyecto se han detectado y corregido imprecisiones, sugerencias incorrectas e información desactualizada en sus respuestas, lo que ha exigido una validación constante. Por ello, todo el contenido ha sido revisado y, cuando ha sido necesario, corregido por el alumno antes de incorporarse al proyecto.

13. Conclusiones

El resultado del proyecto es un sistema web que lleva el paradigma de las Líneas de Productos a un dominio como es la configuración de equipos informáticos. El núcleo del trabajo (un Modelo de Características formalizado en UVL y razonado en tiempo real con FlamaPy) ha servido de base para construir una experiencia de usuario guiada, en la que las incompatibilidades de hardware se resuelven de forma automática. El sistema cumple su propósito: no solo se limita a validar a posteriori combinaciones introducidas por el usuario, como hacen las herramientas comerciales existentes (PCPartPicker, Coolmod o PcComponentes), sino que también propone configuraciones completas y equilibradas a partir de un perfil de uso y un presupuesto, repartiendo el dinero entre categorías de componentes. Este carácter propositivo, sumado a la interpretación de consultas en lenguaje natural, constituye el principal valor diferencial del trabajo. Todo ello se ha integrado en una arquitectura moderna y mantenible (backend en FastAPI, frontend en React, persistencia en MongoDB y apoyo de IA para consultas en lenguaje natural) respaldada por una batería de pruebas y un flujo de integración continua que aseguran su calidad.

En el plano personal, el desarrollo de este TFG ha supuesto una de las experiencias más completas de toda la carrera. Me ha permitido aplicar de forma práctica y conectar entre sí gran parte de los conocimientos que había estudiado. Pero, por encima de lo técnico, lo más valioso ha sido afrontar un proyecto de principio a fin de forma individual: planificar el trabajo, tomar decisiones de diseño y asumir sus consecuencias, superar los bloqueos y los imprevistos, y aprender tecnologías nuevas por mí mismo cuando el proyecto lo ha requerido. El resultado me deja una satisfacción notable y la confianza de ser capaz de llevar adelante un sistema completo de forma autónoma.

De cara al futuro, el sistema deja abiertas varias líneas de evolución natural. La más inmediata es la ampliación del catálogo a nuevas categorías de producto (periféricos, monitores y otros complementos) y la incorporación de más tiendas al comparador de precios, junto con un mecanismo de actualización automática de precios que mantenga la información siempre vigente. En segundo lugar, la introducción de cuentas de usuario permitiría guardar y recuperar configuraciones, llevar un historial y ofrecer una experiencia personalizada. Una tercera vía pasa por profundizar en el uso de la inteligencia artificial: afinar la interpretación de las consultas en lenguaje natural, ofrecer recomendaciones personalizadas según el historial del usuario y hacer más robusta la clasificación automática del catálogo. Por último, sería deseable una versión móvil nativa y la internacionalización de la aplicación, con soporte para varios idiomas y monedas, que ampliaría notablemente su alcance.

A estas líneas se suman tres funcionalidades que llegaron a plantearse durante la fase de Design Thinking y que finalmente se dejaron fuera del alcance del TFG, no por falta de interés sino por las dificultades que entrañaba su implementación, y que constituyen una continuación especialmente atractiva del trabajo. La primera es la validación del espacio físico, es decir, comprobar automáticamente que los componentes elegidos caben en la caja seleccionada. Se descartó por la dificultad de mantener actualizada una base de datos con las dimensiones de cada pieza. La segunda es el cálculo del consumo energético del conjunto, que permitiría dimensionar con precisión la fuente de alimentación. Se descartó por la misma razón, la dificultad de mantener actualizada una base de datos con el consumo de cada componente. La tercera es el cálculo de cuellos de botella, que detectaría desequilibrios de rendimiento entre componentes. Se descartó por la complejidad de estimar y comparar los rendimientos reales de las distintas piezas. Las tres comparten el hecho de que su obstáculo no es conceptual sino de disponibilidad y mantenimiento de datos fiables, por lo que resolver esa base de información abriría la puerta a incorporarlas y enriquecería de forma significativa las configuraciones propuestas. En conjunto, todas estas propuestas confirman que el sistema desarrollado no es un punto final, sino una base sólida sobre la que seguir construyendo.

14. Bibliografía

- Adobe. (s.f.). *Processor, memory, and GPU recommendations for Premiere Pro*. Recuperado el 2026, de <https://helpx.adobe.com/premiere/desktop/get-started/technical-requirements/processor-memory-and-gpu-recommendations.html>
- AMD. (s.f.). *AMD Socket AM4 chipsets — Specs & CPU compatibility*. Recuperado el 2026, de <https://www.amd.com/en/products/processors/chipsets/am4.html>
- AMD. (s.f.). *AMD Socket AM5 chipset for motherboards*. Recuperado el 2026, de <https://www.amd.com/en/products/processors/chipsets/am5.html>
- Benavides, D., Segura, S., & Ruiz-Cortés, A. (2010). Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 615-636.
- Benavides, D., Sundermann, C., Feichtinger, K., Galindo, J. A., Rabiser, R., & Thüm, T. (2025). UVL: Feature modelling with the Universal Variability Language. *Journal of Systems and Software*, 225. doi:10.1016/j.jss.2024.112326
- Blecker, T., & Abdelkafi, N. (2006). Mass customization: state-of-the-art and challenges. En T. Blecker, & G. Friedrich, *Mass customization: challenges and solutions* (págs. 1-25). Boston, MA: Springer.
- Brown, T. (2008). Design thinking. *Harvard Business Review*, 86(6), 84-92.
- Brown, T. (2009). *Change by design: How design thinking transforms organizations and inspires innovation*. Nueva York: HarperCollins.
- Buchanan, R. (1992). Wicked Problems in Design Thinking. *Design Issues*, 8(2), págs. 5-21.
- BuiltToFrag. (s.f.). *Gaming PC budget guide*. Recuperado el 2026, de <https://builttofrag.com/gaming-pc-budget-guide/>
- Felfernig, A., Hotz, L., Bagley, C., & Tiihonen, J. (2014). *Knowledge-based configuration: From research to business cases*. Waltham, MA: Newnes.
- Galindo, J. A., Horcas, J.-M., Felfernig, A., Fernandez-Amoros, D., & Benavides, D. (2023). FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models. *Proceedings of the 27th ACM International Systems and Software Product Line Conference*, (págs. 16-19). doi:10.1145/3579028.3609008
- Google. (2026). *Gemini API documentation*. Obtenido de Google AI for Developers: <https://ai.google.dev/>
- Intel Corporation. (1997). *ATX Specification (Version 2.01)*. Obtenido de https://www.bitsavers.org/pdf/intel/ATX/ATX_Specification_2.01_199702.PDF
- Intel Corporation. (s.f.). *Intel Core Ultra 9 processor 285K — Product specifications*. Recuperado el 2026, de <https://www.intel.com/content/www/us/en/products/sku/241060/...specifications.html>
- Junta de Andalucía. (2023). *INSTRUCCIÓN SOBRE PERFILES, PRECIOS DE REFERENCIA Y DESGLOSE DE COSTES EN CONTRATOS TIC*.
- NVIDIA. (s.f.). *GeForce RTX 4090 graphics cards*. Recuperado el 2026, de <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>
- PerfGamer. (s.f.). *Gaming PC budget allocation: How to split your money the smart way*. Recuperado el 2026, de <https://www.perfgamer.com/gaming-pc-budget-allocation-how-to-split-your-money-the-smart-way/>
- Project Management Institute. (2017). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)* (6 ed.). Newtown Square, PA: Project Management Institute.
- Schwaber, K., & Sutherland, J. (Noviembre de 2020). *La Guía Scrum: La Guía Definitiva de Scrum: Las Reglas del Juego*. Obtenido de Scrum.org: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-European.pdf>
- Sommerville, I. (2011). *Ingeniería de Software* (9 ed.). Pearson.
- Sundermann, C., Feichtinger, K., Engelhardt, D., Rabiser, R., & Thüm, T. (2021). Yet another textual variability language? A community effort towards a unified language. *Proceedings of the 25th ACM International Systems and Software Product Line Conference*.